

Implementation Plan — Modular friction/nucleation/Riemann for the SAFS spatial driver, with byte-exact TPV205/104/102/31 regression and a rate-and-state SAFS option

SEAS-MFEM · fullelasticity_dev / parts_dev

2026-05-25 (rev 6 — sixth pass; addresses REVIEW.md round-2 R-009...R-011)

Contents

1	§0 Review reconciliation (rev 6 — REVIEW.md R-001...R-011)	4
2	Overview	6
3	Resolved decisions (was: open questions)	7
3.1	D1 — Fault-basis source → KEEP safes's; add a planar-TPV correctness test	7
3.2	D2 — CFL safety → adopt opt-in cfl_safety, keep BOTH options	7
3.3	D3 — Stress-sign convention → two complementary moves	7
3.4	D-rename — general method name → DO the rename + consistency check	9
4	Byte-exactness contract (the non-negotiable)	10
5	Git provenance, reference, and existing scaffolding	11
5.1	RS scaffolding already in safs (the SAFS+RS head start)	12
6	Module architecture (the “parts”)	13
6.1	friction_laws part (unified, method-oriented)	13
6.2	nucleation_methods part	15
6.3	riemann_flux part (homogeneous scalar vs heterogeneous matrix)	16
6.4	Shared-rank correctness constraints (must hold for every fault-flux / exchange change)	17
6.5	What leaves vs stays in the driver	17
7	Code graph — full structure	18
7.1	G1 — Code map (files by role)	18
7.2	G2 — Runtime wiring (setup: config -> objects)	19
7.3	G3 — Per-macro-step call & data flow (time loop)	19
7.4	G4 — Stress-source dispatch (both paths: positive input = right-lateral on-fault) . .	20
7.5	G5 — Riemann-flux dispatch (scalar vs matrix; fault flux unaffected)	20

8	Master phase / file table	21
9	Phases	23
9.1	Phase 1 — RS primitives » (aging-iterator hooks + equilibrium- ψ seed + build) . .	23
9.2	Phase 2 — Friction strategy interface » (IFrictionIterator + adapters + factory) .	27
9.3	Phase 3 — SAFS + rate-and-state run » (driver RS branch + D3.1 + config) — THE NEAR-FUTURE GOAL	30
9.4	Phase 4 — Name generalization (D-rename)	35
9.5	Phase 5 — Unified method-oriented iterator (replaces the Phase-2 adapters)	36
9.6	Phase 6 — TPV/material/flux config schema + D3.2 fault-local stress	37
9.7	Phase 7 — nucleation_methods module	40
9.8	Phase 8 — TPV205/102/104 benchmarks (configs, meshes, jobs, regression)	42
9.9	Phase 9 — Heterogeneous (matrix) Riemann solver option (PORT from hrs-ref) . .	44
9.10	Phase 10 — TPV31 (depth-heterogeneous LSW) verification case (PORT from hrs-ref)	47
9.11	Phase 11 — Depth-varying rate-and-state $a(z)$ / $b(z)$ (depth-profile input)	48
9.12	Phase 12 — Absorbing boundaries (PML) for the SAFS dynamic-rupture run	56
10	Testing strategy	61
11	Risk assessment	62
11.1	Phase 13 — Split scalar / matrix into separate WaveOperator classes (kill the flux_placeholder leak class)	66
11.2	Phase 14 — Explicit Runge–Kutta (RK45) time-integrator option for mixed (central) flux	74
12	Appendix	81

STATUS: DRAFT — APPROVAL REQUIRED. NO IMPLEMENTATION until this pass is signed off. This is rev 6 of the TPV-via-spatial_dyn_driver plan. It keeps the SAFS structure, wires the TPV benchmarks, and adds the missing functionalities as clean reusable modules.

Rev-6 deltas (this pass — the round-2 re-audit REVIEW.md R-009/R-010/R-011): - **R-009 (MODERATE→CRITICAL if $v_0 \neq 1e-6$):** the force solve hardcodes FrictionSolver::V0 = 1e-6 while the RS seed + aging law use config blk.V_0_default — a silent dual-source. Phase 1 SeedEquilibriumPsi_RS now asserts they agree (defensive copy in the factory); the deliverable keeps $V_0 = 1e-6$. Proper fix (thread config V0 into the solver, per the no-hardcoded-numbers rule) is a documented follow-up gated out of the byte-exact TPV oracle path. - **R-010 (MODERATE — would not compile):** Phase-2 req 5 wrongly said the body “already calls iterator.Advance(…).” It calls Advance-WithSubStepStates(…) at driver :439, and also GetDeltaT()/GetTimeWeights() at :384/:385/:404. Rev 6 renames the :439 call to Advance(…) **and** adds the two getters to IFrictionIterator (the reviewer’s narrower fix would still not compile). - **R-011 (LOW/POSSIBLE):** ResolveRateState:1070 hard-aborts unless $a < b$, forbidding the velocity-strengthening border a bounded tpv102-like rupture needs. Rev 6 relaxes it to allow $a > b$ (keep $a, b > 0$). Optional for the fully-VW first smoke; required before a bounded run. - **Cleared (no change):** Makefile inline-link sufficiency, AgingLawPsi

ctor order, driver RS-resolver scoping — all verified correct by the reviewer (recorded in §0).

Rev-5 deltas (carried — addresses the plan-level audit in REVIEW.md round 1): every finding below is now closed in the priority phases (full mapping in §0 **Review reconciliation**): - **R-001 (CRITICAL):** SeedEquilibriumPsi_RS pinned to d.eta_s (= rs.eta(i) = $0.5 \cdot \sqrt{\mu\rho}$ for homogeneous SAFS) — **never** rs.eta_s (no such member); added the missing #include `"../friction/dieterich_ruina.hpp"` and the in-body `V_init>0` guard. - **R-002 (CRITICAL):** Phase-3 resolver branch now **gates** the ungated `MFEM_VERIFY(cfg.slip_weakening.has_value())` at driver :1146 under `is_lsw`, and declares both `lsw` and `rs` at **outer scope** so the DOF-init at :1220 still sees them. - **R-003 (MODERATE):** ResolveRateState is now called with a **zero** PorePressureSpec{} (`geom.sigma_n_per_dof()` is already effective $\sigma_n - P_p$) — no double pore-pressure subtraction. - **R-004 (MODERATE):** the shared-fault RS ψ accuracy gate is reframed as a **temporal** (interior-vs-shared ψ) check plus the existing cross-rank bit-identity check; `np=2` test added. - **R-005 (MODERATE):** Phase-3 branches the ParaView “state” channel to write d.psi for RS (the LSW `LSWFrictionCoefficient_TPV205` path is 0/NaN for RS). - **R-006 (MODERATE):** ResolveRateState now **rejects** per-DOF `b/V_0/f_0` spatial overrides (the aging law + DOFData are scalar in those) — making the plan’s claim true. - **R-007 (LOW):** D3.1 **kept in Phase 3** per the user’s standing decision, but hardened — factory negation + **all** config flips (`Dc2/Dc8/Dc10` + RS) land in **one atomic commit**, and the golden sign test is **parametrized over every flipped config**. - **R-008 (LOW):** the `tpv102` parity AC now **pins NewtonRaphsonStable on both overloads** (production uses Brent); a separate Brent test checks finiteness/monotonicity only.

Rev-4 deltas (carried — the prioritization that rev 5 hardens): 1. **SAFS + rate-and-state is now the near-future goal, delivered in Phases 1–3 ».** A source audit (§4.1) shows the **entire RS config/resolver/setup/kernel/iterator/wave-op dispatch already exists** in safs — SAFS+RS is a *driver-wiring* job. Phases 1–3 close the small gap (a `Tpv102` nucleation-callback overload, an equilibrium- ψ seed, an `IFrictionIterator` strategy with adapters over the **existing proven** iterators, and the driver RS branch); everything else (unification, TPV regression, matrix Riemann, TPV31) follows in Phases 4–10. 2. **D3.1 unchanged this pass** (the user confirmed “stick with current plan”): the `constant_tensor` input is right-lateral-positive via a factory-level negation that leaves the stored tensor and all on-fault quantities bit-identical; it now lands in **Phase 3**. 3. **Expanded code + unit-test detail** for the prioritized phases — concrete signatures (`SeedEquilibriumPsi_RS`, the `Tpv102` overload, `IFrictionIterator/adapters/factory`, the RS driver branch) and bit-for-bit/residual test specs.

Carried from rev 3 (still in scope, later phases): 1. **Friction iterators unified, method-oriented (§5.1).** The three benchmark-named sub-step iterators collapse into one templated `RateStateSubStepIterator<StatePolicy>` (aging / slip-law-SRW differ by *one* policy line) plus a separate `LinearSlipWeakeningIterator`, sharing one `RunSubSteps_` skeleton — byte-exact to the standalone drivers, ~250 duplicated lines removed. 2. **D3 rewritten (§2-D3).** The fault-local TPV option is kept; *additionally* the regional `constant_tensor` projection now takes **right-lateral-POSITIVE** shear input and

yields right-lateral-positive on-fault shear, with a full global $\leftrightarrow$ local derivation and a **bit-identity** proof (the stored Cauchy tensor and all on-fault quantities are unchanged; the no-flip projection rule R-501 is *not* reverted). 3. **Heterogeneous (matrix) Riemann solver enabled as a first-class option (§5.3, Phase 9)** alongside the homogeneous (scalar) solver; mixed-flux remains scalar-only. **TPV31** (depth-heterogeneous material) is added as the matrix-path verification case (Phase 10). 4. **Proposed layout simplified (§5, §6)** — design only; no real-code edits in this pass. 5. **Shared-rank correctness constraints made explicit (§5.4)** and threaded through every phase that touches a fault flux or a cross-rank exchange.

1 §0 Review reconciliation (rev 6 – REVIEW.md R-001...R-011)

Every finding from the plan-level audit (REVIEW.md, rounds 1–2, 2026-05-25) was checked against live source (greps in the notes below) and is closed in the phase shown. Severity and “blocks the deliverable?” are the reviewer’s. **Round 2 added R-009/R-010/R-011 and cleared three round-1 suspicions** (see the end of this section); R-001...R-008 carry over from rev 5.

ID	Sev	Where	Resolution in this rev	Phase	Test
R-001	CRIT	SeedEquilibriumPsi	Use <code>rs.eta_s (= rs.eta(i)); not rs.eta_s (absent)</code> . Add <code>#include dieterich_ruina.hpp</code> ; add in-body <code>V_init>0 abort</code> .	1	seed_equilibrium_psi_ (extended)
R-002	CRIT	driver :1146	Gate <code>slip_weakening.has_value()</code> under <code>is_lsw</code> ; declare <code>lsw+rs</code> at outer scope; RS branch asserts <code>rate_state.has_value()</code> .	3	rate_state_config_no_ (dry-run)
R-003	MOD	ResolveRateState	Parse <code>PorePressureSpec{}</code> (input already effective $\sigma_n - P_p$); documents the <code>sigma_n_total_per_dof</code> trap.	3	resolve_rate_state_no_
R-004	MOD	shared-fault ψ	Reframe gate as temporal (interior-vs-shared ψ) + cross-rank bit-identity; bounded (mirrors LSW slip path); <code>np>1</code> RS stays “open until validated”.	3	shared_fault_rs_psi_co
R-005	MOD	ParaView state writer :1809	Branch: RS writes <code>d.psi</code> ; LSW path byte-unchanged.	3	paraview_state_channe
R-006	MOD	resolve_rs_i	Reject <code>per DOF/DOF/f_0</code> (aging law + DOFData scalar) — makes the plan’s claim true.	3	resolve_rate_state_re

ID	Sev	Where	Resolution in this rev	Phase	Test
R-007	LOW	D3.1 multi-config flip	Keep D3.1 in Phase 3 (user’s standing decision) but land factory negation + all flips (Dc2/8/10 + RS) atomically ; golden test parametrized over every flipped config.	3	constant_tensor_sign (parametrized)
R-008	LOW	tpv102 parity AC	Pin NewtonRaphsonStable on both overloads for EXPECT_EQ; separate Brent finiteness test.	1	tpv102_nuc_callback_pa (clari-fied)
R-009	MOD→CRF if V0≠1e-6	FictionSolver vs config V_0	Add: V_0 == FrictionSolver::V0 guard in SeedEquilibriumPsi_RS (+ defensive in factory); deliverable keeps V_0=1e-6. Proper fix (thread config V0 into the solver) is a gated follow-up.	1	seed_equilibrium_psi_r (V0 case)
R-010	MOD (build)	Phase-2 retype vs driver :439	Rename the :439 AdvanceWithSubStepStates(...) call → Advance(...); and add GetDeltaT/GetTimeWeights to IFrictionIterator (body calls them at :384/:385/:404).	2	advance_interface_comp
R-011	LOW (POSSIBLE)	ResolveRateState	Relax a < b → allow a > b (velocity-strengthening; keep a, b > 0). Optional for the fully-VW first smoke; needed for a bounded tpv102-like rupture.	3	resolve_rate_state_al

Source-grounding notes (verified 2026-05-25): RateStatePerDOFParams member list is Vector a,b,Dc,V_init,f_0,V_0,eta,sigma_n_eff (spatial_friction.hpp:295) — **eta, no eta_s**; DOFData.eta_s is set at spatial_setup.hpp:82,128. Un-gated guard confirmed at spatial_dyn_driver.cpp:1146; ResolveSlipWeakening at :1151; InitializeFaultDOFs_Spatial at :1220. PorePressureSpec is a struct (spatial_friction.hpp:72); ResolveRateState takes const PorePressureSpec& pp (:313/:325). ParaView LSW writer at spatial_dyn_driver.cpp:1809. resolve_rs_impl applies per-DOF b/f_0/V_0 with **no rejection** at spatial_friction.cpp:1014/1017/1018. tpv102 plain overload defaults method = NewtonRaphsonStable (tpv102_substep_iterator.hpp:112,127). **Round 2:** FrictionSolver::V0 = 1e-6 is static constexpr (friction_solver.hpp:60), consumed by the force solve (fault_face_flux.cpp:224); AdvanceADERWithSubStep_Spatial’s body calls iterator.GetDeltaT()/GetTimeWeights()/SetSubSteps()/AdvanceWithSubStepStates() at :384/:385/:404/:439 (so the interface needs the two getters **and** the :439 rename); the a < b validator is spatial_friction.cpp:1070. Both Tpv205/Tpv102 iterators expose GetDeltaT/GetTimeWeights (tpv205...hpp:138-139, tpv102...hpp:130-133).

The reviewer’s R-001 quoted rs.eta_s; the live .md already used d.eta_s — rev 5 keeps d.eta_s (the reviewer’s “more robust” choice, exactly what FaultFaceFlux consumes) and adds an explicit

“**never rs.eta_s**” guard-comment so a fix agent cannot drift to the bad member. **On R-010:** the reviewer flagged only the :439 rename, assuming GetDeltaT/GetTimeWeights were already on the interface; they were **not** in the rev-5 draft, so rev 6 adds them too — a deliberate over-fix to make the retype actually compile.

Round-1 suspicions the reviewer cleared (no change needed — recorded so they are not re-litigated): (a) linking only TPV102_SUBSTEP_ITERATOR_OBJ is sufficient (ApplyNucleationIncremental_TPV102 is inline); (b) AgingLawPsi(b, V0, f0) ctor order matches the Phase-2 adapter — no arg swap; (c) dof_to_elem/dof_to_attr/dof_coords_3d/ material/pmesh are all populated and in scope before the RS resolver call.

2 Overview

Today the SAFS dynamic-rupture driver miniapps/seas/drivers/spatial_dyn_driver.cpp runs **only** linear slip weakening (LSW) and **rejects** [meta].law = "rate_state". This plan keeps that driver and its SAFS-specific fixes as the backbone and:

1. **Factors friction laws, nucleation methods, and Riemann solvers into reusable modules** — friction_laws = {LSW, rate-and-state aging, rate-and-state slip-law strong rate-weakening (SRW)}, nucleation_methods = {static/spontaneous overstress, gradual overstress (Gaussian), gradual overstress compact-circular, instantaneous circular overstress}, riemann_flux = {homogeneous scalar Godunov, heterogeneous matrix (bimaterial)} — sharing the existing ADER, FaultBasis, impedance, and per-DOF resolver infrastructure.
2. **Wires four SCEC benchmarks** TPV205 (LSW), TPV102 (RS aging), TPV104 (RS SRW), and TPV31 (LSW on a depth-heterogeneous medium → matrix Riemann) through that driver via per-case TOMLs, with physics **byte-exact** against the verified standalone drivers drivers/tpv{205,104,102}_driver.cpp (and, for TPV31, the hrs-ref implementation).
3. **Adds a rate-and-state SAFS run:** the existing jobs/safs/spatial_dyn_resolution_Dc2_8N_400r_dev_2hr_s problem, but with user-defined RS parameters from a TOML (aging-law, “tpv102-type”).

Central facts (verified). (a) The three primary TPVs (205/102/104) are homogeneous, so they run at [numerics] interior_flux = "scalar" — the existing safs scalar WaveOperator path, untouched. (b) **TPV31 is depth-heterogeneous**, so it requires the **matrix (bimaterial) Riemann path:** in safs the gating member owned_flux_pool_ and heterogeneous_material.{hpp,cpp} (Constant/Coefficient modes) already exist, but the BimaterialFlux class, the per-face dispatch, and DepthProfile1D are **only in hrs-ref** and must be ported (Phases 9–10). The two solvers become an explicit interior_flux ∈ {scalar, matrix} choice; **mixed-flux is valid only with scalar** (homogeneous). (c) The **entire friction + state-evolution + native-nucleation kernel is already byte-identical on safs and the reference branch** — so there is **no friction physics to re-derive; that work is wiring, unification, and modularization.** The new physics this pass *adds* is the bimaterial Riemann flux and the depth-profile material, both ported verbatim from hrs-ref.

3 Resolved decisions (was: open questions)

3.1 D1 – Fault-basis source → KEEP safes’s; add a planar-TPV correctness test

Keep safes’s driver-local per-QP FaultBasis (Compute + AppendSharedFaces + R-005 ComputeQPBasis/ComputeQPBasisShared, driver ~:1033–:1065) and the $\text{ref_normal} = (0, -1, 0)$ convention. Do **not** adopt the reference branch’s `wave.GetFaultBasis()` reuse. Layer in only the empty-fault-rank guard (`GetFaultBasis()` null-check + `static const FaultBasis empty_fault_basis_{}.`). **Verification (new):** a unit test asserts that for the planar $y=0$ TPV fault the driver-local per-QP basis yields the canonical frame $\text{dip} = (0, 0, -1)$, $\text{strike} = (+1, 0, 0)$ at every fault QP (Phase 8). This is the “test if planar TPV is correct” the user asked for.

3.2 D2 – CFL safety → adopt opt-in `cfl_safety`, keep BOTH options

Replace safes’s unconditional `cfl /= 3*(2p+1)` with the reference branch’s opt-in `[numerics] cfl_safety ∈ {raw, dg}` (`dg` applies the factor). Set `cfl_safety = "dg"` in **all** configs (TPV and SAFS), which is byte-identical to safes’s present behaviour for every existing config while exposing `raw` for experiments.

3.3 D3 – Stress-sign convention → two complementary moves

The user’s final direction (this pass) is two-part: (a) the fault-local pre-stress input is correct *as an option* for the TPV cases — keep it; (b) for the **regional projection** (`constant_tensor`), change the input convention so a **positive (right-lateral) input maps to a positive (right-lateral) on-fault** shear, prove the on-fault quantities are unchanged, prove nothing else is affected, and derive the $\text{global} \leftrightarrow \text{local}$ transform with equations. The derivation and proof are in §2-D3.1; the (kept) fault-local option is in §2-D3.2.

D3.1 – Regional projection: right-lateral-POSITIVE input convention (new)

Setup (the $\text{global} \leftrightarrow \text{local}$ transformation). Let S be the regional Cauchy stress tensor (3×3 symmetric) in the SEAS **compression-positive** convention. The fault-local orthonormal frame for the canonical $y=0$ vertical strike-slip fault (from $\text{ref_normal}=(0, -1, 0)$, $\text{up}=(0, 0, 1)$; confirmed by the D1 planar-basis test) is

```
n = ( 0, -1,  0)  (fault normal)
t1 = ( 0,  0, -1)  (dip)
t2 = ( 1,  0,  0)  (strike)
```

The on-fault traction is $T = S \cdot n$ (i.e. $T_r = \sum_c S_{rc} n_c$), resolved in the local frame by the **safs no-flip rule** (`fault/fault_geometry_safs_templated.inl:102–104`, mirrored in `io/field_coefficient.cpp:482–487`):

```
 $\sigma_n = n \cdot T = n^T S n$ 
 $\tau_{\text{dip}}(\tau_1) = t_1 \cdot T = t_1^T S n$ 
 $\tau_{\text{str}}(\tau_2) = t_2 \cdot T = t_2^T S n$ 
```

Substituting the canonical frame (n has only a $-y$ entry, so $T = S \cdot n = (-S_{xy}, -S_{yy}, -S_{yz})$):

```
 $\sigma_n = n \cdot T = (-1)(-S_{yy}) = + S_{yy} = + \sigma_{yy}$   (compression positive)
 $\tau_{\text{dip}} = t_1 \cdot T = (-1)(-S_{yz}) = + S_{yz} = + \sigma_{yz}$   (coefficient +1)
```

$$\tau_{\text{strike}} = t_2 \cdot T = (+1)(-S_{xy}) = -S_{xy} = -\sigma_{xy} \quad (\text{coeff } -1 \leftarrow \text{inversion})$$

This $\tau_{\text{strike}} = -\sigma_{xy}$ is the rotation artifact, not a bug. Physical check (displacement, not traction-direction): a simple-shear field $u_x = \gamma y$ ($\gamma > 0$) has strain $\epsilon_{xy} = \gamma/2 > 0$, i.e. tension-positive $\sigma_{xy}^{\text{tens}} > 0$; it moves the $+y$ block in $+x$ relative to the $-y$ block — **right-lateral**. In the compression-positive store $S_{xy} = -\sigma_{xy}^{\text{tens}}$, so a **right-lateral** prestress means $S_{xy} < 0$, and the projection gives $\tau_{\text{strike}} = -S_{xy} > 0$. That is exactly why SAFS today writes $\text{sigma_xy_pa} = -9.89\text{e6}$ to obtain $\tau_{\text{strike}} = +9.89\text{e6}$.

The change. Define the TOML input field sigma_xy_pa to be **right-lateral-positive** (equivalently, the tension-positive Cauchy shear $\sigma_{xy}^{\text{tens}}$), while normal-stress fields stay compression-positive. The TOML→source factory absorbs the rotation’s -1 for the strike off-diagonal **at construction time**:

```
S_xy(stored) = - sigma_xy_pa      # strike off-diag: negate at build
S_yy(stored) = + sigma_yy_pa      # normals: compression-positive (as-is)
S_yz(stored) = + sigma_yz_pa      # dip shear:  $\tau_{\text{dip}} = +\sigma_{yz}$ , sign matches
```

(For the fixed canonical $y=0$ fault used by **every** config, the only off-diagonal whose rotation coefficient is -1 is xy ; yz/xz keep coefficient $+1$, so only sigma_xy_pa is negated. The rule is “negate the components the rotation inverts,” stated per-frame.) The unchanged no-flip projection then yields the user’s requirement:

```
 $\tau_{\text{strike}} = -S_{xy}(\text{stored}) = -(-\text{sigma\_xy\_pa}) = +\text{sigma\_xy\_pa}$  ✓ in==on-fault
 $\sigma_n = +S_{yy}(\text{stored}) = +\text{sigma\_yy\_pa}$  (unchanged)
```

Bit-identity proof (same on-fault quantities; nothing else affected). - *Tensor unchanged.* Today: $\text{sigma_xy_pa} = -9.89\text{e6}$, builder stores $S_{xy} = -9.89\text{e6}$. After: $\text{sigma_xy_pa} = +9.89\text{e6}$, builder stores $S_{xy} = -(+9.89\text{e6}) = -9.89\text{e6}$. The **stored Cauchy tensor S is bit-identical**, hence every on-fault scalar (τ_{strike} , τ_{dip} , σ_n , and the derived per-DOF $a/\eta/Dc/V_{\text{init}}$) is bit-identical. - *Projection rule unchanged.* We do **not** touch `fault_geometry_safs_templated.inl` or `field_coefficient.cpp`; the no-flip rule (R-501/R-502, `PLAN_onfaultstress.md:142–164`) is preserved. **Nothing is reverted** — consistent with the CLAUDE.md “don’t revert a fix” rule. The only code change is one sign at the TOML→ConstantTensorStressSource call. - *No other consumer.* **Verified (Agent-1 audit):** the regional StressSource is consumed *only* by `ComputeParams/ProjectFaultPreStress` to seed fault DOFData (`spatial_dyn_driver.cpp:1123–1141`); it is **never** applied as a bulk initial condition and the wave operator never receives it. Negating the input therefore provably cannot affect any bulk/elastic calculation. - *Atomic config update.* The parser negation and the **sign flip of sigma_xy_pa in all SAFS production configs (Dc2/Dc8/Dc10) land together**, so net on-fault stress is unchanged; a golden-value test locks it (below). - *Scope of the convention.* This applies to the hand-authored `constant_tensor` TOML path only. The machine-written **HDF5 sidecar** stays compression-positive (the resolver writes it); `ProjectFaultPreStress` and **T_65_5 (templated == sidecar, fixture $\text{sigma_xy}=0$)** stay green. A documented note warns that expressing the *same* physical state both as a `constant_tensor` TOML and as an HDF5 sidecar now needs opposite sigma_xy signs.

Consistency check before production (new test) — `test_constant_tensor_sign.cpp`: for the SAFS Dc2 stress block (post-flip $\text{sigma_xy_pa} = +9.89\text{e6}$), build the ConstantTensorStressSource via the **factory** (so the negation is exercised), project onto a canonical-frame fault DOF, and assert

$\tau_{\text{strike}} == +9.89\text{e6}$ (right-lateral positive), $\sigma_n == +\sigma_{yy}$, $\tau_{\text{dip}} == +\sigma_{yz}$, **bit-identical to the pre-change golden values** captured from the current Dc2 config. Precondition assert: `dip==(0,0,-1), strike==(+1,0,0)`.

D3.2 – Fault-local pre-stress input (kept as an option for the TPV cases)

For TPV205/102/104 the pre-stress is naturally expressed in **fault-local** terms; the user confirmed this is “correct for TPV to use fault-local, this is an option.” Keep it:

```
[stress]
kind          = "fault_local_prestress"
tau_strike_pa = 75.0e6      # right-lateral + (TPV102; 205:70e6 104:40e6)
tau_dip_pa    = 0.0
sigma_n_pa    = 120.0e6     # compression POSITIVE
```

- **The source seeds DOFData directly** — `tau1_0(dip)=tau_dip_pa, tau2_0(strike)=tau_strike_pa, sigma_n0=sigma_n_pa - P_p` — **bypassing the Cauchy projection entirely**. This is exactly how the native drivers seed (`dynamic/tpv102_setup.hpp:76-78: d.tau2_0 = TPV102Params::tau_ini`), so it is **byte-exact to native with no projection round-trip**. No sign flip anywhere; positive `tau_strike_pa` $\Rightarrow$ positive `tau2_0` $\Rightarrow$ native right-lateral. (*The user confirmed: “we don’t need stress projection for TPV benchmark.”*)
- **Note the harmony with D3.1:** both the kept fault-local input *and* the new `constant_tensor` convention now make **positive == right-lateral on-fault**, so the two stress paths agree on sign at the TOML surface; they differ only in mechanism (direct seed vs Cauchy projection). A TPV case may use either; the SAFS regional case uses `constant_tensor`.
- **TPV205** carries positive `tau_strike` **patches** (background +70 MPa; central +81.6 MPa, left +78 MPa, right +62 MPa) instead of Cauchy `sigma_xy` patches. The Cauchy `ConstantTensorWithPatches` source is therefore **not** required for the three primary cases.
- **Implementation note:** `FaultGeometry::ComputeParams` gains a fault-local seeding overload `ComputeParamsFaultLocal(tau_strike, tau_dip, sigma_n, P_p)` that assigns the three per-DOF scalars directly; seeding is frame-agnostic (the friction solve still uses the canonical frame for slip *directions* — guarded by the D1 planar-basis test).
- **Consistency check (new test):** `test_tpv_toml_stress_sign.cpp` builds each TPV `FaultLocalPrestressSource` from its TOML and asserts `tau2_0 == +tau_strike_pa == +tau_ini_native, tau1_0 == +tau_dip_pa == 0, sigma_n0 == sigma_n_pa - P_p` (native refs: TPV205 70e6 / patch 81.6e6, TPV102 75e6, TPV104 40e6; σ_n 120e6).

3.4 D-rename – general method name $\rightarrow$ DO the rename + consistency check

Rename `FaultGeometry::ComputeSAFSParams` $\rightarrow$ `ComputeParams`, `HasSAFSParams` $\rightarrow$ `HasParams`, member `safs_params_computed_` $\rightarrow$ `params_computed_` across `fault/fault_geometry.hpp`, `fault/fault_geometry_safs.inl`, `fault/rate_state_fault.hpp`, `config/seas_config.hpp`, `config/seas_config_bridge.hpp`, `spatial/code/spatial_stress.{hpp,cpp}`, and `tests/unit/test_compute_safs_params.cpp`. The fault kind string stays informational (no code branches on it). **Consistency check before production:** after the rename the full unit suite (esp. `test_compute_safs_params`) plus the new `test_tpv_toml_stress_sign` must be green, and a symbol grep must show zero remaining `ComputeSAFSParams/HasSAFSParams` references (Phase 1 acceptance).

4 Byte-exactness contract (the non-negotiable)

The verified standalone drivers `drivers/tpv{205,104,102}_driver.cpp` are the oracle. **No function is approximated.** Verified by diff (`/tmp/branchcmp/pass2_exactness.md`): the kernel files `friction/{dieterich_ruina,state_evolution,slip_law_srw_psi, friction_coeff_stable}.hpp`, all `config/tpv*_params.hpp`, all `dynamic/tpv*_setup.hpp`, and `dynamic/tpv10{2,4}_nucleation.hpp` are **byte-identical on safes and the reference branch**. Therefore the modules below *call* these existing functions; they do not reimplement them.

Case	Friction-law solve (canonical fn)	State evolution	Nucleation (canonical fn)	Unified-iterator mapping (§5.1)
TPV205 (LSW)	<code>SolveLSW_TPV205</code> + <code>LSWFrictionCoefficient_TPV205</code> (<code>dynamic/tpv205_friction.hpp:54,94</code>) barrier sentinel $\mu_s=1e4$	<code>none</code>	static IC — <code>ComputeTau2_0_TPV205</code> (<code>config/tpv205_params205sub150p1B9</code>) seed <code>tau2_0/lsw_mu_s</code> ; spatial uses fault-local prestress (D3.2) or <code>constant_tensor</code> (D3.1), seeded directly	<code>LinearSlipWeakeningIterator</code> <code>RateStateSubStepIterator::Step0</code> closed-form body verbatim (<code>SolveLSW_TPV205</code> , callback nucleation)
TPV102 (RS aging)	<code>DieterichRuina::SolveSlipRatePsi</code> (Brent, <code>friction/dieterich_ruina.hpp:322,470</code>)	<code>SlipLawRatePsi</code> <code>UpdateStateAnalytic</code> (<code>friction/tpv102_state.hpp:177,304</code>)	<code>CompactShell</code> <code>F(r)=exp(r²/(r²-R²))</code> <code>ApplyNucleationIndicator</code> (<code>dynamic/tpv102_nucleation.hpp:100</code>) $\equiv$ spatial <code>ResolveGradualOverturn</code> (<code>math-identical</code> , proven)	<code>RateStateSubStepIterator<RateStateAnalytic></code> policy line is <code>UpdateStateAnalytic(...)</code> <code>local id</code> sequence <code>IdenticalCompactCircular/Apply</code> <code>Tpv102SubStepIterator</code>
TPV104 (RS SRW)	same Brent chain	<code>SlipLawSRWPsi</code> + <code>UpdateStateAnalytic</code> (<code>friction/slip_law_srw_psi.hpp:92,100</code>) per-QP V_w	same compact <code>SlipLawSRW</code> <code>UpdateStateAnalytic</code> (<code>friction/slip_law_srw_psi.hpp:92,100</code>)	<code>RateStateSubStepIterator<RateStateAnalyticSlipLawSRW></code> — policy line is <code>UpdateStateAnalyticSlipLawSRW</code> <code>V_w[i], ...</code> ; V_w span owned by the iterator

Case	Friction-law solve (canonical fn)	State evolution	Nucleation (canonical fn)	Unified-iterator mapping (§5.1)
TPV31 (LSW, het)	SolveLSW chain + depth-linear cohesion C0(depth)	none	instantaneous circular cosine-tapered overstress (hrs-ref tpv31.toml:45– 50)	LinearSlipWeakeningIterator (same as TPV205) on the matrix Riemann path (§5.3); nucleation = InstantaneousOverstressCircular

Shared infrastructure (must NOT be duplicated by any per-law module): ADER predictor `WaveOperator::ComputeADERSubStepStates + EvaluateBulkAtFaultQPsCanonical`; the driver helper `AdvanceADERWithSubStep_Spatial`; `FaultFaceFlux::`{`ComputeTrialTraction`, `ComputeStageState`, `BuildImposedState`, `WriteBackState`}; `FaultBasis::ComputeOrientedFrame`; per-DOF η/Z impedances; `SpatialFrictionResolver`; and the new `SubStepIteratorBase::RunSubSteps_` skeleton (§5.1) shared by every iterator.

Deviations to fix (wiring/unification + the bimaterial/material port; zero friction-physics reimplementation): 1. **Unify the iterators (§5.1).** Create one `dynamic/friction_substep_iterator.{hpp,cpp}` holding `SubStepIteratorBase`, `RateStateSubStepIterator<StatePolicy>`, and `LinearSlipWeakeningIterator`, plus `friction/state_policies.hpp`. The unified RS iterator has the per-sub-step `nuc_callback` built in (no need to retrofit the old iterators). The standalone `tpv{205,102,104}_substep_iterator.{hpp,cpp}` are **left untouched as the byte-exact oracle**; parity tests prove the unified iterator reproduces them bit-for-bit. Retiring the old iterators is a deferred follow-up. 2. Add the additive `GradualOverstressCompactCircular` and `InstantaneousOverstressCircular` resolver/applicators to `dynamic/spatial_nucleation.{hpp,cpp}` (additive, 0 deletions). 3. D3 (both moves): add `FaultLocalPrestressSource + ComputeParamsFaultLocal` (D3.2, direct seed, no projection), **and** the right-lateral-positive `constant_tensor` factory convention (D3.1, one sign at construction; stored tensor bit-identical; projection rule untouched). 4. TPV104 V_w boxcar reproduced by the `[friction.rate_state]` TOML (config-level). 5. **Port the matrix Riemann path** (`dynamic/godunov_flux_bimaterial.{hpp,cpp}` + the `owned_flux_pool_dispatch + ExchangeBiMaterialNeighbours_`) and the **DepthProfile1D material** from hrs-ref (Phases 9–10) — these are *new physics*, copied verbatim from the reference implementation, not reimplemented.

5 Git provenance, reference, and existing scaffolding

- The reference TOML workflow lives as **unpushed commits in a separate clone**: `/Users/chunhuizhao/projects/feature/heterogeneous_riemann_solver`, HEAD `e30d003`. It is *not* on origin (origin’s same-named branch is the merge-base `b18bed5`, an ancestor of `safs`).
- True merge-base of the two working lines: `0e13b25`. In this repo the reference tip is fetched as tag **hrs-ref**. Diffs reproducible via `git diff 0e13b25 hrs-ref -- <path>` (reference side) / `git diff 0e13b25 safs -- <path>` (safs side).

- Per-area analysis archived at /tmp/branchcmp/agent{1..5}*.md (pass 1) and /tmp/branchcmp/pass2_{exact} (pass 2).

5.1 RS scaffolding already in safs (the SAFS+RS head start)

A source audit (this pass) confirms the SAFS + aging rate-and-state goal is a **driver-wiring** job, not a build-the-module-system job — almost every piece exists and is reusable:

Piece	Status in safs	Location
[friction.rate_state] parse + RateStateBlock + [meta].law=rate_state dispatch	EXISTS	spatial_friction.{hpp:241–253, cpp:388–466,795–818}
ResolveRateState(...) (ParMesh + serial; takes pp + sigma_n_total_per_dof) + RateStatePerDOFParams	EXISTS	spatial_friction.{hpp:293–326, cpp:963–1118}
InitializeFaultDOFs_Spatial_RS (seeds a/Dc/impedance/pre-stress; psi=0 stub)	EXISTS	spatial_setup.hpp:301–346
Kernels DieterichRuinaFriction::InitializePsi, AgingLawPsi, UpdateStateAnalytic	EXISTS (byte-identical to native)	friction/{dieterich_ruina.hpp:506, state_evolution.hpp:180,304}
Aging RS sub-step iterator Tpv102SubStepIterator(FaultFaceFlux&, const AgingLawPsi&)	EXISTS	dynamic/tpv102_substep_iterator.hpp:73
FaultFrictionLaw::RateAndState wave-op dispatch (EvaluateADER on interior+shared faces)	EXISTS	dynamic/wave_operator.hpp:79,254–264
Gaussian gradual_overstress nucleation (law-agnostic; writes tau{1,2}_nuc)	EXISTS	dynamic/spatial_nucleation.{hpp:145–174, cpp:80–178}

Missing (the only RS work, all in Phases 1–3): the driver hard-rejects rate_state (spatial_dyn_driver.cpp:716), never calls ResolveRateState/InitializeFaultDOFs_Spatial_RS, hard-types its sub-step machinery to the LSW Tpv205SubStepIterator (:367,:1736), hard-codes SetFaultFrictionLaw(LSW) (:971), has **no equilibrium- ψ seed** (psi=0 stub at spatial_setup.hpp:341), and Tpv102SubStepIterator lacks a nuc_callback overload + SetDiagNumLocalFaultQPs. That is the entire gap, and it is what Phases 1–3 close.

6 Module architecture (the “parts”)

Chosen shape (simplified this pass): three orthogonal “parts,” each a small strategy interface with a one-line factory. The driver carries no friction if/else, no nucleation switch, and no flux if/else — it holds three `unique_ptrs` and calls through interfaces. Adding a friction law, a nucleation method, or a flux variant is a new class + one factory line, never a driver edit. The three parts are **method-oriented**, not benchmark-oriented:

```
friction_laws      : IFrictionIterator    <- {LSW, RS-aging, RS-slip-law-SRW}
nucleation_methods : INucleationMethod   <- {Gaussian, CompactCircular,
                                             InstantaneousCircular, Static}
riemann_flux      : interior_flux        <- {scalar (homogeneous),
                                             matrix (heterogeneous/bimaterial)}
```

6.1 friction_laws part (unified, method-oriented)

Phasing (see §7): to land the SAFS+RS goal fast and on proven code, **Phases 1–3 wire a thin IFrictionIterator strategy with adapters over the existing Tpv205/TPV102 iterators** (`LswFrictionIterator`, `RateStateAgingFrictionIterator`). **Phase 5** then introduces the unified design below (`SubStepIteratorBase` + `RateStateSubStepIterator` <Policy> + `LinearSlipWeakeningIterator`, adding the SRW law) and re-points the factory to it, parity-tested bit-for-bit against the standalone oracle. The `IFrictionIterator` interface and the driver are stable across both; only the iterator *internals* change.

The end-state collapses the three benchmark-named sub-step iterators into a method-oriented set sharing one skeleton. The structural cleavage is **LSW vs rate-state**, not 102-vs-104: TPV102 and TPV104 differ in *exactly one line* (the state-variable update), while TPV205 has a different kernel *shape* (closed-form, no root-find, no state). So:

New file `dynamic/friction_substep_iterator.{hpp,cpp}` holds:

- **SubStepIteratorBase** — the common ~250-line ADER sub-step envelope that is byte-for-byte identical across all three today (the `SetSubSteps` validator, the $\Sigma \Delta T == dt_macro$ tolerance, the `memset` of `I_imp_*`, the per-(sub-step, QP) $Q_{\pm}$ build for both the time-averaged I/dt and per-sub-step $Q_{pointwise}$ cases, the `accum_scale` accumulation, the slip accumulation $d.slip\{1,2\} += s.V\{1,2\} * dt_sub$, and `WriteBackState` on the last sub-step). It exposes a single templated driver `RunSubSteps_(dofs, coords, qsource, dt_macro, t0, I_imp_p, I_imp_m, nuc_fn, step_fn)` that calls a per-QP `step_fn` functor and a `nuc_fn(t_sub_end, dt_sub)` once per sub-step before the QP loop. *This collapses both the triplication and the Advance vs AdvanceWithSubStepStates duplication.*
- **RateStateSubStepIterator<StatePolicy>** — the rate-state iterator, parameterized by a state-evolution policy. Its per-QP `step_fn` is the existing token sequence `flux_.ComputeStageState(d, Qp, Qm, s, method) → d.slip{1,2} += s.V{1,2} * dt_sub → d.psi = StatePolicy::UpdatePsi(law_, d, s.V_abs, dt_sub, extra, i) → flux_.BuildImposedState(...)`. The policy is the *only* variation point.

```

template <class StatePolicy>
class RateStateSubStepIterator : public SubStepIteratorBase,
                                public IFrictionIterator {
public:
    RateStateSubStepIterator(FaultFaceFlux& flux,
                             const typename StatePolicy::Law& law);
    // IFrictionIterator::Advance(...) forwards to RunSubSteps_ with the
    // policy step_fn; FrictionSolver::Method + ExtraState come from cfg.
private:
    const typename StatePolicy::Law& law_;
    typename StatePolicy::Extra      extra_; // std::monostate for aging;
                                         // owns V_w span for SRW
};

```

- **LinearSlipWeakeningIterator** — a separate small class (no method, no ComputeStageState, no state variable). Its step_fn lifts Tpv205SubStepIterator:: StepOneQP_ **verbatim** (ComputeTrialTraction + SolveLSW_TPV205 + the [SLIP] diagnostic + callback nucleation). Forcing LSW into the RS template would need dead policy hooks, so it stays distinct — but still inherits SubStepIteratorBase for the envelope.

State policies live next to their laws in friction/state_policies.hpp (so the friction-method coupling sits with the friction code):

```

struct RateStateAgingPolicy {                // TPV102
    using Law    = AgingLawPsi;
    using Extra = std::monostate;
    static real_t UpdatePsi(const Law& L, const DOFData& d, real_t V,
                             real_t dt, const Extra&, int /*i*/) {
        return UpdateStateAnalytic(d.psi, V, d.Dc, dt,
                                     L.GetF0(), L.GetB(), L.GetV0());
    }
};

struct RateStateSlipLawSrwPolicy {           // TPV104
    using Law    = SlipLawSRWPsi;
    using Extra = std::span<const real_t>;   // V_w, owned by iterator
    static real_t UpdatePsi(const Law& L, const DOFData& d, real_t V,
                             real_t dt, const Extra& Vw, int i) {
        return UpdateStateAnalyticSlipLawSRW(d.psi, V, d.Dc, dt, Vw[i],
                                                d.a, L.GetB(), L.GetV0(), L.GetF0(), L.GetMuW());
    }
};

using RateStateAgingIterator =
    RateStateSubStepIterator<RateStateAgingPolicy>;
using RateStateSlipLawSrwIterator =
    RateStateSubStepIterator<RateStateSlipLawSrwPolicy>;

```

The driver-facing strategy interface and factory:

```
class IFrictionIterator {
public:
    virtual void SetSubSteps(std::vector<real_t> dT, std::vector<real_t> w) = 0;
    virtual std::vector<real_t> GetDeltaT() const = 0;
    virtual std::vector<real_t> GetTimeWeights() const = 0;
    virtual void Advance(std::vector<DOFData>& dof,
        const std::vector<Vector>& coords,
        const std::vector<std::vector<real_t>>& Qp,
        const std::vector<std::vector<real_t>>& Qm,
        real_t dt, real_t t0, real_t* Iimp_p, real_t* Iimp_m,
        const std::function<void(real_t,real_t)>& nuc_cb) = 0;
    virtual void SetDiagNumLocalFaultQPs(int n) = 0; // safs speckle diag hook
    virtual FaultFrictionLaw WaveOpLaw() const = 0; // LSW | RateAndState
    virtual ~IFrictionIterator() = default;
};

// dynamic/friction_iterator_factory.{hpp,cpp}
std::unique_ptr<IFrictionIterator> MakeFrictionIterator(
    const spatial::SpatialFrictionConfig& cfg, FaultFaceFlux& flux,
    const spatial::RateStatePerDOFParams* rs /*nullptr for LSW*/);
// is_lsw -> LinearSlipWeakeningIterator
// RS && aging_law -> RateStateAgingIterator
// RS && slip_law_srw -> RateStateSlipLawSrwIterator
```

Byte-exactness guarantee. The unified iterators are *new code*; the standalone `tpv{205,102,104}_substep_iterator` stay **untouched as the oracle**. The kernel call tokens are not reordered — the policy only *selects* the `UpdateState...` call that the original code already inlined. Parity tests (`tests/unit/test_tpv104_substep_iterator` and siblings) plus a new RS/LSW one-shot parity assert the unified iterator reproduces each standalone iterator bit-for-bit on a fixed fixture. Diagnostics (`SEAS_DIAG_TPV104_STATE`, `slip_rate_substep_max`) stay behind their `#ifdef/env` gates. *Net: ~250 triplicated lines → one RunSubSteps_; 102 vs 104 → one policy line; 205 stays a focused closed-form class; names are method-oriented.*

6.2 nucleation_methods part

New file `dynamic/nucleation_method.hpp` — interface:

```
class INucleationMethod {
public:
    virtual void ApplyIncrement(std::vector<DOFData>& dof,
        real_t t, real_t dt) = 0;
    virtual void ApplyOnce(std::vector<DOFData>& dof) {} // static/instantaneous
    virtual bool IsPerSubStep() const = 0;
    virtual ~INucleationMethod() = default;
};
```


Concrete methods (all method-oriented, benchmark-agnostic): GaussianGradualOverstress (wraps safs's existing ResolveGradualOverstress/ ApplyGradualOverstressIncrement — SAFS + as a TPV option), CompactCircularGradualOverstress (the new SCEC bell $F(r)=\exp(r^2/(r^2-R^2))\cdot\text{smoothStep}$ — TPV102/104), InstantaneousOverstressCircular (cosine-tapered one-shot patch — **TPV31**; ApplyOnce seeds tau_nuc, ApplyIncrement no-ops), and StaticOverstress (TPV205 — patches already in tau_pre_; both hooks no-op). ApplyIncrement is the per-sub-step nuc_cb body the driver passes into IFrictionIterator::Advance.

```
// dynamic/nucleation_factory.{hpp,cpp}
std::unique_ptr<INucleationMethod> MakeNucleation(
    const spatial::SpatialFrictionConfig& cfg,
    const std::vector<Vector>& dof_coords,
    const std::vector<FaultBasisRow>& dof_basis);
```

6.3 riemann_flux part (homogeneous scalar vs heterogeneous matrix)

The interior (non-fault) face flux becomes an explicit, config-selected choice. **The fault flux is unaffected** — FaultFaceFlux is used on fault faces regardless of the interior choice (verified: the bimaterial path in hrs-ref applies only to interior non-fault faces; the fault still uses FaultFaceFlux). So this part is orthogonal to friction_laws.

- **interior_flux = "scalar"** (homogeneous Godunov, the existing safs path): the scalar WaveOperator(mesh, order, λ , μ , ρ , bc) ctor; owned_flux_pool_ == nullptr; the per-face dispatch uses the single shared flux_. **Mixed-flux (mixed_flux $\in$ {adjacent,...}, Zhang et al. 2023) is valid ONLY here** (a per-face Central-vs-Interior choice on the scalar path).
- **interior_flux = "matrix"** (heterogeneous / bimaterial Riemann): the heterogeneous WaveOperator(mesh, order, const MaterialField&, bc) ctor builds owned_flux_pool_ per-element (material.EvalAt(elem,...)), precomputes per-(face,side) flux matrices via BimaterialFlux::BuildPerFaceFluxMatricesGlobal, and at runtime applies BimaterialFlux::ApplyPerFaceFluxNeighbor, Q_self, Q_nbr, F_h_self) at each interior QP. mixed_flux MUST be unset (mutual-exclusion guard).

Port required (hrs-ref → safs, Phase 9), verbatim: dynamic/godunov_flux_bimaterial.{hpp, cpp} (the BimaterialFlux class, SeisSol positive-stress matrix convention), the owned_flux_pool_ dispatch branches in wave_operator.inl (Mult + ADER interior-face paths), BuildPerFaceBimaterialFluxMatrices, the cross-rank ExchangeBiMaterialNeighbours_ + shared_face_neighbour_material_, and per_elem_lmr_/MaxCpInElement CFL caching. In safs the gating member owned_flux_pool_ and heterogeneous_material.{hpp, cpp} (Constant/ Coefficient) **already exist** — only BimaterialFlux and the dispatch wiring are missing. The depth-dependent MaterialField (DepthProfile1D / MakeDepthProfile1DMaterial, Coefficient mode) is ported in Phase 10 for TPV31.

```
// In the driver (Phase 5/9): one branch, no per-face logic in the driver.
std::unique_ptr<WaveOperator> wave_ptr;
if (cfg.numerics.interior_flux == InteriorFlux::Scalar)
    wave_ptr = std::make_unique<WaveOperator>(pmesh, order, lam, mu, rho, bc);
else /* Matrix */
```



```

    wave_ptr = std::make_unique<WaveOperator>(pmesh, order, material_field, bc);
WaveOperator& wave = *wave_ptr;

```

6.4 Shared-rank correctness constraints (must hold for every fault-flux / exchange change)

The unified friction iterator and the new matrix-flux path both touch cross-rank fault and interior faces. From the shared-fault audit (R-001/R-004/R-101/R-1303/R-1601), the design **must**:

1. **Detect shared fault faces symmetrically across ranks** — gate on GetSharedFaceTransformations/an Allgather of global face IDs, **never** on local boundary-element ownership (R-001: one-sided detection silently dropped RHS → 6% error at np=400). The unified iterator changes *nobody's* face detection; it must keep using the existing symmetric fault_shared_tagged_set.
2. **Keep the R-1601 shared-QP rule intact.** Shared fault QPs **bypass the sub-step iterator and run inline EvaluateADER** in ComputeADERSHaredFaceFluxRHS; only interior fault QPs go through the iterator. The unification must preserve this exact split — RunSubSteps_ operates on the interior-QP set; the shared path is untouched. (At np>1 a rank-local canonical frame in the iterator caused tau=4e28 → SIGABRT within ~7 macro-steps.)
3. **Exchange neighbour bulk Q before reading it** on shared QPs (ghost_gf_full_state_→ExchangeFaceNbrData and map components via GetFaceNbrElementVDofs — **byNODES, not contiguous slabs** (R-004). Per-sub-step ghost coverage is mandatory (R-1303): the predictor is element-local and does not fill ghosts.
4. **Reconcile friction DOFData across the seam before flux assembly** — the boss rank (lower ID) broadcasts {psi, slip1/2, v1/2, tau*_corr, sigma_n_corr, slip_rate} and the canonical imposed state via ExchangeAndPairSharedFaultQPs; the peer overwrites locally so both assemble bit-identical flux. **For rate-state this is the open shared-fault-RS ψ risk** (the audit's R-001/R-004 lineage; = the review's R-004): the reconcile **does** broadcast ψ (payload[5]), so cross-rank ψ is single-valued — the residual issue is *temporal* (end-of-step ψ on the inline shared flux → 1st-order at ader_order $\geq$ 2), gated in **Phase 3** (the SAFS+RS deliverable), not Phase 7. (*The safs-audit "R-001/R-004" here are the shared-fault debug-history findings — a separate numbering from the REVIEW.md R-001...R-008.*)
5. **Honour the collective contract** — every rank with *any* shared face (fault or not) must call the exchange, gated on pmesh.GetNSharedFaces() > 0, NOT on having shared *fault* faces (R-1600: a missing call deadlocked for 13.5 min). The new bimaterial ExchangeBiMaterialNeighbours_ adds one more collective and must obey the same gate.
6. **Use absolute QP indexing** on the shared branch (R-1304), and keep shared fault QPs off the precomputed-flux table (they always run inline so the reconciliation payload is complete).
7. **MPI parity is a gate, not an afterthought** — Phases 3/9 add a forced-2-rank single-shared-fault-face test asserting cross-rank DOFData symmetry and flux bit-identity.

6.5 What leaves vs stays in the driver

Leaves (into the factories/iterators): the three-way iterator construction, AgingLawPsi/SlipLawSRWPsi construction, the nucleation switch/nuc_cb lambda, the equilibrium- ψ seed loop, and the scalar-vs-matrix WaveOperator ctor branch (one if). **Stays (safs fixes — DO NOT MOVE)**:

R-005 per-QP basis (BuildPerDOFFaultTables, ComputeQPBasis*), driver-local FaultBasis + ref_normal=(0,-1,0), R-101 shared-fault tripwires + SEAS_R101_NONFATAL + SEAS_DIAG_BLOWUP + slip_rate_substep_max reset + NaN tripwire, speckle/Phase-1 diag, ParaView fault VTU plumbing (_k4, WriteFaultSurfaceVTU, SetRegisterFaultProjectionsInVolumePV), checkpoint dedup R-011, the R-1601 shared-QP inline bypass, and the AdvanceADERWithSubStep_Spatial body — only its iterator parameter changes from Tpv205SubStepIterator& to IFrictionIterator&; SetDiagNumLocalFaultQPs is a virtual.

7 Code graph – full structure

Five views of the end-state structure. Status legend: [NEW] create, [MOD] modify (preserve existing behaviour), [REUSE] use as-is (no edit), [PORT] copy verbatim from hrs-ref, [ORACLE] keep untouched as the byte-exact reference. The three “parts” — **friction_laws**, **nucleation_methods**, **riemann_flux** — live under dynamic/ (G1) and are exercised in G2/G3/G5.

7.1 G1 – Code map (files by role)

```
miniapps/seas/
|- drivers/spatial_dyn_driver.cpp [MOD] orchestrator; safes fixes stay
|- spatial/code/
|  |- spatial_friction.{hpp,cpp} [MOD] TOML schema + resolver
|  `-- spatial_stress.{hpp,cpp} [MOD] +FaultLocalPrestress; ct sign
|- dynamic/ ( friction_laws + nucleation_methods + riemann_flux )
|  |- friction_substep_iterator.* [NEW] base + RS template + LSW
|  |- friction_iterator.hpp [NEW] IFrictionIterator + factory
|  |- friction_iterator_factory.* [NEW] MakeFrictionIterator(cfg,flux,rs)
|  |- nucleation_method.hpp [NEW] INucleationMethod + 4 methods
|  |- nucleation_factory.* [NEW] MakeNucleation(cfg,coords,basis)
|  |- spatial_nucleation.{hpp,cpp} [MOD] +CompactCircular +InstCircular
|  |- tpv205/102/104_substep_iter* [ORACLE] standalone-driver reference
|  |- spatial_setup.hpp [MOD] + RS IP-aware init overload
|  |- wave_operator.{hpp,inl} [MOD] + matrix dispatch; scalar same
|  |- fault_face_flux.{hpp,cpp} [REUSE] trial/stage/imposed/writeback
|  |- godunov_flux_bimaterial.* [PORT] BimaterialFlux (matrix Riemann)
|  `-- heterogeneous_material.* [MOD] +DepthProfile1D (Coefficient)
|- friction/ ( law kernels, byte-identical to native )
|  |- dieterich_ruina.hpp [REUSE] Brent solve + InitialStatePsi
|  |- state_evolution.hpp [REUSE] AgingLawPsi + analytic update
|  |- slip_law_srw_psi.hpp [REUSE] SlipLawSRWPsi + analytic update
|  `-- state_policies.hpp [NEW] Aging/SlipLawSRW psi policies
|- fault/
|  |- fault_basis.hpp [REUSE] oriented frame (R-005; D1)
|  |- fault_geometry.hpp/_safes.inl [MOD] rename + ComputeParamsFaultLocal
|  `-- rate_state_fault.hpp [MOD] rename knock-on
```

```

|- config/seas_config*.hpp          [MOD] rename knock-on
|- tpv{205,104,102}/configs/*.toml [NEW] stress + friction + nucleation
|- tpv31/ (config,mesh,bench,tests) [PORT] het-material LSW case
|- safs/.../config/*rate_state*     [NEW] SAFS rate-state config
|- jobs/{tpv*_spatial,safs}/*.sbatch [NEW] run scripts (dev + 8-rank)
`- tests/unit/test_*.cpp            [NEW] sign/config/stress/factory/parity

```

7.2 G2 – Runtime wiring (setup: config -> objects)

TOML (--config X.toml)

```

| spatial::LoadSpatialFrictionConfig()
v

```

SpatialFrictionConfig cfg -- read by:

```

|- [meta].law / [friction].state_evolution -> is_lsw, rs_use_srw
|- [stress].kind -> stress seeding (see G4)
|- [friction.*] -> Resolve{SlipWeakening|RateState} -> per-DOF params
|- [nucleation] -> MakeNucleation(cfg,coords,basis)
| -> INucleationMethod {Gaussian|CompactCircular|InstCirc|Static}
|- [material].kind -> Constant | DepthProfile1D (-> MaterialField)
|- [numerics] -> interior_flux {scalar|matrix}, mixed_flux,
| cfl_safety, fault_iterator
v

```

WaveOperator ctor (see G5):

```

| scalar -> WaveOperator(mesh,p,lam,mu,rho,bc) owned_flux_pool_=0
` matrix -> WaveOperator(mesh,p,MaterialField,bc) builds flux pool
v

```

DOFData[] (tau1_0, tau2_0, sigma_n0, a, Dc, psi, ...)

```

| (RS only) SeedEquilibriumPsi_RS -> psi = InitialStatePsi(...)
v

```

MakeFrictionIterator(cfg, fault_flux, rs?) -> unique_ptr<IFrictionIterator>

```

| LinearSlipWeakeningIterator (LSW)
| RateStateSubStepIterator<RateStateAgingPolicy> (aging)
| RateStateSubStepIterator<RateStateSlipLawSrwPolicy> (SRW, owns V_w)
v

```

wave.SetFaultFrictionLaw(fr->WaveOpLaw()) // LSW | RateAndState

7.3 G3 – Per-macro-step call & data flow (time loop)

for each macro step dt: [driver loop -- safs fixes live HERE]

AdvanceADERWithSubStep_Spatial(wave,*fr,dof,coords,Q,dt,...,nuc_cb)

```

|- wave.ComputeADERSubStepStates(...) -- SHARED predictor
| (matrix path: per-element CK via ApplyJacobianPerElementDOF_)
|- wave.EvaluateBulkAtFaultQPsCanonical(...) -- SHARED (R-005;D1)
`- fr->Advance(...) -> SubStepIteratorBase::RunSubSteps_ [INTERIOR QPs]
    ` per ADER sub-step:
        |- nuc_cb(t,dt) -> INucleationMethod::ApplyIncrement

```

```

|         writes DOFData.tau{1,2}_nuc (nucleation_methods)
|- FaultFaceFlux::ComputeTrialTraction/ComputeStageState -SHARED
|- friction: LSW closed-form | RS Brent      (friction_laws)
|- StatePolicy::UpdatePsi: none|Aging|SlipLawSRW
`- FaultFaceFlux::BuildImposedState/WriteBackState -- SHARED

V
wave.ComputeADERSHaredFaceFluxRHS(...) [SHARED-FAULT QPs, R-1601]
  `- inline EvaluateADER (NOT the iterator) + boss-rank DOFData reconcile
wave.AdvanceADER(... imposed states ...) -- SHARED apply
  ` (matrix path: BimaterialFlux::ApplyPerFaceFlux on interior faces)
post-step (UNCHANGED): R-101 tripwire | NaN check | slip_rate_substep_max
reset | station write | checkpoint R-011 | fault VTU

```

7.4 G4 – Stress-source dispatch (both paths: positive input = right-lateral on-fault)

```

[stress].kind
|- "fault_local_prestress" (TPV205/102/104 option; D3.2)
|   -> ComputeParamsFaultLocal(tau_strike, tau_dip, sigma_n, P_p)
|   -> tau1_0=tau_dip ; tau2_0=+tau_strike ; sigma_n0=sigma_n-P_p
|   (direct seed; NO Cauchy tensor, NO projection; == native)
`- "constant_tensor" (SAFS regional; TPV option; D3.1)
    factory: S_xy = -sigma_xy_pa (negate strike off-diag at build)
             S_yy = +sigma_yy_pa (normals compression-positive)
    -> ComputeParams<ConstantTensorStressSource> (no-flip, UNCHANGED)
    -> sigma_n+=S_yy ; tau_dip+=S_yz ; tau_strike=-S_xy+=sigma_xy_pa
        (stored tensor BIT-IDENTICAL to today; on-fault unchanged)

```

Both paths now satisfy **positive input** $\Rightarrow$ **right-lateral-positive on-fault**. The `fault_local` path seeds directly (no projection). The `constant_tensor` path keeps the no-flip projection rule verbatim; only the TOML $\rightarrow$ source factory negates the strike off-diagonal, so the stored Cauchy tensor and every on-fault quantity are bit-identical to the current SAFS configs (proof in §2-D3.1). The regional `StressSource` is fault-only — it seeds `DOFData` and is never a bulk initial condition — so the change is provably local.

7.5 G5 – Riemann-flux dispatch (scalar vs matrix; fault flux unaffected)

```

[numerics].interior_flux
|- "scalar" (TPV205/102/104, SAFS) -- homogeneous Godunov
|   WaveOperator(mesh,p,lam,mu,rho,bc); owned_flux_pool_ = nullptr
|   interior face: flux_.Interior(...) [+ optional mixed Central]
|   (mixed_flux ALLOWED here only)
`- "matrix" (TPV31) -- heterogeneous/bimaterial
    WaveOperator(mesh,p,MaterialField,bc); builds owned_flux_pool_
    precompute per-(face,side) via BuildPerFaceBimaterialFluxMatrices_
    cross-rank: ExchangeBiMaterialNeighbours_ (collective; §5.4)
    interior face: BimaterialFlux::ApplyPerFaceFlux(...)
    (mixed_flux FORBIDDEN; mutual-exclusion guard)

```

FAULT faces: FaultFaceFlux + friction (UNCHANGED on both paths)

The Riemann choice changes only **interior non-fault** faces; fault faces always use FaultFaceFlux + the friction iterator, so friction_laws and riemann_flux are orthogonal.

8 Master phase / file table

Prioritization (this pass): the SAFS + aging rate-and-state run is the near-future goal, and the **entire RS config/resolver/setup/kernel/iterator/wave-op scaffolding already exists in safs** (verified — see §4.1). So **Phases 1–3 deliver a working SAFS+RS run** by wiring the existing pieces (driver was hard-gated to LSW); Phases 4–10 add the method-oriented unification, the TPV regression, the matrix Riemann solver, and TPV31. The » marks the near-future deliverable.

Phase	Files	Action
1 RS primitives »	dynamic/tpv102_substep_iterator. dynamic/spatial_setup.hpp (+dieterich_ruina.hpp include), Makefile; new tests/unit/test_tpv102_nuc_case. test_seed_equilibrium_psi_rs.	add(Tpv102SubStepIterator nuc_callback overload + SetDiagNumLocalFaultQPs (additive, byte-safe); add SeedEquilibriumPsi_RS R-001 d.eta_s+V_init>0; R-009 V_0==FrictionSolver::V0 guard); link tpv102 iterator + test targets
2 Friction strategy »	new dynamic/friction_iterator.hpp friction_iterator_factory.{hpp modify drivers/spatial_dyn_driver.cpp (retype :367 + R-010 rename :439 call→Advance); new tests/unit/test_friction_iterator. test_advance_interface_compile	IFrictionIterator (incl. R-010 SetPlat/GetTimeWeights) + Lsw/RateStateAging adapters over the existing Tpv205/Tpv102 iterators; MakeFrictionIterator (R-009 defensive V0 guard); AdvanceADERWithSubStep→IFrictionIterator LSW byte-unchanged

Phase	Files	Action
3 SAFS+RS run »	drivers/spatial_dyn_driver.cpp, spatial/code/spatial_friction (R-006); SAFS configs (Dc2/8/10 sigma_xy_pa flip, atomic); new RS config + sbatch + test_constant_tensor_sign.cpp (parametrized), test_resolve_rate_state_guard test_paraview_state_channel_rej test_shared_fault_rs_psi_cons	remove is_lsw reject (:716); gated RS branch (R-002: outer-scope lsw+rs, slip_weakening.has_value() under is_lsw) ResolveRateState(R-003 PorePressureSpec{}})+InitializeFaultDOFs SetFaultFrictionLaw(RateAndState); Paraview state→ψ (R-005); rejection per-DOF b/V ₀ /f ₀ (R-006); np2x.cpp →allow a>b (R-011); reuse Gaussian nucleation; D3.1 factory negation → » WORKING SAFS+RS
4 D-rename	fault/fault_geometry.hpp/_saf rate_state_fault.hpp, config/seas_config*.hpp, spatial/code/spatial_stress.{hpp, tests/unit/test_compute_safs_params.cpp	computeSAFSParams→ComputeParams/HasPara grep-zero residual; suite green (cosmetic generalization, no behavior change)
5 Unified iterator	new dynamic/friction_substep_iter friction/state_policies.hpp; modify friction_iterator_factory.cpp new test_friction_substep_iterator_parity.cpp	SubStepIteratorBase+RateStateSubStepIt {hpp,cpp}, ing+SRW)+LinearSlipWeakeningIterator; re-point the Phase-2 adapters; bit-parity vs the standalone oracle
6 TPV config + D3.2	spatial/code/spatial_frictionTPV, map spatial_stress.{hpp,cpp}, dynamic/spatial_setup.hpp	TPV, map /flux vocabulary (interior_flux∈{scalar,matrix}, material.kind, SRW state_evolution, boxcar_taper); FaultLocalPrestressSource+ComputeParams (D3.2)
7 nucleation_methods	new dynamic/nucleation_method.hpp nucleation_factory.{hpp,cpp}; modify dynamic/spatial_nucleation.{hpp,cpp}	INucleationMethod+factory; add compact-circular and instantaneous-circular resolvers

Phase	Files	Action
8 TPV benchmarks	new tpv{205,104,102}/configs/*.toml (scalar); gold/SCEC overlays; meshes, jobs/tpv*_spatial/*.sbatch, review/sign tests; planar-basis test (D1)	wire+run TPV205/102/104 (scalar); gold/SCEC overlays; np>1 MPI parity
9 Matrix Riemann (PORT)	port dynamic/godunov_flux_bimaterial_low{hpp,cpp}; pool_dispatch + modify dynamic/wave_operator.{hpp,inc}, scalar path byte-unchanged; Makefile	BimaterialFlux ExchangeBiMaterialNeighbours_ np>1 parity
10 TPV31 (PORT)	port DepthProfile1D into heterogeneous_material.*; copy tpv31/ (config, mesh, benchmark, tests)	depth-profile matrix Riemann; LSW + depth-linear cohesion; InstantaneousOverstressCircular nucleation; SCEC eqdyna/seisol overlay
— REUSE (no change)	elasticity_operator_setup.inl, already exist; orthogonal or fault_face_flux.*, ADER core, friction/* kernels, spatial_friction.cpp RS parse/resolve, InitializeFaultDOFs_Spatial_RS	byte-identical to native

9 Phases

9.1 Phase 1 – RS primitives » (aging-iterator hooks + equilibrium- ψ seed + build)

Goal

The aging RS sub-step iterator (Tpv102SubStepIterator) can be driven by a SAFS Gaussian nucleation **callback** and carries the diag hook; a per-DOF **equilibrium- ψ** seed exists; the build links the tpv102 iterator into the spatial driver. **Additive and byte-safe — no existing behaviour changes.** This phase makes RS *runnable* without touching the driver’s control flow.

Files to Modify

- dynamic/tpv102_substep_iterator.{hpp,cpp} — add a nuc_callback overload of AdvanceWithSubStepStat and a SetDiagNumLocalFaultQPs(int) member.
- dynamic/spatial_setup.hpp — add SeedEquilibriumPsi_RS(...); **add #include "../friction/dieterich_ruina.hpp"** (R-001 — the header currently includes only fault_face_flux.hpp, heterogeneous_material.hpp, spatial_friction.hpp, so DieterichRuinaFriction is otherwise undeclared). FrictionSolver (for the R-009 V0 guard) is visible transitively via fault_face_flux.hpp → friction_solver.hpp; if a future refactor drops that edge, add

#include "friction_solver.hpp" explicitly.

- miniapps/seas/Makefile — link \$(TPV102_SUBSTEP_ITERATOR_OBJ) into seas_spatial_dyn_driver; add the two new test targets.

Files to Create

- tests/unit/test_tpv102_nuc_callback_parity.cpp
- tests/unit/test_seed_equilibrium_psi_rs.cpp

Detailed Requirements

1. **Nucleation callback overload** on Tpv102SubStepIterator (declared right after the existing plain AdvanceWithSubStepStates(...) at tpv102_substep_iterator.hpp:117):

```
void AdvanceWithSubStepStates(
    std::vector<DOFData>&                dof_data,
    const std::vector<mfem::Vector>&      fault_coords,
    const std::vector<std::vector<real_t>>& Q_plus_sub,
    const std::vector<std::vector<real_t>>& Q_minus_sub,
    real_t dt_macro, real_t t_macro_start,
    real_t* I_imp_plus, real_t* I_imp_minus,
    FrictionSolver::Method method,
    const std::function<void(real_t /*t_sub_end*/,
                             real_t /*dt_sub*/)>& nuc_callback);
```

Implementation: copy the existing AdvanceWithSubStepStates body verbatim (.cpp:333–371 region) but **replace the hard-coded ApplyNucleationIncremental_TPV102(dof_data, fault_coords, t_sub_end, dt_sub) call (.cpp:326) with nuc_callback(t_sub_end, dt_sub)** at the identical point (once per sub-step, before the per-QP loop). Everything else (the ComputeStageState→slip→UpdateStateAnalytic→BuildImposedState sequence) is unchanged. Use method = FrictionSolver::Method::Brent from the caller (CLAUDE.md: Brent, not Newton). The plain overload stays for the standalone TPV102 driver.

2. **SetDiagNumLocalFaultQPs(int n)** on Tpv102SubStepIterator: store into a new member int diag_num_local_fault_qps_ = -1; (mirror Tpv205SubStepIterator:146). It is a no-op accessor for interface symmetry (TPV102 has no [SLIP] trace); having it lets the Phase-2 adapter satisfy IFrictionIterator uniformly.
3. **SeedEquilibriumPsi_RS** in dynamic/spatial_setup.hpp (**R-001 corrections inlined**):

```
inline void SeedEquilibriumPsi_RS(
    std::vector<DOFData>&                dof_data,
    const spatial::RateStatePerDOFParams& rs,    // a,Dc,V_init per-DOF
    const spatial::RateStateBlock&       blk)    // b,V_0,f_0 scalar globals
{
    // R-009: V_0 has two consumers -- the config (this seed + AgingLawPsi)
    // and the hardcoded FrictionSolver::V0 used by the force solve. They
    // MUST agree or the fault is not in equilibrium at t=0. (Proper fix:
```

```

// thread blk.V_0_default into FrictionSolver; minimal guard for now.)
MFEM_VERIFY(std::abs(blk.V_0_default - FrictionSolver::V0)
    <= 1e-30 + 1e-12 * FrictionSolver::V0,
    "SeedEquilibriumPsi_RS: [friction.rate_state].V_0 ("
    << blk.V_0_default << ") must equal FrictionSolver::V0 ("
    << FrictionSolver::V0 << "); the force solve hardcodes V0.");
for (std::size_t i = 0; i < dof_data.size(); ++i) {
    DOFData& d = dof_data[i];
    // R-001 edge guard: else log()/InitialStatePsi -> inf/NaN psi.
    MFEM_VERIFY(rs.V_init(i) > 0.0,
        "SeedEquilibriumPsi_RS: V_init <= 0 at DOF " << i
        << " (ill-posed steady state)");
    const DieterichRuinaFriction fr(
        DieterichRuinaFriction::Constants{
            blk.V_0_default, blk.f_0_default, blk.b_default, rs.Dc(i)});
    const real_t tau0 = std::hypot(d.tau1_0 + d.tau1_nuc,
        d.tau2_0 + d.tau2_nuc);
    // R-001: damping is d.eta_s (== rs.eta(i) == 0.5*sqrt(mu*rho) for
    // homogeneous SAFS, and exactly what FaultFaceFlux uses in the
    // solve). NEVER rs.eta_s -- RateStatePerDOFParams has no such member.
    d.psi = (tau0 > 0.0)
        ? fr.InitialStatePsi(tau0, rs.V_init(i),
            d.sigma_n0, d.eta_s, rs.a(i))
        : (blk.f_0_default
            + blk.b_default * std::log(blk.V_0_default / rs.V_init(i)));
}
}

```

This overwrites the `d.psi = 0.0` stub at `spatial_setup.hpp:341`. The `tau0==0` branch uses the locked steady-state ψ ($f_0 + b \cdot \ln(V_0/V_{\text{init}})$). **R-001 (CRITICAL)**: the radiation-damping argument is `d.eta_s` (a `DOFData` member, set at `spatial_setup.hpp:82`), **not** `rs.eta_s` — `RateStatePerDOFParams` has only a per-DOF `eta` Vector (`spatial_friction.hpp:295`); `rs.eta_s` is a hard compile error. For the homogeneous SAFS material `d.eta_s == rs.eta(i) == 0.5*sqrt( $\mu\rho$ )` at the same element centroid, so the seed's damping matches the friction solve's. (Confirm `DieterichRuinaFriction::Constants{V0,f0,b, Dc}` field order and `InitialStatePsi(tau0,V_init,sigma_n,eta,a)` at impl time — both verified at `friction/dieterich_ruina.hpp:43-49,506-507`.)

R-009 (V0 single source of truth): the leading `MFEM_VERIFY` asserts the config `blk.V_0_default` equals the solver's compile-time `FrictionSolver::V0` ($= 1e-6$, `friction_solver.hpp:60`), which the force solve hardcodes (`fault_face_flux.cpp:224`: $C = \exp(\psi/a)/(2 \cdot \text{FrictionSolver::V0})$). Without it, a config that sets `[friction.rate_state].V_0  $\neq 1e-6$` seeds ψ (and runs `AgingLawPsi`) against one `V_0` while the solve uses another $\rightarrow$ the fault is **not** at `V_init` at $t=0$ and the friction is wrong throughout, with no error (the `RateStateBlock` default is $1e-6$, so a default config is only *accidentally* consistent). `f_0/b` are immune (the regularized $f =$

$a \cdot \text{asinh}[V/(2V_0) \cdot \exp(\psi/a)]$ is single-sourced from `blk` in both seed and ψ -update); only `V_0` has the dual-source hazard. The minimal guard above is byte-safe for the default-`V_0` deliverable; the **proper** fix (a follow-up, per the no-hardcoded-numbers rule) threads the resolved `V_0` into `FrictionSolver/CompleteFromVabs`, removing the `constexpr` entirely — but that touches the byte-exact TPV oracle path, so it is gated out of the priority run.

Edge Cases

- `tau0 == 0` (no pre-stress shear) $\rightarrow$ seed the locked steady state, not `InitialStatePsi` (which divides through the imposed traction).
- `rs.V_init(i) <= 0` $\rightarrow$ abort in `SeedEquilibriumPsi_RS` (ill-posed steady state).
- **R-009**: `blk.V_0_default  $\neq$  FrictionSolver::V0` $\rightarrow$ abort (dual-source `V_0`); the deliverable uses the default `V_0 = 1e-6`, so the guard passes and is a tripwire for future tuning.
- The plain `AdvanceWithSubStepStates` overload must remain byte-identical for the standalone TPV102 driver (the new overload is strictly additive).

Acceptance Criteria

- `seas_test_tpv102_nuc_callback_parity` (**R-008 — pin one method**): on a 2-QP, `O=2` fixture, (a) the new overload called with `FrictionSolver::Method::NewtonRaphsonStable` (the *same* method the plain overload defaults to, `tpv102_substep_iterator.hpp:127`) and `nuc=&](t,dt){ApplyNucleationIncremental_TPV102(dof,coords,t,dt);}` reproduces the plain overload **bit-for-bit** (`EXPECT_EQ` on `psi,slip1/2,V1/2,tau*_corr,sigma_n_corr,I_imp $\pm$`). The parity fixture **must not** use Brent on one side and the default on the other (a legitimate solver-difference would masquerade as a regression). (b) The new overload with an **empty** callback matches a reference run with nucleation suppressed.
- A **second test case in the same target** (`test_tpv102_nuc_callback_parity.cpp`, **R-008 — non-parity Brent**): the new overload with `FrictionSolver::Method::Brent` (the production RS method, `CLAUDE.md`) produces **finite**, **sign-consistent** `tau*_corr` and **monotone** ψ over the sub-steps — a smoke that Brent is wired, **not** a bit-for-bit equality (Brent $\neq$ `NewtonRaphsonStable`, so `EXPECT_EQ` would be wrong here).
- `seas_test_seed_equilibrium_psi_rs` (**R-001 — extends the seed test**): for a synthetic DOF (`tau0=29.2e6`, `$\sigma_{n0}=49.27e6$` , `V_init=1e-9`, `a=0.010`, `Dc=2.0`, `eta=0.5 $\cdot$ sqrt( $\mu\rho$ )`, global `b=0.015`, `V0=1e-6`, `f0=0.6`), after `SeedEquilibriumPsi_RS` the steady residual `|tau0 - ( $\sigma_{n0} \cdot a \cdot \text{asinh}[V_{init}/(2V_0) \cdot \exp(\psi/a)] + \eta \cdot V_{init})| < 1e-6 \cdot \tau_{00}$` . This guards **both** the member name (`d.eta_s`, not `rs.eta_s`) **and** the seed $\leftrightarrow$ solve η consistency. A second case with `tau0=0` asserts `$\psi == f_0 + b \cdot \ln(V_0/V_{init})$` (locked branch), and a `V_init=0` case asserts the new `MFEM_VERIFY` aborts.
- **R-009**: in the same target, a `blk.V_0_default = 2e-6` ($\neq$ `FrictionSolver::V0 = 1e-6`) case asserts `SeedEquilibriumPsi_RS` **aborts** on the `V_0` guard; the control `blk.V_0_default = 1e-6` passes and (optionally) verifies a `t=0` solve sits at `V_init` (`|V - V_init|  $\leq$  1e-6 $\cdot$ V_init`) while `2e-6` *without* the guard would be `O(1)` off (documents the silent-wrong behaviour the guard prevents).
- `make -n seas_spatial_dyn_driver` shows `$(TPV102_SUBSTEP_ITERATOR_OBJ)` on the link line; all three new/extended test targets resolve.

Dependencies

Depends on: nothing. Required by: Phases 2, 3.

9.2 Phase 2 – Friction strategy interface » (IFrictionIterator + adapters + factory)

Goal

A minimal IFrictionIterator strategy interface with two adapters over the **existing, proven** iterators (LSW→Tpv205SubStepIterator, aging→Tpv102SubStepIterator) and a MakeFrictionIterator factory; AdvanceADERWithSubStep_Spatial takes IFrictionIterator&. **The existing LSW path is byte-unchanged** (the adapter is a transparent forwarder). This is the runtime-dispatch foundation; the full method-oriented unification is Phase 5.

Files to Create

- dynamic/friction_iterator.hpp — IFrictionIterator + LswFrictionIterator + RateStateAgingFrictionIterator
- dynamic/friction_iterator_factory.{hpp,cpp} — MakeFrictionIterator.
- tests/unit/test_friction_iterator_factory.cpp.
- tests/unit/test_advance_interface_compiles.cpp (**R-010** — compile-only: the retyped AdvanceADERWithSubStep_Spatial signature + a stub IFrictionIterator whose only sub-step entry is Advance(...) must compile).

Files to Modify

- drivers/spatial_dyn_driver.cpp — change AdvanceADERWithSubStep_Spatial's iterator parameter (:367) from Tpv205SubStepIterator& to IFrictionIterator&; **R-010**: rename the body's iterator.AdvanceWithSubStepStates(...) call at :439 to iterator.Advance(...) (the only call that changes; GetDeltaT/GetTimeWeights/SetSubSteps are unchanged). Replace the concrete Tpv205SubStepIterator substep_iterator(fault_flux); construction (:1736) with auto fr = std::make_unique<LswFrictionIterator>(fault_flux); and pass *fr. (The RS branch is Phase 3; here only LSW is wired, so behaviour is identical.)

Detailed Requirements

1. IFrictionIterator (minimal; SRW arrives in Phase 5):

```
class IFrictionIterator {
public:
    virtual void SetSubSteps(std::vector<real_t> dT,
                           std::vector<real_t> w) = 0;
    // R-010: AdvanceADERWithSubStep_Spatial's body also reads these two
    // (driver :384/:385/:404), so they MUST be on the interface or the
    // parameter retype (req 5) will not compile.
    virtual const std::vector<real_t>& GetDeltaT() const = 0;
    virtual const std::vector<real_t>& GetTimeWeights() const = 0;
    virtual void Advance(
        std::vector<DOFData>& dof, const std::vector<mfem::Vector>& coords,
        const std::vector<std::vector<real_t>>& Qp,
        const std::vector<std::vector<real_t>>& Qm,
```

```

    real_t dt, real_t t0, real_t* Iimp_p, real_t* Iimp_m,
    const std::function<void(real_t,real_t)>& nuc_cb) = 0;
virtual void SetDiagNumLocalFaultQPs(int n) = 0;
virtual FaultFrictionLaw WaveOpLaw() const = 0;
virtual ~IFrictionIterator() = default;
};

```

R-010 note: the reviewer flagged only the :439 call rename, believing GetDeltaT/GetTimeWeights were already on the interface — they were **not** in the rev-5 draft. Both Tpv205SubStepIterator and Tpv102SubStepIterator already expose them (tpv205...hpp:138–139, tpv102...hpp:130–133), so the adapters forward trivially; without them on the interface, the retype breaks at :384/:385/:404 too. They are added above.

2. **LswFrictionIterator** owns a Tpv205SubStepIterator `it_;` (ctor takes FaultFaceFlux&). `Advance(...)` forwards to `it_.AdvanceWithSubStepStates(dof, coords, Qp, Qm, dt, t0, Iimp_p, Iimp_m, nuc_cb)` (Tpv205's existing callback overload). `WaveOpLaw()` → `FaultFrictionLaw::LSW.SetSubSteps/GetDeltaT/GetTimeWeights/SetDiagNumLocalFaultQPs` forward to `it_`.
3. **RateStateAgingFrictionIterator** — **member order matters** (the iterator holds a `const AgingLawPsi&`, so the law must be declared/constructed first):

```

class RateStateAgingFrictionIterator : public IFrictionIterator {
public:
    RateStateAgingFrictionIterator(FaultFaceFlux& flux,
                                   const spatial::RateStateBlock& blk)
        : law_(blk.b_default, blk.V_0_default, blk.f_0_default),
          it_(flux, law_) {}
    void Advance(/*...*/,
                 const std::function<void(real_t,real_t)>& nuc_cb) override
    { it_.AdvanceWithSubStepStates(
        /*...*/, FrictionSolver::Method::Brent, nuc_cb); }
    FaultFrictionLaw WaveOpLaw() const override
    { return FaultFrictionLaw::RateAndState; }
    /* SetSubSteps / GetDeltaT / GetTimeWeights / SetDiagNumLocalFaultQPs
       forward to it_ */
private:
    AgingLawPsi          law_;    // declared BEFORE it_
    Tpv102SubStepIterator it_;
};

```

It uses the Phase-1 `nuc_callback` overload and `FrictionSolver::Method::Brent`. **R-009:** `law_(blk.b_default, blk.V_0_default, blk.f_0_default)` makes `AgingLawPsi` read the config `V_0`, while the force solve hardcodes `FrictionSolver::V0` — the Phase-1 `SeedEquilibriumPsi_RS` guard (and, defensively, `MakeFrictionIterator`) asserts they agree.

4. **MakeFrictionIterator:**

```
std::unique_ptr<IFrictionIterator> MakeFrictionIterator(
    const spatial::SpatialFrictionConfig& cfg, FaultFaceFlux& flux,
    const spatial::RateStatePerDOFParams* rs /*nullptr for LSW*/);
```

cfg.law==SlipWeakening → make_unique<LswFrictionIterator>(flux); cfg.law==RateState → require cfg.rate_state.has_value(), then make_unique<RateStateAgingFrictionIterator>(flux, *cfg.rate_state). **SRW guard:** if a future state_evolution==slip_law_srw field is set, abort with "slip-law SRW lands in Phase 5" (aging is the only RS law wired in Phases 1–3). **R-009 defensive guard:** on the RateState branch, also MFEM_VERIFY(|cfg.rate_state->V_0_default - FrictionSolver::V0| ≤ tol, ...) (same invariant as the Phase-1 seed guard — cheap, and catches a mis-set V_0 before the iterator is built).

5. **AdvanceADERWithSubStep_Spatial** parameter retype: IFrictionIterator& iterator (was Tpv205SubStepIterator&, :367). **R-010 (CRITICAL for build):** the body does **not** already call iterator.Advance(...) — at :439 it calls iterator.AdvanceWithSubStepStates(dof_data, fault_coords, Q_pointwise_plus, Q_pointwise_minus, dt_step, t_step_start, I_imp_plus_flat.data(), I_imp_minus_flat.data(), nuc_callback). **Rename that call to iterator.Advance(...)** with the identical argument list (the interface names the entry point Advance). The body's other iterator calls — GetDeltaT() (:384), GetTimeWeights() (:385/:404), SetSubSteps() (:403) — are now all on the interface (req 1) and need no change. No other logic change; all safes fixes stay.

Edge Cases

- Member-init order in RateStateAgingFrictionIterator (law_ before it_) — a reversed order dangles the const AgingLawPsi&; enforce via the declared order above.
- SRW requested in Phases 1–3 → clean "Phase 5" abort, not a silent fallback.
- **R-010:** the interface must declare **every** method the driver body invokes on iterator (SetSubSteps, GetDeltaT, GetTimeWeights, Advance, SetDiagNumLocalFaultQPs) — a missing one is a compile error at the retype, not a runtime issue.

Acceptance Criteria

- ☐ seas_test_friction_iterator_factory: SlipWeakening → LswFrictionIterator (WaveOpLaw()==LSW); RateState(aging) → RateStateAgingFrictionIterator (WaveOpLaw()==RateAndState); SRW → abort.
- ☐ **R-010:** seas_spatial_dyn_driver **compiles** after the retype — i.e. the :439 call is Advance(...) and the interface declares GetDeltaT/GetTimeWeights. test_advance_interface_compiles is the minimal compile-only guard (pre-fix: fails because the call site uses AdvanceWithSubStepStates, absent from the interface).
- ☐ **SAFS LSW no-regression:** an existing SAFS LSW config runs **byte-identical** to a pre-Phase-2 checkpoint (the adapter forwards to the same Tpv205SubStepIterator).
- ☐ seas_spatial_dyn_driver builds and the LSW smoke runs unchanged.

Dependencies

Depends on: Phase 1. Required by: Phase 3.

9.3 Phase 3 – SAFS + rate-and-state run » (driver RS branch + D3.1 + config) – THE NEAR-FUTURE GOAL

Goal

spatial_dyn_driver runs the **SAFS Dc2 problem with aging rate-and-state friction** from a TOML; an 8-rank smoke produces a physical RS rupture. Also lands the D3.1 right-lateral-positive constant_tensor convention (bit-identical on-fault). **This phase delivers the near-future goal.**

Files to Modify

- drivers/spatial_dyn_driver.cpp — remove the is_lsw reject; add the RS resolver/DOF-init branch (**R-002**: gate the :1146 slip_weakening.has_value() guard under is_lsw, outer-scope lsw+rs); D3.1 factory negation at the ConstantTensorStressSource construction (:1123–1135); SetFaultFrictionLaw from is_lsw; **R-005**: branch the ParaView “state” channel (:1809) so RS writes d.psi.
- spatial/code/spatial_friction.cpp — **R-006**: in resolve_rs_impl’s per-rule loop (:1014/1017/1018), **reject** per-DOF b/V_0/f_0 spatial overrides (the aging law and DOFData are scalar in those), so the plan’s “rejected by ResolveRateState” claim holds. **R-011**: relax the MFEM_VERIFY(a_i < b_i) validator at :1070 to **allow** a > b (velocity-strengthening) — keep only a>0, b>0, finite. VS regions are how aging-law ruptures arrest at the fault edges (TPV102 itself uses a high-a border).
- safs/project_7.0_alternative/config/spatial_friction_slip_weakening_safs_projected_stress*.toml (Dc2/Dc8/Dc10) — flip sigma_xy_pa -9.888...e6 → +9.888...e6 **together with** the D3.1 negation, **in one atomic commit** with the factory change (**R-007**; net on-fault unchanged).

Files to Create

- safs/project_7.0_alternative/config/spatial_friction_rate_state_safs_projected_stress_resolution.toml
- jobs/safs/spatial_dyn_resolution_Dc2_ratestate_8N_400r_dev_2hr_safs.sbatch
- tests/unit/test_constant_tensor_sign.cpp (**R-007** — D3.1 golden bit-identity, **parametrized over Dc2/Dc8/Dc10 + the RS config**)
- tests/unit/test_resolve_rate_state_guards.cpp (**R-003** no-double-PP + **R-006** per-DOF b/V_0/f_0 rejection + **R-011** a > b velocity-strengthening allowed)
- tests/unit/test_paraview_state_channel_rs.cpp (**R-005** — RS state channel writes ψ)

Detailed Requirements

1. **Remove the reject** at spatial_dyn_driver.cpp:716–719 (MFEM_VERIFY(is_lsw, "...rate_state path is a deferred follow-up...")). Keep the `cfg.law ∈ {SlipWeakening, RateState}` verify (:710–714) and is_lsw (:715).
2. **Friction-law tag** at :971: replace the hard-coded `wave.SetFaultFrictionLaw(FaultFrictionLaw::LSW)` with `‘wave.SetFaultFrictionLaw(is_lsw ? FaultFrictionLaw::LSW : FaultFrictionLaw::RateAndState);(is_lsw‘ already in scope)`.
3. **Resolver branch (R-002 + R-003)** — replaces driver :1146–1152. The **un-gated** MFEM_VERIFY(cfg.slip_weakening) at :1146 **must move inside if (is_lsw)**: the parser forbids a [friction.slip_weakening] block when law="rate_state" (spatial_friction.cpp:809–814), so an RS config has `cfg.slip_weakening == nullopt` and the literal :1146 guard would **abort the SAFS-RS run**

during setup (wrong message, before nucleation/DOF-init). Both result structs are declared at **outer scope** so the DOF-init at :1220 (req 4) still sees them:

```

spatial::SpatialFrictionResolver    resolver;
spatial::SlipWeakeningPerDOFParams  lsw; // outer scope; filled iff is_lsw
spatial::RateStatePerDOFParams      rs;  // outer scope; filled iff !is_lsw
if (is_lsw) {
    MFEM_VERIFY(cfg.slip_weakening.has_value(), // R-002: now gated
                "law=slip_weakening requires [friction.slip_weakening]");
    lsw = resolver.ResolveSlipWeakening(*cfg.slip_weakening,
                                        dof_coords_3d, dof_to_attr);
} else {
    MFEM_VERIFY(cfg.rate_state.has_value(),
                "law=rate_state requires [friction.rate_state]");
    rs = resolver.ResolveRateState(
        *cfg.rate_state, dof_coords_3d, dof_to_elem, dof_to_attr,
        material, pmesh,
        spatial::PorePressureSpec{}, // R-003: NOT cfg.stress.pore_pressure
        geom.sigma_n_per_dof());    // sigma_n_per_dof() is ALREADY effective
} // input already effective; non-zero pp would double-subtract P_p

```

R-003 (double pore-pressure): ResolveRateState’s last argument is named `sigma_n_total_per_dof` and it subtracts `pp` internally (`spatial_friction.cpp:1024–1039`) to fill `rs.sigma_n_eff`. But `geom.sigma_n_per_dof()` is **already** effective — it comes from `ProjectFaultPreStress`, “the standard effective normal stress $\sigma_n - P_p$ ” (`field_coefficient.hpp:231`). Passing `cfg.stress.pore_pressure` (Dc2: $P_p=16$ MPa) here would give `rs.sigma_n_eff` = total – $2 \cdot P_p$ (≈ 33 MPa vs the correct ≈ 49 MPa). It is *latent* today (no consumer reads `rs.sigma_n_eff`; physics uses `d.sigma_n0`), but a real trap and it silences the internal `sigma_n_eff > 0` validator — so pass a **zero** `PorePressureSpec{}` and document it at the call site. (verbatim signature, `spatial_friction.hpp:306–326`.)

4. **DOFData init branch:** `is_lsw` → existing IP-aware `InitializeFaultDOFs_Spatial<ParMesh>(...)` (:1220–1224); else

```

InitializeFaultDOFs_Spatial_RS<ParMesh>(
    dof_data, ndof, dof_to_elem, material, pmesh, rs,
    geom.GetTauPre(), geom.sigma_n_per_dof()); // sig: spatial_setup.hpp:301
SeedEquilibriumPsi_RS(dof_data, rs, *cfg.rate_state); // Phase-1 helper

```

Note: `InitializeFaultDOFs_Spatial_RS` has **no IP-aware overload** (uses element centroid). SAFS material is **homogeneous**, so centroid $\equiv$ IP — correct for SAFS; flag an IP-aware RS overload as a heterogeneous-material follow-up (needed only if RS meets a spatially-varying material, i.e. not in this plan).

5. **Iterator + nucleation** (replace the Phase-2 LSW-only construction at :1736): `auto fr = MakeFrictionIterator(cfg, fault_flux, is_lsw ? nullptr : &rs); fr->SetSubSteps(deltaT, weights); fr->SetDiagNumLocalFaultQPs(wave.GetNumLocalFaultQPs()); Nucleation is`

reused unchanged — the existing ResolveGradualOverstress (:1162–1167) and the nuc_cb lambda (:1753–1761) are law-agnostic and flow through AdvanceADERWithSubStep_Spatial→fr→Advance→aging adapter→the Phase-1 Tpv102 overload.

6. **D3.1 factory negation** at the ConstantTensorStressSource construction (:1123–1135):

```
spatial::ConstantTensorStressSource src(
    cfg.stress.sigma_xx_pa, cfg.stress.sigma_yy_pa, cfg.stress.sigma_zz_pa,
    -cfg.stress.sigma_xy_pa,          // D3.1: right-lateral-+ input
    +cfg.stress.sigma_yz_pa, +cfg.stress.sigma_xz_pa);
```

with a documented comment block (normals compression-positive; strike shear right-lateral-positive; $\tau_{\text{strike}}=+\sigma_{xy_pa}$). **R-007 (atomicity)**: the factory negation and the σ_{xy_pa} flip in **all** affected configs (Dc2/Dc8/Dc10 **and** the new RS config) must land in **one commit** — a half-applied change (factory flipped but a config not, or vice-versa) silently inverts the on-fault shear *direction* on the un-flipped config. The parametrized golden test (req 8) guards every flipped config so a partial application fails loudly. *(D3.1 is net-zero physics — kept in Phase 3 per the user’s standing decision; the review’s alternative was to defer it to Phase 4, but the atomic-commit + all-config golden test fully closes the partial-application hazard, so it stays.)*

7. **SAFS-RS config** — clone the LSW Dc2 config; set [meta].law = "rate_state"; add [friction.rate_state] (a_default, b_default, Dc_default, V_0_default, f_0_default, V_init_default, sigma_n_default, eta="auto" — all already parsed, spatial_friction.cpp:388–466); keep [stress] constant_tensor with the post-D3.1 positive σ_{xy_pa} ; keep [nucleation] gradual_overstress (Gaussian). Do **not** set interior_flux (the driver defaults to the scalar WaveOperator ctor; the matrix option is Phase 9). Per-DOF a/Dc [...spatial]] rules are allowed; per-DOF b/V_0/f_0 are **rejected by ResolveRateState** (the aging law + DOFData are scalar in those — see req 11, **R-006**: the rejection does not exist today and is added in this phase). **R-009**: keep V_0_default = 1e-6 (= FrictionSolver::V0); any other value trips the Phase-1 seed guard. **R-011**: the first SAFS+RS smoke is **fully velocity-weakening** (a < b everywhere → a_default = 0.010, b_default = 0.015), which the existing validator permits and which runs to the mesh boundary (does not arrest); a TPV102-faithful **bounded** rupture adds a velocity-strengthening (a > b) border, which requires the req-12 validator relaxation.
8. test_constant_tensor_sign.cpp (D3.1, **R-007 — parametrized over {Dc2,Dc8,Dc10, rate_state}**): for **each** flipped config, build its stress block via the driver’s factory construction path (so the negation is exercised), project onto a canonical-frame DOF, assert $\tau_{\text{strike}}=+\sigma_{xy_pa}$ (right-lateral positive), $\sigma_n=+\sigma_{yy_pa}$, $\tau_{\text{dip}}=+\sigma_{yz_pa}$, **bit-identical to the pre-change golden literals** captured from each current config (EXPECT_NEAR(...,0.0)); precondition dip==(0,0,-1), strike==(+1,0,0). This is what makes the multi-config flip safe — every flipped config is golden-guarded, not just Dc2.
9. sbatch sibling of the LSW Dc2 job (8N/400r dev, 2 h) with SAFS_CONFIG → the new RS config; keep the diagnostic env toggles.
10. **ParaView RS state channel (R-005)** at spatial_dyn_driver.cpp:1809: the snapshot writer

hard-codes the “state” field as an LSW friction coefficient `LSWFrictionCoefficient_TPV205(delta_norm, d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c)`. `InitializeFaultDOFs_Spatial_RS` never sets `lsw_mu_s/mu_d/d_c`, so for RS they are 0 and the channel is 0 (or NaN at `delta==0`, step 0) — the very field that should let the user watch the RS state is garbage. Branch it (`is_lsw` is in scope):

```
if (is_lsw) {
    pv_local_state(i) = mfem::seas::LSWFrictionCoefficient_TPV205(
        delta_norm, d.lsw_mu_s, d.lsw_mu_d, d.lsw_d_c);
} else {
    pv_local_state(i) = d.psi;    // RS state var (matches tau*_corr solve)
}
```

Writing ψ is the cheapest correct choice (the RS friction coefficient is redundant with `slip_rate/tau*_corr`). The LSW branch is byte-identical to today.

11. **Reject per-DOF `b/V_0/f_0` in `ResolveRateState` (R-006)** — `spatial/code/spatial_friction.cpp`, in `resolve_rs_impl`’s per-rule loop (where it currently *applies* them at `:1014/1017/1018`). The aging law is built from the **global** `blk.b_default/V_0_default/f_0_default` (Phase-2 ctor) and `DOFData` has no per-DOF `b/V_0/f_0` slots, so a per-DOF override is silently dropped by every physics path — a “knob that does nothing.” Add an abort:

```
MFEM_VERIFY(std::isnan(r.b) && std::isnan(r.f_0) && std::isnan(r.V_0),
    "ResolveRateState: per-DOF b/f_0/V_0 spatial overrides are not "
    "supported (AgingLawPsi is scalar); set them only in the "
    "[friction.rate_state] defaults. Offending rule at DOF " << i);
```

(Place it before the `if (!std::isnan(r.b))...` assignments, or drop those assignment lines and the unused `rs.b/f_0/V_0` members entirely after a grep confirms no consumer — the abort is the minimal guard for the deliverable. Per-DOF `a/Dc/V_init/sigma_n/eta` stay allowed.)

12. **Relax the `a < b` validator (R-011)** — `spatial/code/spatial_friction.cpp:1070`. Replace `MFEM_VERIFY(a_i < b_i, "ResolveRateState: a >= b at DOF " << i)` so that velocity-strengthening (`a > b`) is **allowed**; keep the positivity/finiteness checks (`a_i > 0, b_i > 0`):

```
MFEM_VERIFY(a_i > 0.0, "ResolveRateState: a <= 0 at DOF " << i);
MFEM_VERIFY(b_i > 0.0, "ResolveRateState: b <= 0 at DOF " << i);
// R-011: a > b (velocity-strengthening) is allowed -- it is how aging-law
// ruptures arrest at the fault edges (TPV102 uses a high-a border). Only
// require a, b finite and > 0; do NOT require a < b.
```

Scope: the first deliverable smoke is fully VW (req 7), so this is the **only** round-2 finding strictly *optional* for the immediate run. It is included in Phase 3 because it is a 2-line change that directly enables the user’s “tpv102-like” (bounded) target and removes a setup-abort trap; landing it now avoids a second touch of `ResolveRateState`. Do **not** remove the positivity checks. (*Pre-existing validator — not introduced by this plan; relaxing it is the documented prerequisite for any bounded SAFS-RS rupture.*)

Edge Cases / Risk gate

- **Shared-fault RS ψ is 1st-order in time at $\text{ader_order} \geq 2$ (top risk; review R-004).** For shared (rank-seam) fault QPs the driver runs **inline** EvaluateADER at macro dt (`wave_operator.inl:4908`), *not* the per-sub-step iterator (the R-1601 fallback). The iterator advances `d.psi` for **all** QPs incl. shared (`tpv102...cpp:348`), then the inline shared flux reads that **end-of-step** ψ for the whole macro step and EvaluateADER does **not** mutate ψ (`assert fault_face_flux.cpp:422`). Net: interior QPs get a sub-step-consistent ψ trajectory; shared QPs use end-of-step $\psi \rightarrow$ **1st-order on shared faces** (SAFS default $\text{ader_order}=2$). This is **bounded, not a blow-up**: it is structurally the *same* as the **working** LSW slip path (the iterator owns slip; inline EvaluateADER_LSW reads end-of-step slip and does not re-accumulate, `fault_face_flux.cpp:815–824`). The reconcile **already broadcasts `d.psi`** (`payload[5], :5304/:5327`), so cross-rank ψ is single-valued — the issue is **temporal accuracy, not cross-rank desync and not a double-advance**. Added risk over LSW: ψ enters friction nonlinearly ($\exp(\psi/a)$), so the end-of-step approximation is less benign than slip $\rightarrow \mu_{\text{eff}}$. **Gate (R-004):** keep RS $\text{np} > 1$ “open until validated”; the consistency check is now **two-pronged** — (a) cross-rank `d.psi` bit-identity post-reconcile, **and (b) a temporal interior-vs-shared ψ drift** at the same physical QP within tolerance (see the $\text{np}=2$ test). If (b) exceeds tolerance, the documented follow-up is to drive shared QPs through a sub-step ψ path. **Do not present 8-rank SAFS-RS results as validated until (a)+(b) pass.**
- Equilibrium ψ at $\tau_0=0 \rightarrow$ locked steady state (Phase-1 helper handles it).
- D3.1 half-applied (factory negation XOR a config flip) $\rightarrow$ the parametrized `test_constant_tensor_sign` (req 8) golden mismatch fails loudly on the un-flipped config.

Acceptance Criteria

- **R-002:** the new RS SAFS config (no `[friction.slip_weakening]` block) **does not abort** at the `slip_weakening.has_value()` guard. `test_rate_state_config_no_abort`: run the driver setup path to just past DOF-init in `--dry-run`; assert it reaches the law: `rate_state` banner and constructs `dof_data` (pre-fix: aborts at `:1146`).
- SAFS-RS Dc2 `--dry-run --verify-dispatch` reports `RateAndState` + the aging iterator + Gaussian nucleation.
- **R-003:** `test_resolve_rate_state_no_double_pp` (in `test_resolve_rate_state_guards.cpp`): $N=1$ DOF, `sigma_n_total_per_dof={49.27e6}` (already effective), `pp=PorePressureSpec{}` $\rightarrow$ `rs.sigma_n_eff(0)==49.27e6` (1 ULP); control with `pp.P_p_pa=16e6` yields `33.27e6` (documents the trap).
- **R-006:** `test_resolve_rate_state_rejects_per_dof_b` (same file): a `RateStateBlock` with one spatial rule `{kind=Depth, b=0.02}` $\rightarrow$ `ResolveRateState` aborts with the “per-DOF `b/f_0/V_0` not supported” message; control rule setting only `a/Dc` resolves cleanly.
- **R-011:** `test_resolve_rate_state_allows_velocity_strengthening` (same file): one DOF with `a=0.02, b=0.015` ($a > b$) $\rightarrow$ `ResolveRateState` does **not** abort; control `a<=0` still aborts. Pre-fix: the $a < b$ MFEM_VERIFY rejects the $a > b$ case.
- **R-005:** `test_paraview_state_channel_rs`: after one RS macro-step, `pv_local_state(i)==dof_data[i].psi` for all `i` and finite at step 0 (`delta==0`); the LSW branch is byte-identical to the pre-change writer.
- **R-007:** `seas_test_constant_tensor_sign` passes for **all** of `{Dc2,Dc8,Dc10, rate_state}`;

an existing SAFS **LSW** run is byte-identical after the `sigma_xy_pa` flip (D3.1 net-zero).

- **R-004:** `test_shared_fault_rs_psi_consistency_np2` (MPI `np=2`, one shared fault QP): after one macro-step (`ader_order=2`), (a) `(*fault_dof_data_)[shared].psi` is bit-identical on both ranks post-reconcile; (b) $|\psi_{\text{shared}} - \psi_{\text{interior_twin}}| \leq \text{tol}$ (`twin` = same physical QP through the interior sub-step path); record `tol` in the benchmark doc.
- » **Working SAFS+RS:** an 8-rank short smoke runs — pre-nucleation the fault sits at $V \approx V_{\text{init}}$ (locked steady state from the ψ seed); nucleation triggers an RS rupture; `max_slip` is physical; the R-004 (a)+(b) checks pass (or the shared-fault-RS follow-up is opened and `np>1` RS is **not** presented as validated).

Dependencies

Depends on: Phases 1, 2. Required by: Phases 4–10 build on the working driver.

9.4 Phase 4 – Name generalization (D-rename)

Goal

`FaultGeometry::ComputeParams/HasParams` is the project-wide name; no behaviour changes. (Deferred behind the SAFS+RS goal because it is purely cosmetic — the RS path in Phases 1–3 uses the existing `ComputeSAFSParams` name.)

Files to Modify

- `fault/fault_geometry.hpp`, `fault/fault_geometry_safs.inl`, `fault/rate_state_fault.hpp`, `config/seas_config.hpp`, `config/seas_config_bridge.hpp`, `spatial/code/spatial_stress.{hpp,cpp}`, `drivers/spatial_dyn_driver.cpp`, `tests/unit/test_compute_safs_params.cpp`

Detailed Requirements

1. Mechanical rename `ComputeSAFSParams` → `ComputeParams`, `HasSAFSParams` → `HasParams`, `safs_params_computed_` → `params_computed_` across all files (incl. the call site `seas_config_bridge.hpp:147` and the driver `:1131–1140`). Fault kind stays informational.
2. **Do not** add the reference R-001 sign flip — the no-flip projection rule and `SEAS_ZERO_DIP_PRESTRESS` stay verbatim (nothing reverted; see §2-D3.1).

Acceptance Criteria

- ☒ `grep -rn 'ComputeSAFSParams|HasSAFSParams' miniapps/seas --include='*.hpp' --include='*.cpp' --include='*.inl' --include='*.toml' --include='*.py' --include='Makefile'` returns nothing. (Per user scope decision 2026-05-27 — REVIEW R-002: 16 historical `.md` docs, **including this plan**, intentionally retain the old names; renaming the plan that documents the rename would make it self-referentially wrong. The `grep` is therefore scoped to code + active source, where it returns nothing.)
- ☒ `seas_test_compute_safs_params` (incl. T_65_5 templated==sidecar) passes unchanged after the rename (13/13; T-65-4 numerics byte-identical). The SAFS+RS path is additionally covered green by `seas_test_seed_equilibrium_psi_rs` (21/21), `seas_test_tpv102_nuc_callback_parity` (18/18), `seas_test_safs_mode_wiring`, and `seas_test_spatial_setup` (71/71); a serial `--print-derived` RS-dispatch run is the runtime smoke (the full MPI smoke runs on the cluster per the `jobs/safs/*.sbatch`).

Dependencies

Depends on: Phase 3. Required by: nothing (independent cleanup).

9.5 Phase 5 – Unified method-oriented iterator (replaces the Phase-2 adapters)

Goal

The three benchmark-named iterators collapse into the method-oriented set (LinearSlipWeakeningIterator, RateStateSubStepIterator<RateStateAgingPolicy>, RateStateSubStepIterator<RateStateSlipLawSrwPolicy>) sharing one RunSubSteps_ skeleton behind IFrictionIterator; the Phase-2 adapters are re-pointed to the unified classes, parity-tested bit-for-bit against the standalone oracle. Adds the SRW law.

[!] PLAN DEVIATION (2026-05-27, user-approved) — RunSubSteps_/step_fn split corrected. Reading the three oracle iterators in full at implementation time found that the §5.1 / req-1 premise — “*the envelope is byte-for-byte identical across all three*” and “*RunSubSteps_ performs the slip accumulation*” — is **inaccurate**. Two concrete mismatches: 1. **Slip-accumulation location differs.** LSW accumulates slip **inside** StepOneQP_ (tpv205_substep_iterator.cpp:129–130), whereas the RS iterators accumulate in the per-QP loop body (tpv102_substep_iterator.cpp:345–346). If RunSubSteps_ also did the slip accumulation while LinearSlipWeakeningIterator lifts StepOneQP_ *verbatim*, slip would be **double-counted** on the LSW path. 2. **LSW writes two DOFData diag fields the RS path does not.** tpv205_substep_iterator.cpp:413 (d.slip_rate_substep_max = std::max(...)) and :417 (d.sigma_n_substep_min = std::min(...)) run **unconditionally** in the LSW envelope (only the [SLIP] fprintf is env-gated). The RS iterators have no such writes. To stay bit-exact with standalone tpv205 these must be reproduced on the LSW path, but they would be wrong on the RS path.

Resolution (this is what is implemented): SubStepIteratorBase::RunSubSteps_ owns **only** the *truly* common envelope — the SetSubSteps validator, the Advance preconditions, the Σ deltaT == dt_macro check, memset(I_imp_*,0), the per-substep loop with nuc_fn(t_sub_end,dt_sub) + running t_sub_end, the per-(o,i) Q± slice from Q_pointwise[o], and the I_imp += accum_scale*Q_imp accumulation. **Slip accumulation, the ψ /LSW solve, the LSW-only diag writes, and WriteBack-State all move into a richer per-QP step_fn** with signature step_fn(i, d, Qp_i, Qm_i, dt_sub, t_sub_end, last_sub_step, Q_imp_p, Q_imp_m) (the iterator’s fault_coords/method_/law_/V_w_/diag_num_local_fault_qps_ are captured). This still collapses the triplicated envelope into one skeleton and keeps each step_fn a verbatim lift of the oracle’s per-QP body, achieving the bit-exactness goal — it only moves the slip-accumulation responsibility out of RunSubSteps_. Requirement 1 below is amended accordingly.

Files to Create

- dynamic/friction_substep_iterator.{hpp,cpp} — SubStepIteratorBase (skeleton), RateStateSubStepIterator<StatePolicy>, LinearSlipWeakeningIterator, the two using aliases; explicit template instantiations of both policies in the .cpp.
- friction/state_policies.hpp — RateStateAgingPolicy, RateStateSlipLawSrwPolicy

(§5.1).

- tests/unit/test_friction_substep_iterator_parity.cpp.

Files to Modify

- dynamic/friction_iterator_factory.cpp — re-point MakeFrictionIterator to the unified classes (the RateStateAgingFrictionIterator/LswFrictionIterator shells either wrap the unified iterator or are replaced by the unified classes implementing IFrictionIterator directly); extend Advance to the full signature and add the SRW branch.
- **None of the standalone `tpv{205,102,104}_substep_iterator.*` (oracle, untouched).**

Detailed Requirements

1. SubStepIteratorBase::RunSubSteps_ lifts the **common** envelope verbatim (the SetSubSteps validator, Advance preconditions, $\Sigma \text{ deltaT} == \text{dt_macro}$ check, `memset(I_imp*,0)`, the `per-(o,i) Q $\pm$  slice from Q_pointwise[o]`, the `accum_scale = weight*dt_macro` accumulation `I_imp += accum_scale*Q_imp`, and `nuc_fn(t_sub_end,dt_sub)` once per sub-step with running `t_sub_end`, before the QP loop). **Per the 2026-05-27 deviation above, slip accumulation and WriteBackState are NOT in RunSubSteps_** — they live in `step_fn` (LSW does both inside the lifted `StepOneQP_`; RS does slip accumulation in its `step_fn` body and `WriteBackState` on the last sub-step). See §5.1 for the full structure.
2. RateStateSubStepIterator<StatePolicy> per-QP `step_fn`: `flux_.ComputeStageState(d,Qp,Qm,s,method)` $\rightarrow$ slip accumulation $\rightarrow$ `d.psi = StatePolicy::UpdatePsi(law_, d, s.V_abs, dt_sub, extra_, i)` $\rightarrow$ `flux_.BuildImposedState(...)`. Aging policy $\rightarrow$ `UpdateStateAnalytic(...)`; SRW policy $\rightarrow$ `UpdateStateAnalyticSlipLawSRW(..., V_w[i], ...)`. The SRW `V_w` span is owned by the iterator (driver-supplied from `rs->V_w`).
3. LinearSlipWeakeningIterator `step_fn` lifts `Tpv205SubStepIterator::StepOneQP_` verbatim.
4. **Shared-rank invariant (§5.4-2):** `RunSubSteps_` operates on the **interior** fault-QP set only; shared fault QPs keep bypassing the iterator (inline `EvaluateADER`, R-1601).

Acceptance Criteria

- ☐ `seas_test_friction_substep_iterator_parity`: on a fixed multi-QP, $O \in \{1,2,3\}$ fixture, each unified iterator reproduces its standalone counterpart bit-for-bit (`D0FData + I_imp $\pm$`).
- ☐ After re-pointing the factory, the **SAFS+RS smoke and the LSW smoke are bit-identical** to their Phase-3 results.

Dependencies

Depends on: Phase 3. Required by: Phase 8 (TPV104 needs the SRW law added here).

9.6 Phase 6 – TPV/material/flux config schema + D3.2 fault-local stress

Goal

The TPV TOML vocabulary parses into `SpatialFrictionConfig`: the SRW state-evolution selector, `boxcar_taper` rules, `interior_flux/material.kind`, the TPV nucleation kinds, and the fault-local stress source (D3.2). (The aging RS block, the Gaussian nucleation, `constant_tensor`, and `ResolveRateState` already exist from Phases 1–3 / safs — this phase adds only the TPV/SRW/flux/material extensions.)

Files to Modify

- spatial/code/spatial_friction.{hpp,cpp}, spatial/code/spatial_stress.{hpp,cpp}, dynamic/spatial_setup.hpp

Detailed Requirements

1. New blocks: ProblemSpec{tag}, BoundarySpec{fault_attr,natural_attrs,absorbing_attrs}, FaultGeometrySpec{ref_normal=(0,-1,0),up=(0,0,1),kind}, HypocenterSpec{x,y,z,nucleation_radius}, MaterialKind{Constant,DepthProfile1D,SidecarHDF5}+MaterialSpec.
2. **D3.2 fault-local stress:** extend StressSourceKind (currently {ConstantTensor, SidecarHDF5}, spatial_friction.hpp:153) with FaultLocalPrestress. Add FaultLocalPrestressSource (spatial_stress.{hpp,cpp}) with constant background {tau_strike_pa, tau_dip_pa, sigma_n_pa} (right-lateral / compression POSITIVE) + optional rectangular-tau_strike patches (center_{x,y,z}_m NaN, half_{x,y,z}_m +inf, last-match-wins). Add FaultGeometry::ComputePara (tau_dip, sigma_n, P_p) assigning tau1_0=tau_dip, tau2_0=tau_strike, sigma_n0=sigma_n-P_p per DOF directly (no projection). (*D3.1 constant_tensor already lands in Phase 3.*)
3. [numerics]: parse cfl_safety∈{raw,dg} (default raw), fault_iterator∈{one-shot, substep} (default one-shot), interior_flux∈{scalar,matrix} (default scalar). **Mutual-exclusion guard:** mixed_flux only with interior_flux=="scalar"; matrix + mixed_flux aborts (R-1203 sibling); matrix requires material.kind!=Constant.
4. **SRW state evolution:** add StateEvolutionKind{AgingLaw,SlipLawStrongRateWeakening} (the selector does **not** exist today — aging is implicit); add RateStateBlock.{state_evolution,f_w_default, (default AgingLaw, so existing RS configs are unaffected), per-rule V_w, RateStatePerDOFParams.V_w; thread V_w through ResolveRateState (p.V_w filled per-DOF, like p.a).
5. boxcar_taper rule: extend SpatialRule::Kind (currently {Depth,Box,RegionAttribute,Barrier}, spatial_friction.hpp:177) with BoxcarTaper; add boxcar_center/half/trans_{x,y,z}_m, cohesion-taper fields, per-rule V_w; add BoxcarTaperFactor(...) and free fn SCECBoxcar(offset,half,trans). **R-114 guard:** per-rule mu_s>1e5 aborts (use kind="barrier"). SpatialRule exposes named members (r.a, r.b, r.V_w, ...) — no string field.
6. Nucleation config: extend NucleationKind (currently {GradualOverstress}, spatial_friction.hpp:219) with GradualOverstressCompactCircular (TPV102/104) and InstantaneousOverstressCircular (TPV31: center_{x,y,z}_m, radius_m, taper_m, delta_tau_pa); [nucleation] dispatches on the enabled [nucleation.<kind>] sub-block; absent [nucleation] (TPV205) → static.
7. Guards: [meta] schema_version==1 (exists); R-008 hypocenter-z (up[2]>0 ⇒ z≤0); string time forms ("0s","12s","auto"); [boundary] disjoint-set + fault_attr>0; [fault_geometry] unit-norm + non-parallel + R-003.

Acceptance Criteria

- ☒ seas_test_spatial_friction_config_phaser + seas_test_spatial_stress_with_patches pass. (Implemented as seas_test_spatial_friction_config 138/138 — incl. FLP-1..3 patch coverage — plus seas_test_compute_safs_params T-66/T-67 for the direct-seed / strike-override paths.)
- ☒ All existing SAFS LSW/RS TOMLs still parse (the new fields default to current behaviour — no regression): all 8 safs/.../config/*.toml round-trip OK through LoadSpatialFrictionConfig (3 RS, 5 LSW).
- ☐ All three TPV TOMLs parse with no abort — **DEFERRED:** the dedicated TPV TOMLs are

authored in Phases 7–10 (TPV205/102/104/31 drivers). The schema now *supports* them (fault_local_prestress + patches, the three nucleation kinds, boxcar_taper, material kinds, numerics selectors, the [problem]/[boundary]/[fault_geometry]/[hypocenter]/[material] blocks all parse).

9.6.4.1 Phase 6 status (2026-05-27) — COMPLETE (reqs 1–7) All seven detailed requirements implemented on system/spatial_dyn_driver: - req 1 — [problem]/[boundary]/[fault_geometry]/[hypocenter]/[material] blocks (commit caac616). req 2 — fault_local_prestress + patches (D3.2). req 3 — cfl_safety/fault_iterator/interior selectors + mutual-exclusion + matrix⇒non-Constant-material guard (completed with req 1). req 4 — SRW state-evolution + per-DOF V_w threading. req 5 — boxcar_taper rule + SCECBoxcar/BoxcarTaperFactor (commit d2ced8e). req 6 — compact-circular + instantaneous-circular nucleation kinds. req 7 — guards (schema_version, R-008 hypocenter-z, string time forms, [boundary] disjoint+fault_attr>0, [fault_geometry] non-degenerate + normalize-on-read + non-parallel; “R-003” in the req-7 line = the pre-existing no-double-PP guard in test_resolve_rate_state_guards).

9.6.4.1.1 req 5 scope clarification — boxcar_taper is CONFIG-ONLY this phase (review R-001, 2026-05-27) Justification for the deferral (decided during the REVIEW_phase456 /code-fix pass): req 5 specifies only the BoxcarTaper rule *kind*, its geometry/cohesion- taper *fields*, and the SCECBoxcar / BoxcarTaperFactor *helpers* — it does **not** specify how the resolver consumes the taper (the per-DOF blend formula and which parameters taper are unspecified). Implementing a specific blend now would invent unspecified physics; and req 6’s sibling new kinds (the nucleation methods) are likewise explicitly “config-only; applicator wired in a later phase”. For consistency and to avoid a silent wrong-result trap (the generic resolver loop would otherwise apply a matching boxcar_taper rule as a HARD region over the whole boxcar+transition footprint, dropping cohesion_inner/cohesion_outer), SpatialFrictionResolver::Resolve{SlipWeakening,RateState} now **explicitly reject** a boxcar_taper rule (MFEM_VERIFY abort with a “config-only this phase” message). The kind + helpers still parse and are unit-tested; the per-DOF taper-blend consumption is deferred to the phase that authors the TPV TOMLs needing it. Tests: resolver B-3/R-11 (reject), config BOX-1..4 (parse + factor).

Verification (all objects freshly recompiled vs the new spatial_friction.hpp; counts include the REVIEW_phase456 R-001/R-003 fixes): spatial_friction_config 142/142, friction_iterator_factory 14/14, friction_substep_iterator_parity 36/36, compute_safs_params 23/23, constant_tensor_sign 27/27, friction_depth_profile 23/23, phaseh_lsw_forced_rupture 47/47, resolve_rate_state_guards 10/10, spatial_setup 71/71, spatial_friction_resolver 100/100, spatial_print_derived 33/33, spatial_stress_bundle 5/5.

[!] Build-hygiene finding (not a code bug): the per-test .o rules in the Makefile do NOT list spatial_friction.hpp as a prerequisite, so changing that header does not trigger recompilation. This silently produced a struct- layout (ABI) mismatch in test_friction_iterator_factory until the stale .o was force-removed (same class as the Phase-5 tpv104.o staleness). All Phase-6 results above were obtained AFTER force-rebuilding the affected objects. Recommend a follow-up to add header prerequisites (or -MMD auto-deps) to the test .o rules; left out of Phase 6 scope as a build-system change.

Dependencies

Depends on: Phase 3. Required by: Phases 7, 8, 9, 10.

9.7 Phase 7 – nucleation_methods module

Goal

MakeNucleation(cfg,...) returns the correct method behind INucleationMethod; compact-circular (TPV102/104) and instantaneous-circular (TPV31) exist and match native math. (The driver keeps calling the Gaussian path directly for SAFS+RS until it adopts the factory here.)

Files to Create

- dynamic/nucleation_method.hpp, dynamic/nucleation_factory.{hpp,cpp}, tests/unit/test_nucleation

Files to Modify

- dynamic/spatial_nucleation.{hpp,cpp} (additive), drivers/spatial_dyn_driver.cpp (swap the inline ResolveGradualOverstress/nuc_cb for MakeNucleation).

Detailed Requirements

1. Add GradualOverstressCompactCircularSpec + ResolveGradualOverstressCompactCircular + ApplyGradualOverstressCompactCircularIncrement (SCEC $F(r)=\exp(r^2/(r^2-R^2))\cdot\text{smoothStep}$) — additive; leave the existing Gaussian ResolveGradualOverstress/ApplyGradualOverstress Increment (spatial_nucleation.{hpp,cpp}) untouched.
2. Add InstantaneousOverstressCircularSpec + ResolveInstantaneousOverstressCircular (one-shot cosine-tapered patch — TPV31): $\Delta\tau(r)=\Delta\tau_0$ for $r\leq R$, cosine-tapered to 0 over $R<r\leq R+\text{taper}$, 0 beyond; seeds tau{1,2}_nuc once at t=0. Math from hrs-ref TPV31.
3. Define INucleationMethod (§5.2) + concretes GaussianGradualOverstress, CompactCircularGradualOverstress, InstantaneousOverstressCircular (ApplyOnce seeds the patch; ApplyIncrement no-ops; IsPerSubStep()==false), StaticOverstress (TPV205 no-op). Factory maps cfg.nucleation.kind → concrete; absent [nucleation] → StaticOverstress.
4. Driver: replace the inline ResolveGradualOverstress (:1162) + nuc_cb lambda (:1753) with

```
auto nuc = MakeNucleation(cfg, dof_coords_3d, dof_basis); and nuc_cb=[&](real_t t,real_t dt){ nuc->ApplyIncrement(dof_data,t,dt);}; nuc->ApplyOnce(dof_data)
```

 after init for the one-shot kinds. The SAFS+RS Gaussian path is then routed through GaussianGradualOverstress (byte-identical to the inline call).

Acceptance Criteria

- ☒ seas_test_spatial_nucleation: compact-bell at $r=0(=1)$, $r=R(=0)$, mid; instantaneous at $r=0(=\Delta\tau_0)$, $r=R(=\Delta\tau_0)$, $r=R+\text{taper}(=0)$. (73/73 — T-N13..N17.)
- ☒ seas_test_nucleation_factory: each kind (and the absent-block case) returns the expected method. (16/16 — F-1..F-5.)
- [~] SAFS+RS smoke is **bit-identical** to its Phase-3 result after the Gaussian path moves behind the factory. PROVEN at the unit level (F-5: factory Gaussian path == inline resolve+apply, exact equality on params + accumulated tau_nuc) and by construction; the **end-to-end** smoke is pending the SAFS mesh artifact (gmsh pipeline + multi-rank run — deferred since Phase 4).

Dependencies

Depends on: Phase 6. Required by: Phase 8, 10.

9.7.6.1 Phase 7 status (2026-05-28) — reqs 1–4 COMPLETE (end-to-end SAFS+RS smoke pending SAFS mesh)

- **req 1** — GradualOverstressCompactCircularSpec + CompactBellFactor (SCEC Eq.13 $\exp(r^2/(r^2-R^2)))$ + ResolveGradualOverstressCompactCircular + ApplyGradualOverstressCompactCircular (strike-only, tau2_nuc; smoothStep telescoping + dS<=0 guard). Additive; Gaussian path untouched.
- **req 2** — InstantaneousOverstressCircularSpec + CosineTaperFactor (1 for $r \leq R$, $0.5(1+\cos(\pi(r-R)/\text{taper}))$ over the taper, 0 beyond; taper<=0 $\Rightarrow$ hard cutoff) + ResolveInstantaneousOverstressCircular (strike-only, seeded once via ApplyOnce).
- **req 3** — INucleationMethod (§5.2) + StaticOverstress, GaussianGradualOverstress, CompactCircularGradualOverstress, InstantaneousOverstressCircular (each exposes Params() for the driver's diagnostics/ParaView consumers); MakeNucleation dispatches on cfg.nucleation.kind, absent block $\Rightarrow$ StaticOverstress.

Decisions / deviations (documented): - The two Phase-6 nucleation specs were **moved** from spatial_friction.hpp to dynamic/spatial_nucleation.hpp (alongside GradualOverstressSpec), per req 1's stated placement and to avoid a circular include (spatial_friction.hpp already includes spatial_nucleation.hpp, so the move is transparent to all consumers — verified: config 142/142, the driver object still compiles). - MakeNucleation signature uses (cfg, const Vector& dof_coords_3d, const DenseMatrix& dof_basis) instead of the plan's (cfg, std::vector<Vector>, std::vector<FaultBasisRow>) because FaultBasisRow does not exist; these are the concrete types the driver and the existing Resolve* overloads use (9×N column-major basis).

- **req 4 (driver swap) — COMPLETE (code)**. drivers/spatial_dyn_driver.cpp now builds std::unique_ptr<INucleationMethod> nuc = MakeNucleation(cfg, dof_coords_3d, dof_basis) in place of the inline ResolveGradualOverstress; the per-sub-step nuc_cb calls nuc->ApplyIncrement(dof_data, t, dt); and nuc->ApplyOnce(dof_data) runs once after init for the one-shot kind — **guarded to fresh-start only** (restart_prefix.empty()) so a restart (whose checkpoint already restored the seeded tau2_nuc) does not double-seed. The three nuc_params consumers (PrintDerivedAndCheck* ~:1486/:1505, ParaView fields ~:1737) are re-sourced from GaussianGradualOverstress::Params() via a dynamic_cast (empty for non-Gaussian kinds — those consumers already guard on .Size()), so the SAFS+RS path resolves once and is unchanged.

Verification (objects force-rebuilt vs the edited headers): seas_test_spatial_nucleation 73/73 (compact-bell + instantaneous acceptance values), seas_test_nucleation_factory 16/16 (each kind + absent block + the F-5 **byte-identity** check: factory Gaussian path == inline resolve+apply, exact equality on per-DOF params AND accumulated tau_nuc), seas_test_spatial_friction_config 142/142, ..._resolver 100/100, compute_safs_params 23/23, spatial_setup 71/71, spatial_print_derived 33/33, spatial_stress_bundle 5/5; **seas_spatial_dyn_driver links**.

Acceptance criteria 1 & 2 met. Criterion 3 (bit-identical SAFS+RS smoke) is proven at the

unit level by F-5 (the swapped nucleation computation is byte-identical by construction — GaussianGradualOverstress wraps the exact same resolve+apply free functions). The **end-to-end SAFS+RS smoke is still pending** because the SAFS mesh (safs/.../meshing/results/msh/safs_fault_box_nwcu) is a generated artifact not present in the tree — running it needs the gmsh meshing pipeline (conda activate pythonenv) + a multi-rank run, the same infra deferred since Phase 4. This is the only outstanding Phase-7 item.

9.8 Phase 8 – TPV205/102/104 benchmarks (configs, meshes, jobs, regression)

Goal

The three primary TPV cases run through the driver (scalar Riemann) and match their SCEC references within tolerance; the review/sign/basis tests are green; $np > 1$ parity holds.

Files to Create

- `tpv{205,104,102}/configs/*.toml`, `meshes`, `jobs/tpv*_spatial/*.sbatch`; `tests/unit/test_tpv_toml_stress_sign.cpp`, `test_spatial_dyn_tpv205_review.cpp`, `test_spatial_dyn_tpv102_tpv104_review.cpp`, and the planar-TPV basis-correctness test (D1).

Detailed Requirements

1. Author the three TOMLs: (a) `[numerics] interior_flux="scalar", cfl_safety="dg", TPV205 fault_iterator="one-shot"`; (b) **D3.2 fault-local stress** `kind="fault_local_prestress"` with `tau_strike_pa=+tau_ini` (TPV205 +70e6, TPV102 +75e6, TPV104 +40e6), `tau_dip_pa=0`, `sigma_n_pa=+120e6`; TPV205 adds positive `tau_strike` patches (central +81.6e6, left +78e6, right +62e6) with a header comment; (c) TPV102 `[friction.rate_state] aging + boxcar_taper`; TPV104 `state_evolution="slip_law_strong_rate_weakening" + V_w + boxcar`; both `[nucleation] gradual_overstress_compact_circular`; TPV205 `[friction.slip_weakening]`
 - barrier rules, no `[nucleation]`.
2. `test_tpv_toml_stress_sign.cpp` (D3.2): build each `FaultLocalPrestressSource`, seed a canonical-frame DOF via `ComputeParamsFaultLocal`, assert $\tau_z(\text{strike}) == +\tau_{\text{ini}}$, $\tau_1(\text{dip}) == 0$, $\sigma_n == +(120\text{e6} - P_p)$; precondition `dip==(0,0,-1)`, `strike==(+1,0,0)`.
3. Planar-TPV basis test (D1): build the driver-local per-QP basis on a small $y=0$ TPV mesh, assert canonical frame at every fault QP
4. Meshes (conda activate pythonenv, v2.2): `tpv102_1000m.msh` present; generate gmsh `-format msh22 -3 tpv104/mesh/tpv104_1000m.geo ... and ... tpv205/mesh/tpv2053d_200m.geo ...`
5. Regression: TPV station traces vs `tpv*/gold/results_*/results/*.dat` (native-driver runs) and SCEC `benchmark_data/` overlays; document tolerance; store a first `tpv*/gold_spatial/`.
6. **$np > 1$ MPI parity (§5.4-7)**: forced-2-rank single-shared-fault-face cross-rank DOFData symmetry + flux bit-identity, for LSW and rate-state.

Acceptance Criteria

- ☐ Each config `--dry-run` parses; a `tfinal=0.2s` 8-rank smoke writes SCEC `.dat` + a fault collection.
- ☐ `seas_test_tpv_toml_stress_sign, _spatial_dyn_tpv205_review, _spatial_dyn_tpv102_tpv104_review`, the planar-basis test pass; $np=2$ parity passes.

- TPV station traces match gold/SCEC within tolerance (rupture front, peak V, final slip).

Dependencies

Depends on: Phases 5, 6, 7. Required by: nothing (TPV31 is independent via Phases 9–10).

9.8.5.1 Phase 8 status (2026-05-28) — code-authoring + tests + dry-run COMPLETE; runs/regression DEFERRED Scope this pass (user-chosen): the deterministic deliverables (TOMLs + unit tests + Makefile + driver dry-run); the heavy multi-rank smoke + gold regression were deferred to a run session.

- **req 1 — three TPV TOMLs authored** (tpv{205,102,104}/configs/*_spatial.toml): all kind="fault_local_prestress" (D3.2; tau_strike = +70/+75/+40 MPa, sigma_n = +120 MPa), interior_flux="scalar", cfl_safety="dg", mixed_flux="adjacent" (matches the gold *_mfadj_p1_02 runs). TPV205: LSW ($\mu_s/\mu_d/d_c = 0.677/0.525/0.40$) + 3 stress patches (nuc/left/right = +81.6/+78/+62 MPa) + 3 barrier rules; fault_iterator="one-shot"; no [nucleation] (static). TPV102: aging RS; TPV104: slip-law SRW ($f_w=0.2$); both gradual_overstress_compact_circular nucleation ($\Delta\tau = 25/45$ MPa); fault_iterator="substep".
- **req 2 — test_tpv_toml_stress_sign.cpp**: parse each TOML, seed via ComputeParamsFaultLocal, assert tau2_0(strike)=+tau_ini, tau1_0(dip)=0, sigma_n0=sigma_n-P_p; canonical-frame precondition. 32/32 PASS.
- **req 3 — test_planar_tpv_basis.cpp (D1)**: FaultBasis::ComputeOrientedFrame yields the exact canonical frame dip=(0,0,-1)/strike=(+1,0,0) for the ref-aligned y=0 normal; mesh-backed per-QP check (canonical up to the documented v61 global sign-flip). 77/77 PASS.
- **test_tpv_config_parse.cpp** (deterministic core of the _review tests / AC “each config parses”): all three TOMLs parse + every Phase-8 field lands as authored. 60/60 PASS.
- **AC “each config --dry-run parses”**: ALL THREE construct successfully (TPV205 fault=103 → 79980 fault DOFs; TPV102/104 fault=3 → 4938 each; banner stress kind: fault_local_prestress, friction + nucleation wired).

Deviations (documented): 1. **Driver change (user-approved, outside Phase 8’s stated file list)**. spatial_dyn_driver.cpp hardcoded the SAFS boundary attrs (fault=101/natural=102/absorbing=103/104) and ignored [boundary], so a TPV dry-run found 0 fault DOFs and aborted. Fixed: the driver now reads `cfg.boundary.{fault_attr,natural_attrs,absorbing_attrs}` when present, falling back to the hardcoded SAFS values when [boundary] is absent (verified: none of the 8 SAFS configs set [boundary] → SAFS byte-unchanged). Also fixed the banner stress-kind ternary (was display-only “sidecar_hdf5” for the FaultLocalPrestress case). **TPV205 mesh uses fault=103/free=101/absorb=105** (not TPV102/104’s 1/3/5). 2. **boxcar_taper deferred** → **box rules**. The resolver rejects boxcar_taper rules (Phase 6 made the kind config-only). TPV102/104’s VW-core a/V_w spatial variation is expressed with hard box rules (the resolver’s own recommended substitute); the smooth SCEC tanh transition is deferred. 3. **Inside-out VW/VS expression**. The parser enforces $a_{\text{default}} < b_{\text{default}}$ (velocity-weakening) for the scalar fallback. TPV102/104 set the VW value as a_{default} (passes the guard) and the VS border via box rules (the resolver permits per-DOF $a>b$, R-011). Net per-DOF a/V_w field is identical to the native “VS background + VW core” form.

DEFERRED to a run session (heavy multi-rank compute + tolerance judgment): the $t_{\text{final}}=0.2s$

8-rank smoke writing SSEC .dat, the trace-comparison `_spatial_dyn_tpv{205,102,104}_review` tests vs `tpv*/gold/*.dat`, the `np=2` MPI cross-rank parity test, and the gold/SSEC tolerance match (req 5/6 + the run-dependent ACs). The smooth boxcar `V_w/a` taper (deviation 2) should be wired before claiming byte-exact TPV102/104 gold parity.

9.8.5.1.1 Phase 8 review fixes (2026-05-28, REVIEW.md R-001...R-005) The adversarial review found the three [numerics] method selectors were parsed + tested but **not consumed by the driver** (dead knobs), plus a σ_n double-source. Fixed (code-fix pass): - **R-001** — driver now MFEM_VERIFYs `InteriorFluxSupported(cfg)` before the `WaveOperator` ctor: `interior_flux="matrix"` aborts (deferred Phase-9 port) instead of silently running scalar. - **R-002** — driver honors `cfl_safety` via `CflSafetyFactor(cfg)` ($Dg=1/(3 \cdot (2p+1))$, $Raw=1.0$); the `Dg` branch is byte-identical to the prior hardcoded (verified: TPV205 `dt_cfl=1.15225e-4`, `nsteps=1736` unchanged). **The parser/struct default flipped raw→dg** (user-approved) so every config that omits the key (all 8 SAFS configs) keeps the long-standing always-DG-factored behavior — `raw` is now an explicit opt-in. - **R-003** — driver MFEM_VERIFYs `FaultIteratorSupported(cfg)`: one-shot aborts (the spatial driver always sub-steps). TPV205 TOML relabeled `one-shot`→`substep` (matches the O2-substep gold); parse test updated. - **R-004** — NOT fixed (tracked feature deferral): the hard-box `VW/V_w` approximation still bounds byte-exact TPV102/104 gold parity; needs `boxcar_taper` resolver consumption (= deviation 2 above). - **R-005** — parser asserts `sigma_n_default == sigma_n_pa - P_p` for a `fault_local_prestress RS` config (gated; SAFS exempt). - New decision helpers `CflSafetyFactor` / `InteriorFluxSupported` / `FaultIteratorSupported` (inline, `spatial_friction.hpp`) make the selectors unit-testable. Tests: config-parse 68/68 (+8 helper), `spatial_friction_config` 145/145 (+3 R-005), stress-sign 32/32, planar 77/77; resolver 100/100, rate-state-guards 10/10, iterator-factory 14/14, constant-tensor-sign 27/27 (re-verified vs the edited header — no regression). Driver one-shot abort + TPV205/102/104 dry-run construction re-confirmed.

9.9 Phase 9 – Heterogeneous (matrix) Riemann solver option (PORT from hrs-ref)

Goal

`interior_flux = "matrix"` routes interior non-fault faces through the bimaterial Riemann flux; `interior_flux = "scalar"` is byte-unchanged; the fault flux is unchanged on both paths; `np>1` parity holds.

Files to Port / Modify

- **Port (verbatim from hrs-ref):** `dynamic/godunov_flux_bimaterial.{hpp,cpp}`.
- **Modify:** `dynamic/wave_operator.{hpp,inl}` (add the heterogeneous ctor; the per-face pre-compute, the cross-rank material exchange, and the dispatch branches), `drivers/spatial_dyn_driver.cpp` (the scalar/matrix `WaveOperator` ctor branch + `MaterialField` from [material]), `miniapps/seas/Makefile`.

Detailed Requirements

1. Port `godunov_flux_bimaterial.{hpp,cpp}` exactly (the `BimaterialFlux` class: `BuildGodunovStateFaceLocal` 9×9 face-local projectors, `BuildPerFaceFluxMatricesGlobal` global rotation, `ApplyPerFaceFlux` runtime per-QP apply $F_{h_self} = fluxLocal \cdot Q_self + fluxNeighbor \cdot Q_nbr$). Keep the **SeisSol positive-stress matrix convention** (cols 0–2 positive) — required for the exact

linearised bimaterial Riemann solver; the MFEM-negative convention reduces to average-flux and is wrong here.

2. Add the heterogeneous ctor `WaveOperator(mesh, order, const MaterialField&, bc)` that sets `material_ = &material`, builds `owned_flux_pool_` per element via `material.EvalAt(elem, T, centroid)`, populates `per_elem_lmr_` and the `per_elem_h_/MaxCpInElement` CFL cache, and calls `ExchangeBiMaterialNeighbours_()`.
3. Port `BuildPerFaceBimaterialFluxMatrices_()` → fills `per_face_bimaterial_flux_[face][side]` `[{fluxLocal, fluxNeighbor}]` from `per_elem_lmr_` and (shared faces) `shared_face_neighbour_material_`.
4. Wire the dispatch: in `wave_operator.inl` `Mult (~:1311)` and `ADER (~:5191)` interior non-fault face paths, `if (owned_flux_pool_) → BimaterialFlux::ApplyPerFaceFlux(...)` for each element side, `else → flux_.Interior(...)` (the existing scalar/mixed path). `ADER CK` recursion uses `ApplyJacobianPerElementDOF_` when `owned_flux_pool_` is set, `else ApplyJacobianPerDOF_`.
5. **Shared-rank (§5.4):** `ExchangeBiMaterialNeighbours_` is one more collective and must be gated on `pmesh.GetNSharedFaces() > 0` (NOT shared-fault faces), so a rank with shared non-fault faces but no shared fault faces still participates (R-1600). **Fault faces stay on Fault-FaceFlux regardless of interior_flux** — the bimaterial path touches only interior non-fault faces; the friction iterator and `ComputeADERSharedFaceFluxRHS` are unchanged.
6. **Driver scalar/matrix branch:** convert the operator to `std::unique_ptr<WaveOperator> wave_ptr;` and select `scalar → WaveOperator(pmesh, order,  $\lambda, \mu, \rho$ , bc)` (the existing path) vs `matrix → WaveOperator(pmesh, order, material_field, bc)`, with `material_field` (`MaterialField`) built from `[material]` (`Constant` vs `DepthProfile1D`, Phase 6 / Phase 10). This is where the matrix path becomes reachable; before this phase the driver only ever built the scalar ctor, so the `SAFS+RS / TPV2xx` scalar runs are untouched.
7. Makefile: define `GODUNOV_FLUX_BIMATERIAL_{SRC,OBJ}`, add to the driver link line.

Edge Cases

- `owned_flux_pool_ == nullptr` (scalar ctor) must reproduce today's scalar path **byte-for-byte** (no new branches taken) — the `SAFS+RS` and `TPV2xx` cases never select `matrix`.
- A near-equal-material face (λ, μ, ρ continuous) must reduce the bimaterial flux to the scalar Godunov flux to round-off (sanity check on the projector).
- `matrix + mixed_flux`, or `matrix + Constant material` → config-parse abort (Phase 6 mutual-exclusion guard).

Acceptance Criteria

- ☐ All scalar `TPV/SAFS` runs are byte-unchanged vs a pre-Phase-9 checkpoint (`owned_flux_pool_` null path untouched).
- ☐ A 2-block bimaterial unit test (`seas_test_godunov_flux_bimaterial`): `ApplyPerFaceFlux` reproduces the hrs-ref reference values for a known material jump.
- ☐ `np > 1` parity on a heterogeneous mesh: interior-face flux is bit-identical across the rank seam (cross-rank `ExchangeBiMaterialNeighbours_` correct).

Dependencies

Depends on: Phases 3, 6. Required by: Phase 10.

9.9.6.1 Phase 9 status (2026-05-28) — Stage A COMPLETE; Stage B (core dispatch) DEFERRED Split into two stages (user-approved) because Stage B is a high-risk integration into the core wave operator (affects every simulation).

- **Stage A — COMPLETE (committed dee6931).**
 - `dynamic/godunov_flux_bimaterial.{hpp,cpp}` ported **byte-identical** to hrs-ref (req 1: `BuildGodunovStateFaceLocal/BuildPerFaceFluxMatricesGlobal/ApplyPerFaceFlux`; SeisSol positive-stress convention).
 - `tests/unit/test_godunov_flux_bimaterial.cpp` (ported from hrs-ref `test_phaser_bimaterial_flux`) **21/21 pass**. R.1.T-1 is the homogeneous-limit byte-exact reduction to `GodunovFlux::Interior` (1e-10 rel) = **AC #2 + the near-equal-material edge case**; plus P/S transmission, rotation covariance, role-swap symmetry, partition-of-identity, and the acoustic/size/nor-unit guards.
 - Makefile: `GODUNOV_FLUX_BIMATERIAL_{SRC,OBJ} + test obj rules + target * build-group + aggregate test:`.
 - Note: safs already had the **Stage-1 scaffolding** (the heterogeneous `WaveOperator(mesh,order,MaterialF` ctor, `owned_flux_pool_`, `BuildGodunovFluxPool_`, `ExchangeBiMaterialNeighbours_`, `per_elem_lmr/ per_elem_h/shared_face_neighbour_material_ members`) from a prior partial port — so reqs 2 (ctor) and the pool build/exchange are present.
- **Stage B — COMPLETE (parts 1–3 committed; AC #3 deferred to Phase 10).**
 - **Part 1 (DONE, feb3863) — infrastructure.** `wave_operator.{hpp,inl}`: `per_face_bimaterial_flux_ member + accessor`; `FluxForElem_ helper`; `ApplyJacobianPerElementDOF_ + BuildPerFaceBimaterialF` (bodies ported from hrs-ref); `phaser_dispatch_count_ diagnostic`; het ctor now calls `BuildPerFaceBimaterialFluxMatrices()`. Makefile: `$(GODUNOV_FLUX_BIMATERIAL_OBJ)` appended after every `$(WAVE_OPERATOR_OBJ)` link reference (req 7 — the `.inl` now references `BimaterialFlux`, so ALL `wave_operator`-linking targets need it; obj-rule line excluded). Verified: constant-parity 15/15 STILL bit-identical (the precompute runs on the real Constant mesh but the scalar `flux_` is still used — dispatch unwired); `seas_test_wave_operator` 17/0.
 - **Part 2 (DONE, 1c7b5b3) — CK-recursion dispatch.** The 2 ADER CK sites (`wave_operator.inl` `ComputeADERTimeIntegrated + ComputeADERSubStepStates`) gated `if (owned_flux_pool_)` → `ApplyJacobianPerElementDOF_`, else scalar. Bit-identical on `Mode::Constant` → constant-parity stays 15/15 bit-identical.
 - **Part 3a (DONE, 39f2d16) — interior-face dispatch + activation.** All 4 interior non-fault flux sites wired with the gated `if (owned_flux_pool_)` bimaterial branch (existing scalar code kept as the untouched else), ported from hrs-ref: `ComputeFaceFluxRHS` (RK4 Mult, local: `F_h_e1+F_h_e2` → each element), `ComputeSharedFaceFluxRHS` (Mult, shared: `side-0 F_h` → local), `ComputeADERFaceFluxRHS` (ADER, local, via `I_self/I_nbr`), `ComputeADERSharedFaceFluxRHS` (ADER, shared). The 2 ADER CK sites were wired in part 2. Relaxed the het-ctor **Mode::Coefficient abort** (now SUPPORTED — every flux site + CK routes per-element) and the constant-parity test to **1e-9 relative** (the bi-material flux collapses to scalar Godunov to LU rounding; observed ~1.6e-10 on the ParMesh assembly path). Verified: **constant-parity 15/15 (np=1) + 48/48 (np=4)** — the Constant het ctor Mult now routes through `BimaterialFlux` and matches the scalar ctor to LU rounding end-to-end through the assembled operator; `seas_test_wave_operator`

17/0, seas_test_godunov_flux 32/32.

- **Part 3b (DONE, 4026f03) — driver scalar/matrix branch (req 6).** Driver converted to `std::unique_ptr<WaveOperator> + a reference bind (WaveOperator<ParMesh>& wave = *wave_ptr — all 22 wave. uses unchanged)`, branching on `cfg.numerics.interior_flux`: Scalar → existing ctor (byte-identical); Matrix → `het ctor WaveOperator(pmesh, order, material, bc)` guarded by `MFEM_VERIFY(material.mode != Constant)` (no silent homogeneous). The Phase-8 R-001 driver guard + the obsolete `InteriorFluxSupported` helper + its test assertions were REMOVED (matrix is now branched, not aborted; the parser's matrix⇒non-Constant + matrix⇒no-mixed-flux guards remain). The D-1 material.mode guard is gated on the scalar path. Verified: scalar TPV102/104 dry-run `dt_cfl=0.00066147` unchanged; a matrix config with no non-Constant material aborts loudly with the Phase-10 message; config-parse 66/66, planar 77/77.
- **Byte-exact scalar AC preserved by construction throughout** (scalar ctor leaves `owned_flux_pool_ == nullptr` → no new branch executes).
- **DEFERRED to Phase 10 / a run session: AC #3 (np>1 bimaterial parity)**
 - * a true heterogeneous run need the **DepthProfile1D MaterialField** (built in Phase 10) on a multi-rank heterogeneous mesh. The matrix operator + driver branch are fully wired; only the depth-profile material source + TPV31 config remain (Phase 10), at which point the driver's matrix branch becomes end-to-end runnable.

9.10 Phase 10 – TPV31 (depth-heterogeneous LSW) verification case (PORT from hrs-ref)

Goal

TPV31 runs through `spatial_dyn_driver` on the matrix Riemann path with a 1-D depth-profile material and matches the SCEC eqdyna/seisol references.

Files to Port / Create

- **Port:** `DepthProfile1D / MakeDepthProfile1DMaterial` into `dynamic/heterogeneous_material.{hpp,cpp}`.
- **Copy (from hrs-ref):** the `tpv31/` tree — `tpv31/configs/tpv31.toml`, `tpv31/mesh/tpv31_50m.{geo,msh}`, `tpv31/benchmark_document/`, `tpv31/benchmark_data/`, `tests/unit/test_tpv31_*.cpp`.
- **Create:** `jobs/tpv31_spatial/tpv31_spatial_dyn_50m_*.sbatch`.

Detailed Requirements

1. Port `MakeDepthProfile1DMaterial(layers, depth_axis)` verbatim (Coefficient-mode `MaterialField`; `depth = max(0, -z)` along the depth axis; discontinuous layers with linear interpolation between specified depths). Wire `MaterialKind::DepthProfile1D` (parsed in Phase 6) → this builder → the matrix `WaveOperator` ctor (Phase 9).
2. Adapt `tpv31.toml` to the spatial schema: `[meta].law = "lsw"` (`fault_geometry.kind = tpv31_lsw`), `[material] kind = "depth_profile_1d" + [[material_profile.layer]]`, `[numerics] interior_flux = "matrix", cfl_safety = "dg"`; `[friction.slip_weakening]` with `mu_s=0.58`, `mu_d=0.45`, `d_c=0.18`; **depth-linear cohesion** `C0(depth) = 0.000425 MPa/m · max(0, 2400 - depth)`; `[nucleation] instantaneous overstress_circular` (hypocenter `z=-7500` m, `radius/taper/Δτ0` from the SCEC spec).
3. Wire per-DOF cohesion `C0(depth)` into the LSW resolver (`ResolveSlipWeakening`) and into

the LSW friction kernel's strength $\mu_s \cdot \sigma_n + C0$ (additive; if safs's LSW path lacks a cohesion term, add it guarded so $C0=0$ reproduces the current behaviour exactly).

4. Mesh (conda activate pythonenv): use the committed v2.2 tpv31_50m.msh, or regenerate `gmsht -format msh22 -3 tpv31/mesh/tpv31_50m.geo -o tpv31/mesh/tpv31_50m.msh`.
5. Verification: TPV31 station traces vs `tpv31/benchmark_data/scec_{eqdyna,seisol}/`; document tolerance and store a `tpv31/gold_spatial/` set from the validated run.

Edge Cases

- Depth-axis sign: $up[2]>0 \Rightarrow z \leq 0$ (R-008); the profile uses $depth = -z$.
- $C0=0$ must reproduce the pre-cohesion LSW path byte-for-byte (the cohesion term is additive and zero for the other TPV cases).

Acceptance Criteria

- ☐ `tpv31.toml --dry-run --verify-dispatch` reports matrix Riemann + `depth_profile_1d` material + `tpv31_lsw` + `instantaneous overstress_circular`.
- ☐ An 8-rank smoke completes and writes SCEC station output; full run station traces match the `eqdyna/seisol` overlays within the documented tolerance.
- ☐ `seas_test_tpv31_*` (canonical rotation, nucleation+cohesion, station writer) pass.

Dependencies

Depends on: Phases 6, 7, 9.

9.11 Phase 11 – Depth-varying rate-and-state $a(z)$ / $b(z)$ (depth-profile input)

Overview

Today the rate-and-state friction parameters a and b are spatially uniform scalars (`[friction.rate_state].a_default / b_default`), with one asymmetry: a is already a fully per-DOF channel (`DOFData.a`, `rs.a(i)`, `FrictionSolver::Solve(..., a)`), while b is **scalar by construction** — it is read from `AgingLawPsi` (`state_evolution.hpp:225`) at three sites in `Tpv102SubStepIterator` and from `blk.b_default` in the equilibrium- ψ seed, and `DOFData` has no b slot. `ResolveRateState` therefore **rejects** any per-DOF b (R-006, `spatial_friction.cpp:1020`).

This phase gives the rate-and-state a and b **two input modes**:

1. **Constant** (the existing path, retained): scalar `a_default / b_default` when no depth profile is configured. *Backward-compatible; byte-identical to today.*
2. **Depth-varying via two CSV files** (the new path): $a(z)$ from `param_a.csv` and $(a-b)(z)$ from `param_a_minus_b.csv`, with $b(z) = a(z) - (a-b)(z)$. Each CSV is **independently** linearly interpolated along depth (the two files may have **different** depth grids) and flat-clamped outside its own range.

The depth coordinate is vertical: $depth = \max(0, -z)$ in metres ($z = 0$ at the free surface, $z < 0$ below), matching the existing `kind = "depth"` spatial rules and the pore-pressure convention (`spatial_friction.cpp:1042`). **The CSV depth column is in kilometres** and is scaled to metres at load time (`depth_m = depth_csv · 1000`, via a config key — no hard-coded 1000). The mapping is `a_i = profile.a(depth_i)`, `b_i = profile.b(depth_i)` per fault DOF.

CSV format (matches the committed param_a.csv / param_a_minus_b.csv): no header, comma-separated, **two columns per row (value, depth_km)** — the parameter value FIRST, depth (km) SECOND. Example param_a.csv row: 0.1495, 52.68 $\Rightarrow$ a = 0.1495 at 52.68 km = 52 680 m.

The work splits into three independently testable sub-phases:

- **Phase 11a** — *Per-DOF b channel* (foundation; **net-zero physics** when no profile is present). Add DOFData.b; set it in **both** RS init paths; switch the Tpv102SubStepIterator ψ -update + the equilibrium seed from the scalar b to per-DOF d.b / rs.b(i); lift R-006's b rejection (keep f_0/V_0 rejected). This is the **documented reversal of prior fix R-006** (see §11a step 6).
- **Phase 11b** — *Two-CSV depth-profile input* (PiecewiseLinear1D + FrictionDepthProfile1D holding the two independent curves + the two-CSV loader + parser + resolver seed-from-profile). This is where a(z)/b(z) actually vary.
- **Phase 11c** — *Config + sbatch deliverable + --print-derived profile summary*.

Constraints carried from Phase 3 (do not regress): - Tpv102SubStepIterator is the **byte-exact TPV102 oracle** shared by tpv102_driver.cpp and the SAFS-RS path. Any change to its ψ -update **MUST** keep native TPV102 bit-identical (§11a step 7). - Only a and b become per-DOF f_0 and V_0 stay scalar (V_0 is pinned to FrictionSolver::V0 by the R-009 guard; per-DOF V_0/f_0 is explicitly out of scope). - No checkpoint-format change: b is a **static** field (io/tpv104_checkpoint.hpp:31–34 serializes only dynamic fields; statics are recomputed each start and preserved by the resize-without-clear at :199–203). b joins a/Dc as a recomputed static field. - Do not require a < b anywhere per-DOF (R-011): velocity-strengthening tapers (a > b) are the whole point of a depth profile.

Phase 11a – Per-DOF b channel (net-zero when no profile)

9.11.2.1 Goal b flows per-DOF end-to-end (DOFData.b $\rightarrow$ iterator ψ -update + equilibrium seed), the resolver accepts per-DOF b, and **every existing run is bit-identical** because rs.b(i) equals the scalar b_default at every DOF until Phase 11b introduces a profile.

9.11.2.2 Files to Modify

- dynamic/fault_face_flux.hpp — add a b field to struct DOFData.
- dynamic/tpv102_substep_iterator.cpp — 3 ψ -update sites read d.b instead of state_evo_.GetB().
- dynamic/spatial_setup.hpp — InitializeFaultDOFs_Spatial_RS sets d.b = rs.b(i); SeedEquilibriumPsi_RS uses rs.b(ii) instead of blk.b_default (two sites) + size guard.
- dynamic/tpv102_setup.hpp — set d.b = TPV102Params::b alongside d.a/d.Dc (:96–97).
- spatial/code/spatial_friction.cpp — relax R-006 (:1020) to reject only f_0/V_0; add the if (!std::isnan(r.b)) { b_i = r.b; } override (:1025–1029 block).
- io/tpv104_checkpoint.hpp — **comment only**: add b to the static-field list at :31–34 and :354 (no serialization change).

9.11.2.3 Detailed Requirements

1. DOFData.b (fault_face_flux.hpp, after :47 real_t a = 0.004;). Add:

```

real_t b = 0.0;    ///< RS state-evolution parameter (per-DOF; set by every RS init path).
                  ///< 0.0 is the "unset/LSW" sentinel: LSW DOFData never reads it, and an
                  ///< RS path that forgot to set it makes UpdateStateAnalytic divide by 0
                  ///< (caught loudly by the equilibrium / blow-up checks), never a silent
                  ///< wrong-physics value.

```

Place it in the rate-and-state group next to a/Dc/psi, NOT the lsw_* group.

2. **Iterator ψ -update (tpv102_substep_iterator.cpp, 3 sites: :186–190, :348–352, :530–534).**
At each site replace the 5th argument state_evo_.GetB() with d.b — where d is the per-QP DOFData& whose .psi is the LHS (d.psi = UpdateStateAnalytic(d.psi, s.V_abs, d.Dc, dt_sub, state_evo_.GetF0(), d.b, state_evo_.GetV0()));. Confirm the loop variable name at each site (it is d at :186; verify at the other two). **Keep GetF0()/GetV0() (scalar).** The owned AgingLawPsi law_ in RateStateAgingFrictionIterator (friction_iterator.hpp:164) is still constructed (its f0_/V0_ are read); only its b_ becomes unused by the iterator — leave the member and the ctor untouched.
3. **InitializeFaultDOFs_Spatial_RS (spatial_setup.hpp:340–341).** After d.a = rs.a(i); and d.Dc = rs.Dc(i); add d.b = rs.b(i);. Add MFEM_VERIFY(rs.b.Size() == ndof, "rs.b size mismatch"); (the function already verifies rs.a/rs.b/rs.Dc at :321–323, so this is already present — confirm rs.b is among them; if not, add it).
4. **SeedEquilibriumPsi_RS (spatial_setup.hpp:395–462).**
 - Up-front guard (:404–409): add rs.b.Size() >= nd to the MFEM_VERIFY conjunction.
 - DieterichRuinaFriction ctor (:441–443): change the 3rd Constants argument from blk.b_default to rs.b(ii): DieterichRuinaFriction::Constants{ blk.V_0_default, blk.f_0_default, rs.b(ii), rs.Dc(ii)}.
 - Locked-fallback branch (:459–460): change blk.b_default to rs.b(ii): (blk.f_0_default + rs.b(ii) * std::log(blk.V_0_default / rs.V_init(ii))).
 - Update the function doc-comment (:373–374, :397–398) from “b ... scalar globals” to note b is now per-DOF from rs.b(ii) (only V_0/f_0 remain scalar globals from blk).
5. **InitializeFaultDOFs_TPV102 (tpv102_setup.hpp:96–97).** After d.a = ComputeA(...) and d.Dc = TPV102Params::Dc; add d.b = TPV102Params::b;. This is the byte-exact requirement: it makes d.b == state_evo_.GetB() for every native-TPV102 DOF (the driver builds AgingLawPsi(TPV102Params::b, ...) at tpv102_driver.cpp:1692), so step 2’s iterator change is a no-op for TPV102.
6. **Lift R-006 for b (spatial_friction.cpp:1013–1029).** This is a **deliberate reversal of a prior fix**, justified per CLAUDE.md:
 - *Cite:* R-006 rejected per-DOF b/f_0/V_0 because “the aging law is built from the scalar global blk.b_default ... DOFData has no per-DOF slots for them, so a per-rule override would be silently dropped — a knob that does nothing.”
 - *Why lifting is now valid:* R-006 was **correct at the time** (no per-DOF b path existed). Phase 11a steps 1-5 add the per-DOF b slot (DOFData.b) and route **all three** consumers (iterator $\times 3$ sites, equilibrium seed $\times 2$ sites) through it, so the “silently dropped” condition no longer holds for b. f_0/V_0 keep no per-DOF path (and V_0 is pinned by R-009),

so they stay rejected.

- *Change:* replace the verify at :1020 with `MFEM_VERIFY(std::isnan(r.f_0) && std::isnan(r.V_0), "ResolveRateState: per-DOF f_0/V_0 spatial overrides are not supported (scalar aging-law globals; V_0 pinned by R-009); set them only in the [friction.rate_state] defaults. Offending rule at DOF " << i);` and update the comment to cite Phase 11a as the slot that makes per-DOF b real.
- In the override block (:1025–1029), add `if (!std::isnan(r.b)) { b_i = r.b; }` (precedence: profile/default seed → spatial-rule override, last-match-wins, identical to a).
- *Evidence (acceptance):* the byte-exact TPV102 regression (step 7) and the scalar-RS no-regression test (Testing) must both pass — that is the “show evidence reverting improves / does not harm correctness” requirement.

7. **Byte-exact regression (verification, not a code change).** Confirm `seas_test_ader_tpv102_smoke` and `seas_test_tpv102_setup` stay green after steps 2 + 5. Extend `test_tpv102_setup.cpp` with an assertion that `d.b == TPV102Params::b` for every DOF after `InitializeFaultDOFs_TPV102`.

9.11.2.4 Edge Cases to Handle

- **LSW DOFData:** b defaults to 0.0 and is never read on the LSW path (the LSW iterator does not call `UpdateStateAnalytic`). No LSW init path sets `d.b`; that is fine.
- **TPV104:** untouched — it owns `tpv104_substep_iterator.cpp` (its own `GetB()` sites at :416, :658) and is not modified by this phase.
- **d.b unset on an RS path:** `b = 0.0 → exp((psi-f_0)/0)` in `UpdateStateAnalytic` → $\pm\text{inf}/\text{NaN}$, caught immediately by the equilibrium check / blow-up monitor (loud, not silent).

9.11.2.5 Acceptance Criteria

- ☐ `seas_test_ader_tpv102_smoke` and `seas_test_tpv102_setup` pass (TPV102 bit-identical); `test_tpv102_setup` asserts `d.b == TPV102Params::b` for all DOFs.
- ☐ A scalar-RS config (no `[friction.rate_state.depth_profile]`) yields `rs.b(i) == b_default` for every DOF (new assertion in `test_spatial_friction_resolver`), and an existing SAFS-RS --dry-run is byte-identical to the pre-Phase-11 baseline.
- ☐ `ResolveRateState` now ACCEPTS a per-DOF b spatial rule and still REJECTS per-DOF `f_0/V_0` (updated `resolve_rate_state_guards` cases).
- ☐ make test baseline green.

9.11.2.6 **Dependencies** Depends on: Phase 3 (RS path; done). Required by: Phase 11b.

Phase 11b – Two-CSV depth-profile input ($a(z) + (a-b)(z)$)

9.11.3.1 **Goal** `[friction.rate_state.depth_profile]` pointing at `param_a.csv` (a vs depth) and `param_a_minus_b.csv` ($(a-b)$ vs depth) produces per-DOF $a(z)$ and $b(z) = a(z) - (a-b)(z)$ via two independent flat-clamped linear interpolants; when the block is absent the scalar `a_default/b_default` path is used and results are byte-identical to Phase 11a.

9.11.3.2 Files to Create

- `tests/unit/test_friction_depth_profile.cpp` — unit tests for the interpolant, the two-CSV loader, and $b = a - (a-b)$ on differing grids (see Testing).

9.11.3.3 Files to Modify

- `spatial/code/spatial_friction.hpp` — add `PiecewiseLinear1D`, `FrictionDepthProfile1D`, `FrictionDepthProfileSpec`, and a `FrictionDepthProfileSpec depth_profile`; member on `RateStateBlock (:241–253)`; declare `LoadFrictionDepthProfileCSVs(...)`.
- `spatial/code/spatial_friction.cpp` — parse the new block in `parse_rate_state (:388–466)`; load + build the profile at parse time; gate the `a_default < b_default` check; implement the interpolant + CSV loader; seed `a_i/b_i` from the profile in `resolve_rs_impl (:997–998)`.
- `safs/project_7.0_alternative/document/spatial_friction_config_schema.md` — document the new block under `[friction.rate_state]` (schema stays version = 1; the block is optional).

9.11.3.4 Detailed Requirements

1. **PiecewiseLinear1D (header).** A generic flat-clamped 1-D linear interpolant — used for both the $a(\text{depth})$ and $(a-b)(\text{depth})$ curves (they may have different sample grids).

```
struct PiecewiseLinear1D
{
    std::vector<real_t> x;    // ascending, strictly increasing; size >= 2 (depth in METRES)
    std::vector<real_t> y;    // same length as x
    real_t operator()(real_t xq) const; // linear interp; FLAT-clamp outside [x.front(), x.back()]
    void Validate() const;    // size>=2, x strictly increasing, x & y all finite
};
```

- `operator()`: binary-search the bracketing samples, linearly interpolate; for $x_q \leq x_{\text{front}}$ return y_{front} , for $x_q \geq x_{\text{back}}$ return y_{back} (**constant** extrapolation — NEVER linear, which could push a/b non-physical past the data range).

2. **FrictionDepthProfile1D (header).** Holds the two curves; computes $b = a - (a-b)$. Named to avoid collision with the **material** `DepthProfile1D` reserved by Phase 10 (dynamic/heterogeneous_material).

```
struct FrictionDepthProfile1D
{
    PiecewiseLinear1D a_of_depth;    // from param_a.csv : a(depth_m)
    PiecewiseLinear1D amb_of_depth;  // from param_a_minus_b.csv : (a-b)(depth_m)
    real_t a(real_t depth_m) const { return a_of_depth(depth_m); }
    real_t b(real_t depth_m) const { return a_of_depth(depth_m) - amb_of_depth(depth_m); }
};
```

a and $(a-b)$ are interpolated **independently** at `depth_m`, then subtracted — this is the one correct way to combine two CSVs with **different depth grids** (a : 0/14.9/52.7 km; $a-b$: 0/13.7/60 km in the committed files). NOTE: $b > 0$ is NOT validated at the profile level

(the difference of two piecewise-linears can dip anywhere); it is guaranteed per-DOF by the resolver validator (:1078, $b_i > 0$), whose abort message must name the depth profile (step 7).

3. FrictionDepthProfileSpec (header).

```
struct FrictionDepthProfileSpec
{
    bool        enabled = false;
    std::string param_a_csv;           // path to param_a.csv           (CWD-relative, like [me
    std::string param_a_minus_b_csv;   // path to param_a_minus_b.csv
    real_t      depth_to_m = 1000.0;  // depth_units="km" -> 1000.0; "m" -> 1.0 (NO hard-cod
    FrictionDepthProfile1D profile;    // built at parse time
};
```

Add FrictionDepthProfileSpec depth_profile; to RateStateBlock.

4. CSV loader LoadFrictionDepthProfileCSVs(const FrictionDepthProfileSpec& spec) (.cpp). Builds and returns a FrictionDepthProfile1D. Helper load_one_csv(path, depth_to_m) returns a PiecewiseLinear1D:

- Open path (taken as-is, relative to the process CWD, exactly like [mesh].path); MFEM_VERIFY(ifs.good(), "[friction.rate_state.depth_profile] cannot open '" << path << "'").
- Skip blank lines and #-comment lines. Each data row is **value, depth_km** (value FIRST, depth SECOND), comma- OR whitespace-separated, exactly 2 fields. Require ≥ 2 rows.
- Convert $\text{depth_m} = \text{depth_km} * \text{spec.depth_to_m}$; build $x = \{\text{depth_m}\}$, $y = \{\text{value}\}$.
- Sort (x, y) ascending by x ; MFEM_ABORT on duplicate x (ambiguous interpolation).
- Call Validate(). (Separately require every a value > 0 for the a curve; the $a-b$ curve may be negative — VW — so only finiteness is required for it.) Run load_one_csv on each of param_a_csv and param_a_minus_b_csv into the two members.

5. Parser wiring (parse_rate_state, :388–466). If rs.tbl.contains("depth_profile"):

- Read param_a_csv and param_a_minus_b_csv (both required, non-empty strings).
- Read depth_units (optional string, default "km"): "km" $\rightarrow$ $\text{depth_to_m} = 1000.0$, "m" $\rightarrow$ $\text{depth_to_m} = 1.0$; abort on any other value.
- Set depth_profile.enabled = true; call step-4's LoadFrictionDepthProfileCSVs into depth_profile.profile. (Reading at PARSE time means a missing/bad file aborts at config load — early and clear. Both LoadSpatialFrictionConfig and ParseSpatialFrictionConfigString get a built profile; a string-test must point param*_csv at real temp files.)

6. Gate the scalar a_default < b_default check (:455–457). Wrap it in if (!out.depth_profile.enabled) { ... }. Keep the unconditional a_default > 0 / b_default > 0 checks (:449–454) — the scalars remain the fallback when no profile is present. Rationale: with a profile, a_default/b_default are unused for a/b, and the data legitimately has $a > b$ (i.e. $a-b > 0$, VS) at depth.

7. **Resolver seed-from-profile (resolve_rs_impl, :990–1004).** Compute depth once at the top of the per-DOF loop (it is currently computed at :1042 for pore pressure — hoist and reuse):

```
const real_t depth_i = std::max(static_cast<real_t>(0.0), -z); // metres
real_t a_i, b_i;
if (cfg.depth_profile.enabled)
{
    a_i = cfg.depth_profile.profile.a(depth_i);
    b_i = cfg.depth_profile.profile.b(depth_i); // = a(depth) - (a-b)(depth)
}
else
{
    a_i = cfg.a_default;
    b_i = cfg.b_default;
}
```

Leave Dc_i , V_{init_i} , $f0_i$, $V0_i$, η_i , sn_i seeded from the scalar defaults (unchanged). The spatial-rule loop (now incl. the Phase-11a per-DOF b override) applies on top. The per-DOF validator (:1076–1089, post-R-011: $a, b > 0$ & finite, no $a < b$) already covers the profile output; augment the $b_i > 0$ message (:1078) to add: “(check param_a_minus_b.csv: $a - (a-b)$ went non-positive at depth $<depth_i>$ m)”.

9.11.3.5 Edge Cases to Handle

- **Different depth grids** between the two CSVs: handled by independent interpolation (step 2) — the whole reason b is computed at query time, not pre-merged onto one grid.
- $depth_i$ beyond a curve’s range (DOFs deeper than the last CSV sample, e.g. > 52.7 km for a): constant flat-clamp to that curve’s last value (step 1). The two curves clamp independently.
- $a - (a-b) \leq 0$ at some DOF (the data makes b non-positive there): the resolver’s per-DOF $b_i > 0$ validator aborts with the augmented message (step 7) — a data error, surfaced loudly.
- CSV with < 2 rows / duplicate depths / non-finite field / non-positive a : abort with a precise message (step 4).
- $depth_units$ other than “km”/“m”: abort (step 5).

9.11.3.6 Acceptance Criteria

- `test_friction_depth_profile` passes: `PiecewiseLinear1D` exact at samples, linear at mid-points, flat-clamped below first / above last sample; the two-CSV loader parses the committed `param_a.csv` / `param_a_minus_b.csv` (incl. the km→m scaling) and $b(depth) = a(depth) - (a-b)(depth)$ on the differing grids; loader rejects < 2 rows, duplicate depths, non-positive a , and a non-finite field.
- `test_spatial_friction_resolver` gains a CSV-profile case (temp CSVs): at hand-picked DOF depths $rs.a(i) = a(depth_i)$ and $rs.b(i) = a(depth_i) - (a-b)(depth_i)$, incl. a deep DOF where $a-b > 0$ so $a > b$ (VS), and a shallow DOF where $a-b < 0$ so $a < b$ (VW).
- `test_seed_equilibrium_psi_rs` gains a depth-varying- b case: two DOFs at different depths get different seeded ψ , and the per-DOF round-trip residual is ~ 0 (uses `rs.b(ii)`).

- Absent-block runs are byte-identical to Phase 11a.

9.11.3.7 Dependencies Depends on: Phase 11a. Required by: Phase 11c.

Phase 11c – Config + sbatch deliverable + --print-derived profile summary

9.11.4.1 Goal A runnable SAFS-RS config that points at the two committed CSVs (param_a.csv / param_a_minus_b.csv), a matching sbatch, and a --print-derived summary that echoes the resolved profile so the depth→(a,b) mapping is auditable before launch.

9.11.4.2 Files to Create

- safs/project_7.0_alternative/config/spatial_friction_rate_state_safs_projected_stress_depthprofile — clone of ...tpv102_defaults.toml with a [friction.rate_state.depth_profile] block:

```
[friction.rate_state.depth_profile]
param_a_csv      = "safs/project_7.0_alternative/friction/rate-and-state/param_a.csv"
param_a_minus_b_csv = "safs/project_7.0_alternative/friction/rate-and-state/param_a_minus_b.csv"
depth_units      = "km"      # CSV depth column is km; scaled *1000 to metres
```

a_default / b_default may stay in the file (now unused for a/b; the a_default < b_default check is gated off when the profile is enabled — Phase 11b step 6).

- jobs/safs/spatial_dyn_ratestate_depthprofile_8N_400r_dev_2hr_safs.sbatch — 8N/400r dev sbatch defaulting to the depth-profile config, --print-derived, tfinal within the reflection window (6s, per R-025; PML is still a no-op — keep use_pml = false).

The CSV inputs already exist (committed): param_a.csv (a vs depth-km) and param_a_minus_b.csv ((a−b) vs depth-km), both in safs/project_7.0_alternative/friction/rate-and-state/.

9.11.4.3 Files to Modify

- dynamic/spatial_print_derived.cpp — in PrintDerivedAndCheckRS (the Phase-3 RS overload), when cfg.rate_state→depth_profile.enabled, print a summary block: the two CSV paths, the sample tables a(depth_m) and (a−b)(depth_m), min/max of resolved a and b over the fault DOFs, and the VW↔VS transition depth (the depth where (a−b) crosses 0 — read directly from the amb_of_depth curve). L_nuc / f_ss reporting is unchanged — it already reads rs.b(i) per-DOF, so it now reflects the profile automatically.

9.11.4.4 Detailed Requirements

1. The config must pass --print-derived: profile summary prints; L_nuc range and f_ss range reflect the depth variation; the gate's existing checks (all-VS / empty-patch FAIL, sub-critical WARN) operate on the per-DOF a/b. Confirm the nucleation patch depth sits where a−b < 0 (VW core) so the patch is super-critical — document the patch depth vs the VW↔VS transition.

2. The config keeps the Phase-3 deliverable invariants (D3.1 stress signs, gradual overstress nucleation, $\eta = \text{"auto"}$, $V_0 = 1e-6$ per R-009, $t_{\text{final}} = 6s$).
3. Sanity-check the committed CSVs at config load: if any fault DOF would get $b \leq 0$ (i.e. $a-b \geq a$ at that depth), the resolver aborts with the augmented message (Phase 11b step 7) — fix the data or the depth range rather than masking it.

9.11.4.5 Acceptance Criteria

- ☐ `mpirun -np N ./seas_spatial_dyn_driver --config <depthprofile>.toml --print-derived` runs to a clean exit, prints the profile summary (both CSV paths + the VW↔VS transition depth) + depth-resolved $L_{\text{nuc}}/f_{\text{ss}}$, and dispatches the aging iterator (no abort).
- ☐ `bash -n` on the sbatch passes; the sbatch points at the depth-profile config and `--print-derived`.

9.11.4.6 Dependencies Depends on: Phase 11b. Required by: nothing (deliverable).

9.12 Phase 12 – Absorbing boundaries (PML) for the SAFS dynamic-rupture run

Promoted from document/pml_dev/PLAN_pml_safs_2026-05-27.md (revised 2026-05-28 against the post-Phase-6/7 tree). That document is the long-form theory + sizing reference; this section is the authoritative, reconciled plan. **Independent of Phases 8–11** — touches only the PML path; gated on `pml_layer_ != nullptr`, so it cannot perturb the TPV/BP5 byte-exact regressions.

Goal

`seas_spatial_dyn_driver` can absorb outgoing waves at the box walls with a free-surface-safe convolutional PML, removing the post- $t \approx 7$ s reflection contamination of the SAFS run ($t_{\text{reflect}} = \min_{\text{box_dim}}/c_p \approx 41.6 \text{ km} / 5996 \text{ m/s} \approx 6.94 \text{ s} \ll t_{\text{final}} = 100 \text{ s}$) and thereby settling whether the LSW V_{max} departure at $t \approx 28 \text{ s}$ is reflection-driven or an intrinsic friction instability.

Background – what already exists vs. what is missing

The codebase **already implements** the unsplit convolutional PML (`dynamic/pml_layer.{hpp,cpp}`), the `WaveOperator` hooks (`SetPML/GetPML wave_operator.hpp:246–247`; `ApplyPMLDamping` in `Mult` and the ADER corrector), the total-Q background (`SetAbsorbingBackground driver:1336`, `GetAbsorbingBackground() wave_operator.hpp:373`), and 4 passing unit tests (`tests/unit/test_pml.cpp`). **Missing (this phase):** (a) no driver ever constructs a `PMLayer` or calls `wave.SetPML()` — `--pml` only suppresses the reflection warning; (b) `ComputeDamping` damps **both** z-walls (`pml_layer.cpp:84–91`), which would wrongly damp the **free surface** at $z=0$ (attr 102); (c) no config knobs for thickness / target reflection / faces.

Background math (full derivation: standalone doc §A)

The bulk solver advances the first-order velocity–stress system as $\partial_t \mathbf{Q} = \sum_k \mathbf{A}_k \partial_{x_k} \mathbf{Q} \equiv \mathcal{L}(\mathbf{Q})$ with $\mathbf{Q} \in \mathbb{R}^9 = (\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}, v_x, v_y, v_z)$. The PML adds one local (zeroth-order) damping term that pulls $\mathbf{Q}$ toward the static-pre-stress background $\mathbf{Q}_{\text{bg}}$ (total-Q mode — damping toward $\mathbf{0}$ would erode the pre-stress and create an interface artefact):

$$\partial_t \mathbf{Q} = \mathcal{L}(\mathbf{Q}) - \mathbf{d}(\mathbf{x}) \odot (\mathbf{Q} - \mathbf{Q}_{\text{bg}}), \quad (\text{Eq. 16})$$

with the per-component rate $d_c(\mathbf{x}) = d_x D_c^x + d_y D_c^y + d_z D_c^z$ summing the three directional cubic ramps $d_\xi = d_{\text{max}}(s_\xi/L)^3$ (s_ξ = penetration depth from the inner edge), and binary selection vectors $\mathbf{D}^x = (1, 0, 0, 1, 0, 1, 1, 0, 0)$, $\mathbf{D}^y = (0, 1, 0, 1, 1, 0, 0, 1, 0)$, $\mathbf{D}^z = (0, 0, 1, 0, 1, 1, 0, 0, 1)$ (pml_layer.cpp:24–26; corners add). The peak rate **as coded** (pml_layer.cpp:44) is

$$d_{\text{max}} = \frac{3 c_p}{2 L} \ln \frac{1}{R_0}. \quad (\text{Eq. 17})$$

Precision caveat (carry into 12.1): the consistent constant for a cubic ($n = 3$) profile is $\frac{n+1}{2} c_p / L = 2 c_p / L$, but the code uses the $n = 2$ prefactor $\frac{3}{2}$. The realized reflection is therefore $R_{\text{eff}} = R_0^{3/4}$ (e.g. $R_0 = 10^{-3} \Rightarrow R_{\text{eff}} \approx 5.6 \times 10^{-3}$). **Decision:** either switch the prefactor to $2 c_p / L$ (true $n = 3$) or keep $\frac{3}{2}$ and document that the knob is R_{eff} , not R_0 . The Phase-12.3 metrics measure the *true* reflection regardless, so this only affects how the R0 input is labelled. The damping enters the **ADER corrector** (the operative path at ader_order=2), **not** the local predictor (intentional — wave_operator.hpp:427); the corrector damps the time-integrated state with target $dt \cdot \mathbf{Q}_{\text{bg}}$ (do **not** “simplify” it to the Mult form).

Phase 12.0 – Domain sizing (analysis only; no code)

Conclusion: no remesh / no box extension. Measured 500 m triq geometry: box X[314840,672330] Y[3642278,3888985] Z[-41608,0] (357×247×42 km), fault X[364840,622330] Y[3692278,3838985] Z[-16608,0], buffers ± 50 km (x,y) / 25 km (bottom); $\lambda = \mu = 3.2 \times 10^{10}$, $\rho = 2670 \Rightarrow c_p = 5996.2$, $c_s = 3461.9$ m/s; lc_far=3000 m, order 1, ader_order=2. A no-PML box reaching tfinal=100 s would need walls >c_p\,t/2\approx 300 km away (~ 600 km pads — infeasible). **Default L_pml = 12 km (4 far-field cells)**, keeping a 13 km bottom clearance; 15 km (5 cells) optional. Resulting d_{max} ($c_p = 5996$):

L_pml	R0=1e-3	R0=1e-4
12 km	5.18 s ⁻¹	6.90 s ⁻¹
15 km	4.14 s ⁻¹	5.52 s ⁻¹

$1/d_{\text{max}} \approx 0.19\text{--}0.24$ s $\ll$ the 7 s reflection window. Body wavelengths $\lambda_s \approx 0.7$, $\lambda_p \approx 1.2$ km (rise time ~ 0.2 s) $\rightarrow$ a 12–15 km PML is many body-wavelengths thick (the dominant $t \approx 7$ s body-wave contamination absorbs to $\sim R_0$); the longest-period Rayleigh content ($\lambda \approx 3.5\text{--}35$ km) is the residual the 12.3 metrics quantify.

Constraints

- **Do not modify** the elastic RHS, the friction laws (fault_face_flux.*, tpv205_friction.hpp), SetAbsorbingBackground, or any TPV102/104/205/BP5 byte-exact path. PML is additive, gated on pml_layer_ != nullptr.
- **Preserve the PMLayer ctor signature + the 4 existing tests** — extend, don’t break.
- **Free surface (attr 102) is never damped** — z-PML is bottom-only for SAFS.

- No hardcoded magic numbers in the driver — thickness/R0/faces come from config or are derived from the mesh bbox + lc_far.
- The PMLayer object must outlive the time loop (held by a `std::unique_ptr` in driver scope; `wave.SetPML(pml.get())`).

Phase 12.1 – Make PMLayer free-surface-safe (per-half-face mask) + tests

Goal. PMLayer damps any subset of the 6 half-faces (SAFS: $x_{\pm}$, $y_{\pm}$, $z_{\min}$; **not** $z_{\max}$), with today’s symmetric behavior preserved as the default.

Files to modify: `dynamic/pml_layer.hpp`, `dynamic/pml_layer.cpp`, `tests/unit/test_pml.cpp`.

Detailed requirements: 1. Add a 6-bit half-face mask with named constants `static constexpr int PMLayer::FaceXLo=1, FaceXHi=2, FaceYLo=4, FaceYHi=8, FaceZLo=16, FaceZHi=32, FaceAll=0x3F;`. 2. Extend the ctor with an **optional trailing** arg, preserving the current signature: `PMLayer(const Vector&, const Vector&, real_t thickness, real_t cp, real_t target_R=1e-3, int dirs=7, int half_face_mask=-1)`. When `half_face_mask==-1`, derive it from `dirs` (XLO|XHI if `dirs&1`, YLO|YHI if `dirs&2`, ZLO|ZHI if `dirs&4`) so existing call sites are byte-identical; store the resolved mask in `int face_mask_`. 3. In `ComputeDamping`, gate each half-side on its bit, e.g. `if (face_mask_ & FaceXLo) { real_t d=(x_min_(0)+L_pml_)-x; if (d>0) dx=DampingProfile(d); } if (face_mask_ & FaceXHi) { real_t d=x-(x_max_(0)-L_pml_); if (d>0) dx=std::max(dx,DampingProfile(d)); }` and likewise `y/z`. Keep `DampingProfile`, `Dx/Dy/Dz`, `d_max_`, accessors unchanged. Keep the `dirs` ctor argument for source compatibility. 4. **(Optional, from the Eq.17 caveat)** if the team elects `true-n = 3` grading, change the `d_max_` prefactor in `pml_layer.cpp:44` from `3.0/2.0` to `2.0` and update the 4 tests’ tolerance labels; otherwise leave it and rename the config knob `pml_target_R` semantics to “effective R” in the 12.2 banner. Pick one; record the choice in the check doc.

Edge cases: `mask==-1` → exact today’s behavior (regression-critical for the 4 tests); `mask==0` → no damping (degenerate, allowed — “ABC-only” A/B); a point in a disabled half-face returns 0 for that direction even if geometrically inside the shell.

Acceptance criteria: - [] The 4 existing `test_pml.cpp` tests pass **unchanged** (default mask = symmetric). - [] New `TestPMLFreeSurfaceTopUndamped`: PML on XLO|XHI|YLO|YHI|ZLO (not ZHI) ⇒ `dz==0` within `L_pml` of `z_max`, `dz>0` within `L_pml` of `z_min`. - [] New `TestPMLFreeSurfacePulseStable`: a P-pulse reflecting off the undamped `z_max` stays finite for 500 steps (no NaN); the `z_min` side absorbs (interior energy decays). - [] make test green.

Dependencies: depends on nothing; required by 12.2.

Phase 12.2 – Construct + wire the PML in seas_spatial_dyn_driver

Goal. When enabled, the driver builds a PMLayer from the mesh bbox + config, calls `wave.SetPML(&pml)`, and the banner reports real parameters; when disabled, behavior is byte-identical to today.

Files to modify: `spatial/code/spatial_friction.hpp` (+ `.cpp`), `drivers/spatial_dyn_driver.cpp`.

Detailed requirements: 1. **Config fields** added to `NumericsSpec` (the struct that already holds `use_pml`, `cfl_safety`, `fault_iterator`, `interior_flux` — Phase 6 req 3), defaults meaning “derive”: `real_t pml_thickness_m=-1.0` (`<0` ⇒ `pml_cells*lc_far`); `real_t pml_target_R=1e-3`; `int pml_cells=4`; `bool pml_damp_bottom=true`; `bool pml_damp_top=false` (**MUST stay false for**

SAFS). Parse in `parse_root`'s `[numerics]` block (where `cfl_safety/interior_flux` are read) via `toml_bool/toml_real/toml_int`. 2. **CLI overrides** alongside `--pml (:533)`: `--pml-thickness <m>`, `--pml-target-R <r>`, `--pml-cells <n>`, `--pml-damp-bottom {0,1}`, `--pml-damp-top {0,1}` via `GetRealArg/GetIntArg/HasFlag`; CLI wins over config (the `cli_pml→use_pml` pattern at :625). 3. **lc_far source**. `MeshSpec` is currently `{path, order}` only — **add real_t lc_far_m=-1.0 to MeshSpec** (+ parse from `[mesh]`). Derived thickness $L_{\text{pml}} = \text{pml_cells} * \text{cfg.mesh.lc_far_m}$. If `lc_far_m` is unset **and** no `--pml-thickness` is given, **abort** with a clear message (do not hardcode 3000 in C++). 4. **Construct after the bbox is known**. The reflection-warning block (:955–980) computes `pmesh.GetBoundingBox(lo,hi,1)` and `cp`; refactor so `lo/hi/cp` survive past it. Then, only when `cfg.numerics.use_pml`, build the face mask (`FaceXLo|FaceXHi|FaceYLo|FaceYHi, |FaceZLo` if `pml_damp_bottom`, `|FaceZHi` if `pml_damp_top`), construct `pml = std::make_unique<PMLayer>(lo,hi,L_pml,cp,cfg.numerics.pml_target_R,7,face_mask)`, and call `wave.SetPML(pml.get())` **after** `SetAbsorbingBackground (:1336)` and **before** the time loop. Guard: `MFEM_VERIFY(wave.GetAbsorbingBackground()!=nullptr, "PML requires a total-Q background")` (the API exists — `wave_operator.hpp:373`). On **restart** (`!restart_prefix.empty()`) PML is stateless — reconstruct identically post-restart; verify `bbox/cp` recompute the same. 5. **Banner (:758)**: when enabled, print real L_{pml} , R_0 (or R_{eff}), d_{max} , the PML inner edges and the fault→PML clearances (computed from `lo/hi`, L_{pml} , and the fault `bbox` if known), and the damped-face list incl. “free surface $z=0$ undamped”. 6. The `!cfg.numerics.use_pml` guard on the reflection warning (:974) already suppresses it correctly — leave as-is.

Edge cases: `use_pml=false` ⇒ no `PMLayer`, no `SetPML` ⇒ byte-identical to today; `pml_damp_top=true` on **SAFS** ⇒ allowed but log a prominent **WARNING** (damping $z=0$ is unphysical for a half-space); $L_{\text{pml}} \geq \text{buffer}$ (PML reaching the fault) ⇒ **abort** with the computed clearance + a remedy (reduce `pml_cells/thickness` or remesh).

Acceptance criteria: - [] `use_pml=false`: a short **SAFS** smoke is byte-identical to the current binary (diff the first 200 step/[DIAG] lines). - [] `--pml`: banner prints real $L_{\text{pml}}/R_0/d_{\text{max}}$ /inner-edges/clearances; `wave.GetPML()!=nullptr`. - [] A 2–4 rank local `--pml` smoke runs with no `has_bulk_bg_assert` and no NaN through nucleation. - [] `make seas_spatial_dyn_driver` clean; `make test` green.

Dependencies: depends on 12.1; required by 12.3.

Phase 12.3 – Validate PML effectiveness on the SAFS problem

Goal. Quantitatively show the PML removes post-7 s reflection contamination and classify the $t \approx 28$ s LSW V_{max} departure.

Files to create: `safs/project_7.0_alternative/spatial/code/scripts/pml_reflection_metrics.py`; three sbatch variants (`pml_on`, `pml_off`, `bigbox_ref`).

Detailed requirements (increasing strength): 1. **A/B $V_{\text{max}}(t)$ overlay** — identical configs `--pml` vs no-PML to $t_{\text{final}} \geq 34$ s (ideally 40–50 s). **Decision rule**: if the $t \approx 28$ s departure disappears/strongly delays with PML → reflection-driven; if it persists → intrinsic friction/LSW instability. Either is a definitive result. 2. **Reflection-free-window extension** — [DIAG-SIGN] $\sigma_{n_{\text{min}}}(t) / V_{\text{max}}(t)$ stay smooth past 7 s (no kink at the reflection arrival). 3. **Big-box reference (gold standard)** — remesh `--pad-x/--pad-y 100000 --pad-bottom 60000` ($t_{\text{reflect}} \approx 14\text{--}17$ s), **no PML**; over $[0, \sim 14$ s] it is reflection-free truth. **Accept PML** if $\max_t$

$|V_{\max}^{\text{PML}} - V_{\max}^{\text{bigbox}}| / V_{\max}^{\text{bigbox}} < 5\%$. 4. **Near-boundary seismogram** — 2–3 receivers ~one wavelength inside the PML inner edge; reflected/incident peak ratio target $\leq \sim 1\%$. 5. **Interior energy monitor** — elastic energy over the **interior only** (exclude the PML shell) rises during rupture then decays monotonically; no $t \approx 7$ s re-injection bump (reuse EnergyDensity from test_pml.cpp).

Acceptance criteria: - [] $V_{\max}(t)$ PML-vs-no-PML overlay produced; the $t \approx 28$ s departure classified per req 1. - [] max relative $V_{\max}$ difference (PML vs big-box) $< 5\%$ over the big-box window. - [] near-boundary reflected/incident peak ratio $\leq \sim 1\%$. - [] interior energy decays monotonically (no $t \approx 7$ s bump). - [] PML step rate within $\pm 20\%$ of no-PML.

Dependencies: depends on 12.2; required by 12.4 (optional).

Phase 12.4 (optional) – Production integration + deep-box remesh

Goal. Promote the validated PML to production sbatches; ship a deeper-bottom mesh only if 12.3 flags the bottom buffer.

Detailed requirements: 1. Add `--pml (+ [numerics] use_pml=true, pml_cells, pml_target_R)` to the normal-queue parents + dev smokes, with a banner-grep guard (“PML: ACTIVE (L=... faces=...)”) mirroring the floor-banner guard. 2. **Only if 12.3 req-3/4 fail at the bottom:** remesh `run_z0cut_meshing.py --pad-bottom 40000` (deepen 15 km, ≥ 25 km bottom clearance), regenerate the triq mesh, rerun 12.3 metrics. 3. No change to `scripts/estimate_output_size.py` (PML registers no output fields).

Acceptance criteria: - [] Production sbatch banners confirm PML active; a restart-chain dev run resumes with PML and stays bounded. - [] (If remeshed) bottom clearance ≥ 25 km; 12.3 metrics pass.

Dependencies: depends on 12.3.

Risk assessment (PML)

- **R1 ADER-corrector path** — PML is intentionally skipped in the predictor (`wave_operator.hpp:427`); the SAFS `ader_order=2` corrector must be the active path. *Detect:* `--pml` must show interior-energy decay; if PML looks inert, instrument the corrector branch with a one-time rank-0 “PML corrector active”.
- **R2 free-surface damping (main trap)** — if `FaceZHi` is ever enabled, the PML kills the free surface. *Detect:* 12.1 `TestPMLFreeSurfaceTopUndamped` + the 12.2 banner `face-list` + the `pml_damp_top` WARNING.
- **R3 PML reaching the fault** — `L_pml > buffer` (esp. the 13 km bottom) arrests slip. *Detect:* 12.2 clearance abort + banner clearance line + 12.3 big-box agreement.
- **R4 long-period leakage** — 12–15 km is ~ 0.5 long-period Rayleigh wavelengths. *Detect:* near-boundary seismogram (12.3 req 4); mitigate by thickening x/y PML or relying on the 35–50 km side buffer.
- **R5 cost** — per-QP damping over the thin PML shell only (if `d_x=d_y=d_z=0` continue). *Detect:* 12.3 step-rate comparison ($< \text{a few } \%$).
- **R6 restart determinism** — PML is stateless but `bbox/cp` must recompute identically. *Detect:* a checkpoint→restart smoke reproduces the pre-checkpoint $V_{\max}$ to round-off.

File-touch summary

Step	File	Change
12.1	dynamic/pml_layer.hpp	half-face mask enum + optional ctor arg + GetFaceMask; (opt.) note prefactor
12.1	dynamic/pml_layer.cpp	honor face_mask_ in ComputeDamping; derive from dirs when -1; (opt.) 2c_p/L
12.1	tests/unit/test_pml.cpp	+2 free-surface tests; keep the 4 existing
12.2	spatial/code/spatial_friction.hpp	+5 PML NumericsSpec fields; +lc_far_m on MeshSpec
12.2	spatial/code/spatial_friction.cpp	+5 PML fields + lc_far_m in [numerics]/[mesh]
12.2	drivers/spatial_dyn_driver.cpp	construct PMLayer from bbox+config, SetPML, CLI flags, banner, clearance + Q_bg guards
12.3	safs/.../scripts/pml_reflection_metrics_tool	new metrics tool
12.3	jobs/safs/*pml*	A/B + big-box sbatch variants
12.4	production sbatches / mesher	optional promotion + deep-bottom remesh

Dependencies

Depends on: Phase 3 (the SAFS+RS driver path) + the existing PML machinery. Independent of Phases 8–11. Required by: nothing (it is an orthogonal capability that unblocks the reflection-vs-instability question for the SAFS LSW/RS runs).

10 Testing strategy

- » **Near-future milestone (Phases 1–3):** tpv102_nuc_callback_parity (byte-safe overload, **R-008 method-pinned** + a Brent smoke) + seed_equilibrium_psi_rs (steady-residual, **R-001 d.eta_s + v_init>0**, **R-009 V0-guard** case) + advance_interface_compiles (**R-010** compile-only) + friction_iterator_factory (dispatch) + rate_state_config_no_abort (**R-002** dry-run) + resolve_rate_state_guards (**R-003** no-double-PP, **R-006** per-DOF reject, **R-011** a>b allowed) + paraview_state_channel_rs (**R-005**) + constant_tensor_sign (**R-007** parametrized over all flipped configs) gate the SAFS+RS wiring; the deliverable is the **8-rank SAFS+RS smoke** (locked at V_init pre-nucleation, physical rupture after), with the **R-004 two-pronged** ψ -consistency check (cross-rank bit-identity + temporal interior-vs-shared drift) as the open risk gate.

- **Config + sign contract (cheap, deterministic, no MPI):** `tpv_toml_stress_sign` (D3.2), `constant_tensor_sign` (D3.1 bit-identity, parametrized), `resolve_rate_state_guards`, `*_review`, `config_phaser`, `stress_with_patches`, the factory tests — these gate every phase and are the practical regression without an HPC reference.
- **Unification parity (the byte-exactness gate):** `friction_substep_iterator_parity` asserts the unified iterators reproduce the standalone `tpv{205,102,104}_substep_iterator` (the oracle, kept untouched) bit-for-bit across $O \in \{1,2,3\}$.
- **Riemann unit test:** `godunov_flux_bimaterial` checks `ApplyPerFaceFlux` against hrs-ref reference values and against the scalar Godunov flux in the equal-material limit.
- **Dispatch smoke:** `--dry-run/--verify-dispatch` confirm law + iterator + nucleation + flux + material + station writer per case (incl. TPV31 $\rightarrow$ matrix + `depth_profile_1d`).
- **Physics regression:** 8-rank `tfinal=0.2s` smoke then full run (cluster) vs gold/SCEC for TPV205/102/104; TPV31 vs the eqdyna/seisol overlays. The standalone `tpv*_driver.cpp` remain available for same-mesh cross-checks (the byte-exactness oracle).
- **MPI parity ($np > 1$):** forced-2-rank single-shared-fault-face cross-rank `DOFData/flux` bit-identity (friction); heterogeneous-mesh interior-face flux bit-identity (bimaterial).
- **SAFS guards:** LSW no-regression diff (incl. the D3.1 config-sign flip $\rightarrow$ byte-identical); RS shared-fault ψ -consistency — **R-004 two-pronged:** `shared_fault_rs_psi_consistency_np2` checks (a) cross-rank `d.psi` bit-identity post-reconcile **and** (b) bounded temporal interior-vs-shared ψ drift at the same QP
- **Depth-varying a/b (Phase 11):** `friction_depth_profile` (`PiecewiseLinear1D` flat-clamp + linear midpoints; two-CSV loader with `km  $\rightarrow$  m` scaling; `b = a - (a-b)` on differing grids; reject `<2` rows / dup depths / non-positive a) + `spatial_friction_resolver` CSV-profile case (a deep VS DOF with `a > b` and a shallow VW DOF with `a < b`) + `seed_equilibrium_psi_rs` depth-varying-b case. **Byte-exact gate:** `ader_tpv102_smoke` + `tpv102_setup` (asserts `d.b == TPV102Params::b`) stay bit-identical after the iterator's `GetB()` $\rightarrow$ `d.b` switch; a scalar-RS config (no profile block) asserts `rs.b(i) == b_default` (net-zero when no profile).

11 Risk assessment

Risk	Likelihood	Detection / mitigation
Shared-fault RS ψ 1st-order at $O \geq 2$ (review R-004)	MED-HIGH	Bounded (mirrors the working LSW slip path; reconcile broadcasts ψ so it is <i>temporal</i> , not desync). Phase-3 two-pronged gate: cross-rank bit-identity + temporal interior-vs-shared drift; $np > 1$ RS stays “open until validated”; may open a sub-step- ψ -on-shared-QPs follow-up; do not assume LSW correctness transfers

Risk	Likelihood	Detection / mitigation
RS code won't compile / aborts (review R-001/R-002/R-010)	(closed)	R-001 d.eta_s (not rs.eta_s) + include + V_init>0 guard; R-002 gate slip_weakening.has_value() under is_lsw, outer-scope lsw+rs; R-010 rename :439→Advance + add GetDeltaT/GetTimeWeights to the interface. Caught by seed_equilibrium_psi_rs (build) + advance_interface_compiles + rate_state_config_no_abort (dry-run)
Dual-source V0: silent wrong friction if $V_0 \neq 1e-6$ (review R-009)	MED (CRIT if tuned)	Force solve hardcodes FrictionSolver::V0; seed+aging use config V_0. Phase-1 guard asserts equality (deliverable uses 1e-6); proper fix = thread config V0 into the solver (gated follow-up, no-hardcoded-numbers). Caught by seed_equilibrium_psi_rs V0 case
Latent RS inconsistencies (review R-003/R-005/R-006)	(closed)	R-003 PorePressureSpec{} (no double-PP); R-005 ParaView state→ ψ ; R-006 reject per-DOF b/V_0/f_0. Caught by resolve_rate_state_guards + paraview_state_channel_rs
a<b validator blocks a bounded rupture (review R-011)	LOW	Fully-VW first smoke is unaffected; Phase-3 relaxes a<b→allow a>b (keep a, b>0) so a tpv102-like arrest border is configurable. Caught by resolve_rate_state_allows_velocity_stren
Matrix Riemann: scalar path regresses during the port	MED	scalar runs byte-unchanged vs pre-Phase-9 checkpoint; owned_flux_pool==nullptr takes no new branch

Risk	Likelihood	Detection / mitigation
Matrix Riemann: SeisSol vs MFEM matrix sign convention	MED	equal-material limit must reduce to scalar Godunov to round-off; godunov_flux_bimaterial unit test vs hrs-ref values
New bimaterial collective deadlocks (R-1600)	MED	gate ExchangeBiMaterialNeighbours_ on GetNSharedFaces()>0, not shared-fault faces; np>1 parity test
Driver integration drops a safs fix	MED	§5.5 “stays” checklist + SAFS LSW no-regression diff
Unified iterator drifts from the oracle	LOW	friction_substep_iterator_parity bit-for-bit vs standalone iterators; standalone iterators untouched
D3.1 half-applied (factory negation XOR config flip)	LOW	constant_tensor_sign golden test fails loudly on a half-done change; land both together
D3.2 fault-local seeding wrong (slot / sign)	LOW	direct seeding (no projection); tpv_toml_stress_sign asserts tau2_0==+ τ _ini + canonical frame
TPV31 cohesion term perturbs other cases	LOW	C0=0 reproduces the pre-cohesion LSW path byte-for-byte (additive, guarded)
Equilibrium- ψ seed wrong (creep/lock at t=0)	LOW	seed_equilibrium_psi_rs residual test (Phase 1) + the SAFS-RS smoke lock check (Phase 3)
Missing .msh for tpv104/tpv205/tpv31	HIGH (known)	Phase-8/10 gmsh step is a hard prerequisite (does NOT block SAFS+RS, which uses the existing 500 m mesh)
Reference identifier drift (DieterichRuinaFriction::Constants order; Tpv102 overload insertion; hrs-ref BimaterialFlux API)	LOW	confirm at impl time; flagged in Phases 1/3/9/10

Risk	Likelihood	Detection / mitigation
Per-DOF b breaks the TPV102 byte-exact oracle (Phase 11a)	MED	Tpv102SubStepIterator is shared; the <code>GetB()→d.b</code> switch is a no-op ONLY if <code>d.b == TPV102Params::b</code> for native TPV102. Mitigation: set <code>d.b = TPV102Params::b</code> in <code>tpv102_setup.hpp</code> ; gate on <code>ader_tpv102_smoke + tpv102_setup</code> (asserts the equality) staying bit-identical
Lifting R-006 silently no-ops per-DOF b (Phase 11a)	LOW	R-006 was correct pre-slot; lift ONLY after <code>DOFData.b</code> routes all 3 iterator sites + 2 seed sites. Mitigation: scalar-RS test asserts <code>rs.b(i)==b_default</code> ; depth-varying-b seed test asserts different ψ per depth — proves the knob is live, not dropped
CSV depth-unit (km vs m) $1000\times$ trap (Phase 11b)	MED	CSV depth column is km but the mesh is metres; a missed $\times 1000$ maps the whole profile into the top 60 m. Mitigation: explicit <code>depth_units</code> key (default "km" $\rightarrow$ <code>depth_to_m = 1000</code> , no hard-coded constant); <code>friction_depth_profile</code> asserts the km $\rightarrow$ m scaling on the committed files
$b = a - (a-b)$ goes non-positive (Phase 11b/c)	MED	the two CSVs can make $b \leq 0$ at some depth (data error). Mitigation: per-DOF <code>b_i > 0</code> validator aborts with a message naming <code>param_a_minus_b.csv</code> + the offending depth; surfaced at config load, never silently clamped

Risk	Likelihood	Detection / mitigation
Depth-profile flat-clamp vs linear extrapolation (Phase 11b)	LOW	linear extrapolation past the last CSV sample could push a/b non-physical. Mitigation: each curve constant (flat) clamped outside [front,back]; friction_depth_profile asserts the clamp
Two CSVs on different depth grids (Phase 11b)	LOW	a and a-b files have different depth samples (0/14.9/52.7 vs 0/13.7/60 km). Mitigation: interpolate each independently and subtract at query time (FrictionDepthProfile1D::b), never pre-merge onto one grid; resolver test covers it
Profile CSV path resolution / missing file (Phase 11b)	LOW	param_*_csv paths are CWD-relative like [mesh].path; read at PARSE time so a bad path aborts at config-load with a precise message, not mid-run

11.1 Phase 13 – Split scalar / matrix into separate WaveOperator classes (kill the flux_placeholder leak class)

Overview

Phases 9 folded the heterogeneous (bimaterial / interior_flux="matrix") path into the **same** WaveOperator class as the scalar (homogeneous) path, using a single flux_ member that is the real material on the scalar ctor and a **(1,1,1) placeholder** on the matrix ctor. Every site that reads material via bare flux_ instead of the per-element accessor FluxForElem_(e) is therefore correct on scalar but silently (1,1,1)-wrong on matrix — a leak class that recurred across three review rounds (boundary faces, the bulk volume RHS, the fault imposed-state flux; see REVIEW.md rounds 3–4). Phase 13 makes the two paths **physically separate classes** so the leak is impossible by construction:

- **WaveOperator<MeshType>** stays the **SCALAR operator** — the exact class the standalone tpv{102,104,205}_driver.cpp and seas_bp5_full instantiate, and the interior_flux="scalar" branch of spatial_dyn_driver.cpp. Its scalar numerics are **byte-unchanged**; the TPV/BP5 byte-exactness oracle is preserved trivially. All bimaterial members and the het ctor are **removed** from it.
- **BimaterialWaveOperator<MeshType>** : **public WaveOperator<MeshType>** is a NEW, separate class (new files) that holds the per-element flux pool + bimaterial flux matrices + cross-rank exchange, and **overrides** the handful of material-/flux-dependent virtual hooks to use per-

element material at every site. It contains no scalar placeholder logic, so a missed site cannot silently run (1,1,1) — and the inherited `flux_` is **poisoned** so any stray use fails loud.

This is the user-chosen “two separate options” (2026-05-28): `scalar ==` the TPV-driver operator, `matrix ==` a separate class.

Constraints

- **Scalar byte-exactness (non-negotiable).** `WaveOperator`’s scalar output must be bit-identical before/after this phase: `tpv{102,104,205}` station traces, the BP5 path, and the scalar SAFS runs. The only permitted changes to `WaveOperator` are (a) removing bimaterial members/methods, (b) adding protected virtual hooks whose **default bodies are the current scalar code verbatim**, and (c) routing the bare-`flux_` material sites through `FluxForElem_(e)` (which on the scalar class returns `flux_` — byte-identical).
- **TPV/BP5 drivers must not change.** They construct `WaveOperator<Mesh>`; that type, its public API, and its behavior stay identical. (Making a few methods virtual does not change scalar numerics; `FluxForElem_(e)` is per-element/per-face, not in the innermost QP loop, so the virtual-call overhead is negligible.)
- **No duplication of the ~7000-line DG/ADER skeleton.** The skeleton (`Mult`, `AdvanceADER`, `ComputeADER*`, `ComputeVolumeRHS`, `ComputeFaceFluxRHS`, ...) lives ONCE in `WaveOperator` and is inherited by `BimaterialWaveOperator`, which runs the same skeleton with material behavior injected through the overridden virtuals. (A full duplicate class was the rejected alternative — a $2\times$ maintenance burden.)
- **Reference is hrs-ref.** Every per-element override body must reproduce the hrs-ref implementation (`git show hrs-ref:miniapps/seas/dynamic/wave_operator.inl`), which uses `FluxForElem_(...)` at every material site.
- **Driver-facing base interface unchanged.** The 18 `wave.*` calls in `spatial_dyn_driver.cpp` (`AdvanceADER`, `ComputeADERSubStepStates`, `ComputeMaxDt`, `EvaluateBulkAtFaultQPsCanonical`, `GetCentralFluxFaceSet`, `GetFaultInteriorFaces`, `GetFaultSharedFaces`, `GetNumLocalFaultQPs`, `GetNumTotalFaultQPs`, `GetScalarNDof`, `SetAbsorbingBackground`, `SetFaultD0FData`, `SetFaultFlux`, `SetFaultFrictionLaw`, `SetMixedFluxMode`, `SetSubStepFaultImposedStates`, `SetTime`, `VerifySharedFaultD0`) must remain on the `WaveOperator` base so the driver can hold a `std::unique_ptr<WaveOperator<ParMesh>>` and call through it for both paths.

Chosen class structure (precise)

```
// dynamic/wave_operator.{hpp,inl} - SCALAR (unchanged behavior; TPV/BP5/scalar-SAFS operator)
template <typename MeshType = Mesh>
class WaveOperator : public TimeDependentOperator {
    // ... all existing scalar state + the DG/ADER skeleton (Mult/AdvanceADER/ComputeADER*/...) ..
    GodunovFlux flux_; // scalar material (KEPT)
protected:
    // --- material/flux dispatch hooks (NEW; defaults == current scalar code) ---
    virtual const GodunovFlux& FluxForElem_(int e) const { return flux_; }
    virtual void InteriorFaceFlux_(* see §13 hook 2 */) const; // scalar: flux_.Interior / mi
    virtual void ApplyElementJacobian_(int dir, const Vector& X, Vector& Y,
```

```

                                real_t sign) const;           // scalar: ApplyJacobianPerDOF
public:
    virtual real_t ComputeMaxDt(real_t cfl) const;           // scalar: cfl_mixed_flux_fact
    virtual void    SetMixedFluxMode(MixedFluxMode m);       // scalar: as today
    // ... UsesGodunovFluxPool() returns false here ...
};

// dynamic/bimaterial_wave_operator.{hpp,inl} - MATRIX (separate, new)
template <typename MeshType = Mesh>
class BimaterialWaveOperator : public WaveOperator<MeshType> {
    std::unique_ptr<GodunovFluxPool> pool_;                 // per-element flux
    const MaterialField* material_ = nullptr;              // non-owning
    std::vector<std::array<real_t,3>> per_elem_lmr_;
    std::vector<real_t> per_elem_h_;
    std::unordered_map<int,std::array<real_t,3>> shared_face_neighbour_material_;
    std::vector<std::array<std::array<DenseMatrix,2>,2>> per_face_bimaterial_flux_;
    mutable std::size_t phaser_dispatch_count_ = 0;
public:
    BimaterialWaveOperator(MeshType& mesh, int order, const MaterialField& material,
                           const BoundaryConfig& bc);       // builds pool + matrices + ex
protected:
    const GodunovFlux& FluxForElem_(int e) const override { return pool_->At(e); }
    void InteriorFaceFlux_(* ... *) const override;        // BimaterialFlux::ApplyPerFace
    void ApplyElementJacobian_(int dir, const Vector& X, Vector& Y,
                                real_t sign) const override; // ApplyJacobianPerElementDOF
public:
    real_t ComputeMaxDt(real_t cfl) const override;         // per-element per_elem_lmr_ w
    void    SetMixedFluxMode(MixedFluxMode m) override;     // aborts on m != None
    // UsesGodunovFluxPool() returns true here
};

```

Why inheritance (not a third base class): it leaves WaveOperator’s scalar code in place (only adds virtual keywords + sheds bimaterial members), best honoring “keep scalar the same as the TPV driver”. The cost is that BimaterialWaveOperator inherits an unused flux_; this is neutralized by poisoning it (§13.2 req 4). A sibling-with-abstract-base design (cleaner inheritance, no inherited flux_) was rejected because extracting a base from the 7000-line class disturbs the scalar oracle more than adding hooks.

Files to Create

- dynamic/bimaterial_wave_operator.hpp — the BimaterialWaveOperator<MeshType> class declaration (members + overrides + ctor), #include "wave_operator.hpp" and #include "godunov_flux_bimaterial.hpp".
- dynamic/bimaterial_wave_operator.inl — the override bodies + ctor (the bimaterial machinery moved out of wave_operator.inl).
- tests/unit/test_bimaterial_wave_operator_parity.cpp — the fault-bearing Coefficient-

vs-scalar parity test (C-6, §Testing).

Files to Modify

- `dynamic/wave_operator.hpp` — remove the bimaterial member declarations (`owned_flux_pool_`, `material_`, `per_elem_lmr_`, `per_elem_h_`, `shared_face_neighbour_material_`, `per_face_bimaterial_flux_`, `phaser_dispatch_count_`) and the het-ctor / het-method declarations (`BuildGodunovFluxPool_`, `ExchangeBiMaterialNeighbours_`, `BuildPerFaceBimaterialFluxMatrices_`, `ApplyJacobianPerElementDOF_`); add the protected virtual hook declarations; make `FluxForElem_`, `ComputeMaxDt`, `SetMixedFluxMode` virtual; `UsesGodunovFluxPool()` returns false.
- `dynamic/wave_operator.inl` — remove the het ctor + the four `BuildGodunovFluxPool_ / ExchangeBiMaterialNeighbours_ / BuildPerFaceBimaterialFluxMatrices_ / ApplyJacobianPerElementDOF_` bodies; replace every `if (owned_flux_pool_) { bimaterial } else { scalar }` dispatch with a single call to the virtual hook (the scalar branch becomes the hook's default body); route the volume / boundary / fault-imposed-state material sites through `FluxForElem_(e)` (fixing REVIEW round-4 R-001 in the shared skeleton). The CFL het branch (`if(!per_elem_h_.empty())`) moves to the override.
- `drivers/spatial_dyn_driver.cpp` — the matrix branch constructs `std::make_unique<BimaterialWaveOperator>` (order, material, bc) (held as `unique_ptr<WaveOperator<ParMesh>>`); the scalar branch is unchanged. Remove the now-obsolete `material.mode != Constant` guard placement comment if it no longer applies; keep the Phase-10/DepthProfile1D guard.
- `Makefile` — add `BIMATERIAL_WAVE_OPERATOR_{SRC,OBJ}` (header-only .inl, so likely just a test obj rule + the new parity-test target); link the new test; add to the aggregate test:.
- `tests/unit/test_phaseh_wave_operator_constant_parity.cpp` — repoint the C-5 Coefficient-mode construction from `WaveOperator` to `BimaterialWaveOperator`; keep the parity assertion.

Detailed Requirements

11.1.6.1 The four virtual hooks (exact intent; defaults == current scalar code, verbatim)

1. **`virtual const GodunovFlux& FluxForElem_(int e) const`** — per-element material for the bulk volume Jacobian, the boundary-face flux, and the fault imposed-state flux.
 - `WaveOperator` (scalar): return `flux_`;
 - `BimaterialWaveOperator`: return `pool_->At(e)`;
 - **Consumers in the shared skeleton (route ALL through this hook):** `ComputeVolumeRHS` (`wave_operator.inl:1294` per-element `Ax_e/Ay_e/Az_e`); the 8 boundary-flux sites (`flux_.{AbsorbingTotal,FreeSurfaceGodunovTotal,FreeSurfaceTotal} → FluxForElem_(e1)....`, `:2846,:2859,:2864,:2900,:4821,:4826,:4831,:4841`); the 8 fault imposed-state sites (`flux_.Interior(can_n..., Q_imp/I_imp...) → FluxForElem_(elem_plus/elem_minus/e1).Interior(...)`, `:3151,:3153,:3824,:4416,:4418,:4746,:4749,:5806`). **The fault sites are REVIEW round-4 R-001 — fix them here** by porting hrs-ref's per-side element selection (`elem_plus = elem1_on_plus ? e1 : e2; elem_minus = elem1_on_plus ? e2 : e1`; and the per-QP `elem1_on_plus_per_qp[qq]` variants — hrs lines 3453/3455/4135/4730/4732/5056/5059/5713). On the scalar class `FluxForElem_(e)==flux_`, so this is byte-identical for scalar.
2. **`virtual void InteriorFaceFlux_(int face, int side, const real_t* Q_self, const real_t* Q_nbr, const real_t* nor, real_t* F_h) const`** — the interior NON-fault face flux.

(The implementer extracts the existing `if(owned_flux_pool_){...}else{...}` interior branches into this hook; the exact parameter list is whatever the four call sites need — pass the per-site locals. The 4 sites: `ComputeFaceFluxRHS ~:3223`, `ComputeSharedFaceFluxRHS ~:3807`, `ComputeADERFaceFluxRHS ~:4811`, `ComputeADERSharedFaceFluxRHS ~:5607`.)

- `WaveOperator` (scalar): the current else body — `flux_.Interior(...)` plus the mixed-flux central-vs-interior per-face choice (`central_flux_face_set_ / flux_.Central`).
 - `BimaterialWaveOperator`: the current `if(owned_flux_pool_)` body — `BimaterialFlux::ApplyPerFace` [...][1], `Q_self`, `Q_nbr`, `F_h`).
3. **virtual void ApplyElementJacobian_(int dir, const Vector& X, Vector& Y, real_t sign) const** — the ADER CK-recursion element Jacobian (2 sites: :1526, :1655).
 - `WaveOperator` (scalar): `ApplyJacobianPerDOF(flux_.GetReferenceStarMatrix(dir), X, Y, ndof_total_, sign);`
 - `BimaterialWaveOperator`: `ApplyJacobianPerElementDOF_(dir, X, Y, sign);` (the moved method).
 4. **virtual real_t ComputeMaxDt(real_t cfl) const** — `CFL` (:6121–:6151).
 - `WaveOperator` (scalar): `return cfl_mixed_flux_factor * cfl * h_min_ / flux_.GetCp();` (drop the `if(!per_elem_h_.empty())` branch — that was the het path).
 - `BimaterialWaveOperator`: the per-element walk over `per_elem_lmr_ / per_elem_h_` with the `MPI_Allreduce(MIN)` (the current :6121–:6148 body).

11.1.6.2 Member / method relocation (`wave_operator` → `bimaterial_wave_operator`)

Move verbatim into `BimaterialWaveOperator`: members `owned_flux_pool_ (→pool_)`, `material_`, `per_elem_lmr_`, `per_elem_h_`, `shared_face_neighbour_material_`, `per_face_bimaterial_flux_`, `phaser_dispatch_count_`; methods `BuildGodunovFluxPool_`, `ExchangeBiMaterialNeighbours_`, `BuildPerFaceBimaterialFluxMatrices_`, `ApplyJacobianPerElementDOF_`. The het ctor body (`material_=&material;` `BuildGodunovFluxPool_`; `ExchangeBiMaterialNeighbours_`; `BuildPerFaceBimaterialFluxMatrices_`) becomes `BimaterialWaveOperator`’s ctor, run AFTER delegating to the scalar base ctor (§13.2 req 4).

11.1.6.3 Phase 13.1a — Virtualize the dispatch in `WaveOperator` (pure refactor; matrix path still in-place)

- Add the four hooks as protected `virtual` with the scalar default = current else/scalar code.
- Replace each `if(owned_flux_pool_){bimaterial}else{scalar}` with a call to the hook, and TEMPORARILY keep the bimaterial members + a `WaveOperator`-internal override-equivalent so the het constant-parity test still builds (i.e., during 13.1a the bimaterial branch bodies move INTO the default hook bodies guarded by `owned_flux_pool_`, an interim form). Route the volume/boundary/fault material sites through `FluxForElem_(e)` (fixes R-001 for both paths).
- **Acceptance:** zero behavior change — scalar TPV/BP5 byte-exact AND the existing `test_phaseh_wave_operator_constant_parity` (C-1..C-5) still passes bit-for-bit. This is the “make the seam virtual” step.

11.1.6.4 Phase 13.1b — Extract `BimaterialWaveOperator`; make `WaveOperator` pure scalar

- Create `dynamic/bimaterial_wave_operator.{hpp,inl}`; move the bimaterial members + the

four het methods + the het-ctor body into it as the ctor + hook overrides.

- Delete the bimaterial members, het ctor, het methods, and the owned_flux_pool_ guards from WaveOperator; the scalar hook bodies become unconditional scalar code (no owned_flux_pool_).
- WaveOperator::SetMixedFluxMode keeps the scalar/mixed-flux logic; BimaterialWaveOperator::SetMixedFluxMode overrides to MFEM_ABORT on m != None (the R-003 guard, now structural).
- **Acceptance:** WaveOperator has zero references to owned_flux_pool_/per_face_bimaterial_/GodunovFluxPool/BimaterialFlux (grep == 0); scalar TPV/BP5 byte-exact; the constant-parity test (repointed to BimaterialWaveOperator) passes.

11.1.6.5 Phase 13.2 — Driver dispatch, fault-flux correctness, tripwire, parity test

1. spatial_dyn_driver.cpp matrix branch: wave_ptr = std::make_unique<BimaterialWaveOperator<ParMesh>(cfg.mesh.order, material, bc); (scalar branch unchanged). WaveOperator<ParMesh>& wave = *wave_ptr; still binds the base; all wave.* calls dispatch virtually.
2. Confirm the 8 fault imposed-state sites (R-001) use FluxForElem_(elem_plus/elem_minus/e1) (done in 13.1a, now correct on the matrix path because the override returns pool_→At(e)).
3. BimaterialWaveOperator::SetMixedFluxMode aborts on non-None (R-003 mutual exclusion, structural).
4. **Sentinel-poison the inherited flux_ + parity tripwire (NaN is infeasible — verified).** BimaterialWaveOperator's ctor delegates to the scalar base ctor with the **valid sentinel triple** $(\lambda, \mu, \rho) = (1, 1, 1)$ (NOT NaN). On the matrix object flux_ must be dead (every material access goes through the overridden FluxForElem_(e) → pool_); the sentinel value is deliberately unphysical so a leak is numerically visible (see the tripwire below).
 - **Why not NaN:** GodunovFlux::GodunovFlux asserts MFEM_VERIFY(mu>0); MFEM_VERIFY(rho>0); MFEM_VERIFY(lambda+2*mu>0) (godunov_flux.cpp:30–33). Since NaN > 0 is false, the base ctor's flux_(NaN,NaN,NaN) would **abort construction of every matrix run**, not just buggy ones — so NaN cannot be used as the poison. (1,1,1) passes the asserts (1>0, 3>0).
 - **Precondition (verified):** the scalar base ctor consumes flux_ ONLY via the member-init flux_(λ, μ, ρ) and flux_.BuildJacobian(0/1/2, Ax_/Ay_/Az_) (wave_operator.inl:30,55–57), whose outputs Ax_/Ay_/Az_ are dead after the volume RHS routes through FluxForElem_(e).GetReference (REVIEW round-4 R-002/R-003). h_min_ is geometric (from mesh_), NOT from flux_, so the (1,1,1) sentinel does not perturb any shared quantity. No other base-ctor flux_ use exists.
 - **The tripwire is the C-6 fault-bearing parity test run with a REAL material (≠ (1,1,1)).** Any skeleton site that bare-uses flux_ (= the (1,1,1) sentinel) instead of the overridden FluxForElem_(e) produces a contribution that differs from the scalar reference (which uses the real per-element material) → C-6 fails at the offending DOFs and names them. So a missed material site becomes a hard parity failure, never silent wrong physics.
 - **Optional CI guard:** a grep that the shared skeleton (wave_operator.inl) has zero bare flux_.{Interior,AbsorbingTotal,FreeSurfaceGodunovTotal,FreeSurfaceTotal,GetCp,GetReferenceStarMatrix} on any material-dependent path (all must be FluxForElem_(e) or a virtual hook), plus git diff hrs-ref HEAD -- wave_operator.inl for residual flux_-vs- FluxForElem_ divergence (the one-pass class-closer from REVIEW round-4).
5. Add the fault-bearing parity test (§Testing C-6).

Edge Cases to Handle

- **Scalar with mixed-flux (the SAFS production case `interior_flux="scalar", mixed_flux="adjacent"`):** stays entirely on `WaveOperator`; `InteriorFaceFlux_` default body keeps the central-vs-interior per-face choice. Byte-exact.
- **Matrix + mixed-flux:** `BimaterialWaveOperator::SetMixedFluxMode(non-None)` aborts (R-003).
- **BimaterialWaveOperator with a homogeneous Coefficient material:** must reduce to the scalar Godunov flux to LU rounding at every site (interior + boundary + volume + CK + fault) — the C-6 parity test (a mesh WITH a fault).
- **Serial Mesh (no shared faces):** `ExchangeBiMaterialNeighbours_` is a no-op (as today); the `MPI_Comm_rank` warning path is `#ifdef MFEM_USE_MPI` and only on `n_shared>0`.
- **UsesGodunovFluxPool() callers:** returns false from `WaveOperator`, true from `BimaterialWaveOperator` (virtual) — check the driver/tests still read it through the base pointer.

Acceptance Criteria

- ☐ `tpv102_driver`, `tpv104_driver`, `tpv205_driver`, `seas_bp5_full` compile **unchanged** and their station/benchmark outputs are byte-identical to a pre-Phase-13 checkpoint.
- ☐ Scalar `spatial_dyn_driver` runs (the 8 SAFS configs + the scalar TPV TOMLs) are byte-identical; `dt_cfl` unchanged (TPV102 0.00066147).
- ☐ `grep -nE "owned_flux_pool_|per_face_bimaterial_|GodunovFluxPool|BimaterialFlux|FluxForElem_.*p dynamic/wave_operator.{hpp,inl}"` returns **zero** (scalar class is bimaterial-free).
- ☐ `test_phaseh_wave_operator_constant_parity` passes (repointed to `BimaterialWaveOperator`), `np=1` and `np=4`.
- ☐ **C-6 (new):** `BimaterialWaveOperator` with a homogeneous Coefficient material on a **fault-bearing** mesh matches the scalar `WaveOperator` Mult/ADER to 1e-9 — this exercises the fault imposed-state flux (R-001), the boundary flux, the volume RHS, interior faces, and CK.
- ☐ `seas_test_godunov_flux_bimaterial` (21/21) and `seas_test_wave_operator` still pass.
- ☐ A NaN-poisoned `flux_in` in `BimaterialWaveOperator` does NOT produce NaN in any Mult/ADER output on the homogeneous-Coefficient fixture (proves no stray bare-`flux_` path survives).

Dependencies

- Depends on: Phase 9 (the bimaterial machinery being moved). Required by: Phase 10 (TPV31 now constructs `BimaterialWaveOperator`), and any future parallel-heterogeneous run.
- **Note:** the cross-rank `ExchangeBiMaterialNeighbours_` local-side stub (REVIEW round-3 R-003 / round-4 informational) is MOVED, not fixed, by Phase 13; the real `MPI_Allgatherv` exchange remains a separate deferred item (lateral-het `np>1` only; TPV31 is depth-only/seam-continuous).

Testing Strategy (Phase 13)

- **Byte-exact scalar gate (every phase):** run the TPV/BP5 unit + dry-run suite and diff against a pre-Phase-13 tag; this is the primary guard that `WaveOperator` is untouched behaviorally.
- **13.1a:** the existing C-1..C-5 constant-parity test must pass bit-for-bit (pure refactor).
- **13.1b:** repoint C-5 to `BimaterialWaveOperator`; re-run `np=1` + `np=4`.
- **13.2 / C-6:** the fault-bearing homogeneous-Coefficient parity test — the test the round-4 review flagged as missing; it is the one that catches the whole `flux_`-placeholder leak class

(fault + boundary + volume) in one shot. Build a small split-box mesh with one interior fault face, seed a fault-imposed state, and assert BimaterialWaveOperator (homogeneous Coefficient) == WaveOperator (scalar) Mult/ADER to 1e-9.

Risk Assessment (Phase 13)

Risk	Severity	Mitigation
NaN-poison corrupts shared base-ctor setup → RESOLVED: NaN is infeasible (GodunovFlux ctor asserts $\mu > 0 / \rho > 0$; $\text{NaN} > 0 == \text{false}$ → aborts construction)	LOW	Use the valid (1,1,1) sentinel (passes the asserts, dead on the matrix path) + the C-6 fault-bearing real-material parity test as the tripwire (req 4). Verified: base ctor uses <code>flux_</code> only via the dead <code>Ax_/Ay_/Az_</code> ; <code>h_min_</code> is geometric
Extracting the InteriorFaceFlux_ hook from the deep assembly perturbs the scalar path	HIGH	Lift the existing else (scalar) branch verbatim into the default body; gate on the byte-exact scalar suite + C-1..C-5 after 13.1a (before any extraction in 13.1b)
Virtual FluxForElem_/ComputeMaxDt regress scalar performance	LOW	hooks are per-element/per-face (not per-QP); hrs-ref already used a non-virtual FluxForElem_ at these sites with no perf issue; measure TPV102 wall-time if concerned
A material-dependent site is missed and only shows on the matrix path	MED	The NaN-poison tripwire (req 4) + the fault-bearing C-6 parity test convert any miss from silent-wrong to a hard NaN/parity failure; also run <code>git diff hrs-ref HEAD -- wave_operator.inl grep for residual flux_.-vs-FluxForElem_ divergence</code> (the one-pass class closer from REVIEW round-4)

BimaterialWaveOperator
inherits scalar-only baggage
(flux_, Ax_, mixed-flux state)

LOW

Acceptable; flux_ poisoned,
mixed-flux disabled via the
SetMixedFluxMode override;
documented as the inheritance
trade-off vs base-extraction

11.2 Phase 14 – Explicit Runge-Kutta (RK45) time-integrator option for mixed (central) flux

Overview

Add an explicit method-of-lines **Runge-Kutta** time integrator — headline target **Dormand-Prince RK45** (and classical **RK4** as the validated stepping-stone) — as a **runtime-selectable alternative** to ADER, so that **mixed/central flux can run stably at p=1**. The companion design study document/mixed_flux_dev/BUILD_mixed_flux_rk_dynamic_rupture_2026-05-28.md (and its root-cause analysis drdg3d_mixed_flux_comparison_2026-04-28.{md,pdf}) is the authoritative source for this phase; this section turns it into the executable contract.

Why RK, why order ≥ 3 (condensed): central flux (GodunovFlux::Central, Interior — Central = $+\frac{1}{2}|A_n|(Q_{\text{self}}-Q_{\text{nbr}})$) deletes the upwind dissipation, so fault-adjacent modes become purely imaginary $\lambda=i\omega$. ADER-O2's stability function $R(z)=1+z+z^2/2$ has $|R(iy)|^2=1+y^4/4>1$ for all $y>0 \rightarrow$ unconditional growth, and at $p=1$ the Cauchy-Kovalevskaya predictor is **locked to O2** ($0 \leq p+1$; $D(k \geq 2) \equiv 0$ for a degree-1 field), so the instability is unfixable inside ADER. An explicit RK of order ≥ 3 has a stability region that **contains a segment of the imaginary axis** (RK4: $|\lambda|dt \leq 2\sqrt{2} \approx 2.83$; DP45: RK4-class), which is exactly what central flux needs. DRDG3D runs central flux + RK54 stably at CFL ≈ 0.3 .

Why the machinery already exists (assembly, not from-scratch): WaveOperator::Mult (wave_operator.inl:1185) is a complete method-of-lines RHS $dQ/dt = M^{-1}(-\text{Face} + \text{Vol})$ that already (a) does the mixed-flux dispatch if (mf_on && central_flux_face_set.count(f)) flux_.Central else flux_.Interior (ComputeFaceFluxRHS:3320, ComputeSharedFaceFluxRHS:~3425), and (b) runs the **instantaneous, ψ -stateless** fault Riemann solve fault_flux_→Evaluate(...) on fault faces (ComputeFaceFluxRHS:3066, ComputeSharedFaceFluxRHS:3736). FaultFaceFlux::Evaluate (fault_face_flux.cpp:328) writes $V1/V2/\text{slip_rate}/\tau*\text{corr}/\sigma_n\text{corr}$ but **never** $\psi/\text{slip1}/\text{slip2}$ (debug-guarded at :341/:428, comment R-V92-H07) — by design, so a coupled RK on the augmented state (Q, ψ, slip) is clean. The proven template is the pre-ADER TPV102 coupled RK4 loop at git show 8461c67:./drivers/tpv102_driver.cpp (~lines 783–965), and the bulk-only RK4 idiom is DoRK4Step in tests/unit/test_rk4_conservation.cpp:68. The Phase-9 bimaterial changes did **not** touch the Mult/fault/mixed-flux path on the scalar branch (owned_flux_pool_==nullptr).

Scope (read before implementing)

- **Rate-and-state ONLY.** The Mult-path instantaneous Evaluate is the RATE-AND-STATE solve. There is **no instantaneous LSW solve** on the Mult path — only EvaluateADER_LSW (fault_face_flux.cpp:746) exists, for the ADER path. So the RK integrator supports [meta].law="rate_state" (SAFS slip-law-SRW + aging) only; an [meta].law="slip_weakening"

(TPV205) run with `--time-integrator rk*` must **abort with a clear message** (LSW-on-RK is a documented follow-up that needs a new instantaneous EvaluateLSW). This matches the BUILD doc's SAFS/RS focus and the configuration where the $p=1$ mixed-flux benefit was measured.

- **Scalar interior flux ONLY.** Mixed flux is a scalar-path feature; `interior_flux="matrix"` already mutually-excludes `mixed_flux` (SetMixedFluxMode abort wave_operator.inl:1821, parser guard spatial_friction.cpp:~1050). The RK path uses `interior_flux="scalar"` (the default).
- **ADER is untouched.** The RK stepper is an **additive, CLI-gated** alternative path. It does NOT use IFrictionIterator, EvaluateADER, EvaluateADER_LSW, ComputeADERSubStepStates, the substep iterators, or the substep_I_imp_* side-channels. The TPV*/BP5/SAFS ADER byte-exactness contract (make test) stays green; the RK path is reachable only via `--time-integrator rk*` (default ader).

Constraints

- **No interface change to WaveOperator::Mult, FaultFaceFlux::Evaluate, or DOFData** — the RK stepper consumes them as-is. In particular Evaluate MUST remain ψ -stateless (its :428 guard is the tripwire; do not add a data.psi write).
- **ψ /slip live in DOFData, not the MFEM Vector** — therefore a stock mfem::ODESolver (e.g. mfem::DormandPrinceRK45) **cannot** drive the coupled system directly (it only integrates the Vector). The integrator is a **hand-written coupled RK loop** over (Q , ψ , slip). (Option B — repack ψ /slip into the Vector to reuse the stock adaptive solver — is explicitly out of scope; it touches the fault-flux plumbing broadly. See BUILD §5.0.)
- **Match existing conventions:** CLI parsing via GetStringArg/HasFlag; config via NumericsSpec — the spatial_friction.cpp parser + a MFEM_VERIFY validator; the friction-method default Brent (CLAUDE.md). Per-DOF $a/b/Dc/V_w$ sourced exactly as the substep iterator (tpv104_substep_iterator.cpp) / the resolver (RateStatePerDOFParams).
- **Numerical:** explicit RK on ψ is only *conditionally* stable (local timescale $\sim Dc/(b \cdot V_0 \cdot e^{\{\psi/a\}})$); for a dynamic event the wave-CFL dt is far below it (TPV102: $dt/\tau \sim 3e-3$), so it is safe — but keep the ψ tripwire and the analytic UpdateStateAnalyticSlipLawSRW as an operator-split fallback if a future stiff regime is hit (BUILD §5.4, R-V92-H06).

Phase 14.0 – Decision record (no code)

- **Schemes shipped:** classical **RK4** (validation; reuse the 8461c67/DoRK4Step idiom) and **Dormand–Prince RK45** (the user-requested integrator). Both via one tableau-driven loop.
- **Adaptive vs fixed step:** ship **fixed-step** first (DP45 5th-order weights, dt from the RK CFL of §14.4) — for a CFL-bound hyperbolic system the embedded step-size control is largely wasted. Adaptive embedded 4(5) error control on the bulk Q is an **optional** sub-phase (§14.6), off by default.
- **Adjacent is the default mixed-flux mode for RK** (central only on fault-adjacent faces, Zhang Fig 4b); AllContinuous is available at a tighter CFL.

Phase 14.1 – Integrator selector + coupled RK stepper scaffold (RK4, bulk validation)

Goal: `--time-integrator rk4` runs a coupled RK4 stepper through wave.Mult that is bit-for-bit the DoRK4Step idiom on the bulk and matches ADER-O2 to $O(dt^2)$ on a frictionless linear problem;

ADER remains the default and is byte-unchanged.

Files to Create - dynamic/rk_time_stepper.{hpp,cpp} — the RKTableau struct + tableau factories + the coupled stepper AdvanceRKCoupled_Spatial(...). - tests/unit/test_rk_time_stepper.cpp — tableau-correctness + bulk RK4-vs-ADER-O2 convergence.

Files to Modify - spatial/code/spatial_friction.hpp — add enum class TimeIntegratorKind { ADER, RK4, RK45 }; and TimeIntegratorKind time_integrator = TimeIntegratorKind::ADER; to NumericsSpec (after interior_flux, :119). - spatial/code/spatial_friction.cpp — parse [numerics].time_integrator $\in$ {"ader","rk4","rk45"} (default "ader") next to the fault_iterator/interior_flux selectors (~:1018); MFEM_ABORT on any other string. - drivers/spatial_dyn_driver.cpp — --time-integrator <s> CLI override (mirror --ader-order, :530/:623); branch the time loop (:2280–2310): ADER $\rightarrow$ AdvanceADERWithSubStep_Spatial (current, :2291); RK4/RK45 $\rightarrow$ AdvanceRKCoupled_Spatial. Reject rk* + law=="slip_weakening" and rk* + interior_flux=="matrix" at setup with a clear MFEM_VERIFY. - miniapps/seas/Makefile — RK_TIME_STEPPER_{SRC,OBJ}; link into seas_spatial_dyn_driver; add the test target + test: aggregate.

Detailed Requirements 1. struct RKTableau { int stages; std::vector<std::vector<real_t>> a; std::vector<real_t> b; std::vector<real_t> bhat; std::vector<real_t> c; bool has_embedded; const char* name; }; (explicit $\Rightarrow$ strictly-lower-triangular a). Factories: RKTableau MakeRK4Tableau(); — classical 4-stage: $c=(0, \frac{1}{2}, \frac{1}{2}, 1)$, $b=(1, 2, 2, 1)/6$, a = the standard lower-triangular $(\frac{1}{2}), (0, \frac{1}{2}), (0, 0, 1)$; has_embedded=false. RKTableau MakeDormandPrinceRK45Tableau(); — the 7-stage Dormand–Prince $c=(0, 1/5, 3/10, 4/5, 8/9, 1, 1)$, b = the 5th-order row, bhat = the 4th-order row, FSAL (verbatim coefficients from the DP(4,5) table); has_embedded=true. A ValidateTableau helper asserts $\sum b = 1$, $\sum bhat = 1$, $c_i = \sum_j a_{ij}$, and strict-lower-triangularity. 2. void AdvanceRKCoupled_Spatial(WaveOperator<ParMesh>& wave, std::vector<DOFData>& dof_data, const spatial::RateStateBlock& rs_cfg, const spatial::RateStatePerDOFParams& rs, const mfem::Vector& Q, real_t dt_step, real_t t_step_start, mfem::Vector& Q_new, const RKTableau& tab, INucleationMethod* nuc); — one macro-step. Allocates s stage-rate Vectors $k[0..stages-1]$ (size height) + per-DOF stage arrays ψ_i, sr_i . For each stage i: build $Q^{(i)} = Q + dt \cdot \sum_{j<i} a_{ij} k_j$; (§14.3) nucleation at $t_{step_start} + c_i \cdot dt$; set each $dof_data[m].\psi$ to its stage value $\psi_n[m] + dt \cdot \sum_{j<i} a_{ij} \psi_k[j][m]$; call $wave.Mult(Q^{(i)}, k[i])$ (does mixed-flux + instantaneous Evaluate at $(Q^{(i)}, \psi^{(i)})$, writes $dof_data[m].slip_rate/V1/V2$); capture $sr_k[i][m]=dof_data[m].slip_rate$ and $\psi_k[i][m]=PsiRate(rs_cfg, dof_data[m], sr_k[i][m], rs, m)$ (§14.2). **Final combine** with b: $Q_new = Q + dt \cdot \sum_i b_i k[i]$; $dof_data[m].\psi = \psi_n[m] + dt \cdot \sum_i b_i \psi_k[i][m]$; $dof_data[m].slip1 += dt \cdot \sum_i b_i (V1$ at stage i), slip2 likewise (RK-consistent $dslip/dt = V$; capture per-stage $V1/V2$ alongside sr_k). **14.1 stub:** with no RS coupling yet, integrate Q only ($\psi_k \equiv 0$) so the scaffold validates against the bulk DoRK4Step. 3. Driver time-loop branch is the ONLY change to the stepping control; $Q \leftarrow Q_new$ swap and $t += dt$ are shared with the ADER branch.

Interfaces: the stepper takes WaveOperator& (calls Mult), DOFData (reads slip_rate/V*, writes psi/slip*), and RKTableau; it returns nothing (writes Q_new + dof_data).

Edge cases: tab.stages mismatch with allocated arrays $\rightarrow$ MFEM_VERIFY; $dt_step \leq 0 \rightarrow$ abort;

law != rate_state reaching the stepper → abort (guarded earlier); a stage Q^i with NaN (blown up) → the existing Mult/Evaluate guards fire.

Acceptance Criteria - [] --time-integrator parses (ader/rk4/rk45); default ader; unknown string aborts. - [] test_rk_time_stepper: MakeRK4Tableau/MakeDormandPrinceRK45Tableau pass ValidateTableau; on a frictionless linear bulk problem RK4 matches DoRK4Step **bit-for-bit** and matches ADER-O2 to $O(dt^2)$ (energy drift bounded — the test_rk4_conservation.cpp pattern). - [] ADER path byte-unchanged: make test green; a scalar TPV/SAFS --dry-run with the default (ader) is bit-identical to pre-Phase-14 (dt + dispatch banner unchanged).

Dependencies: Depends on Phase 9 (mixed-flux dispatch + Mult path) + Phase 3 (RS resolver). Required by 14.2–14.6.

Phase 14.2 – Couple the rate-and-state fault state into the RK stages

Goal: the RK stepper integrates ψ and slip together with Q (same tableau weights), so an rk4/rk45 SAFS run with mixed_flux=none reproduces the ADER-O2 **upwind** SAFS baseline within time-integration tolerance.

Files to Modify: dynamic/rk_time_stepper.cpp (implement PsiRate); tests/unit/test_rk_time_stepper.cpp.

Detailed Requirements 1. `real_t PsiRate(const spatial::RateStateBlock& rs_cfg, const DOFData& d, real_t V, const spatial::RateStatePerDOFParams& rs, int m);` dispatches on `rs_cfg.state_evolution`: - AgingLaw → `AgingLawPsi::Rate(V, d.psi, d.Dc) = 1 - V·d.psi/d.Dc` (friction/state_evolution.hpp:190). - SlipLawStrongRateWeakening → `SlipLawSRWPsi::Rate_SRW(V, d.psi, d.Dc, rs.V_w(m), rs.a(m))` (friction/slip_law_srw_psi.hpp:260), with the law built in production mode (`SetProductionMode()`, R-001) and per-QP $V_w(m)/a(m)$ from the resolved `rs` (same source as `tpv104_substep_iterator.cpp:864`). The bare `Rate(...)` MUST NOT be used (it aborts in production). 2. Capture per-stage $V1/V2$ (not just $|V|$) for the slip accumulation; slip uses the SAME b weights as Q/ψ . 3. The stage- ψ write-then-Mult ordering of §14.1 req 2 is mandatory: Evaluate reads `dof_data[m].psi` as the stage-local ψ^i and must see the correct staged value.

Edge cases: $d.Dc \leq 0 \rightarrow$ abort (resolver already guards); V huge (event) — Rate is finite; ψ -rate stiffness — see §14.4 caveat (tripwire + analytic fallback noted, not implemented here).

Acceptance Criteria - [] With `interior_flux=scalar, mixed_flux=none, --time-integrator rk4`, the SAFS slip-law-SRW run reproduces the ADER-O2 upwind baseline (BUILD §6.2: job 7751974, $V_{\max} \approx 10.5$ m/s) within integration-order tolerance (document the tolerance). - [] Aging-law (TPV102-type) RS config: RK ψ -evolution matches a fine-dt ADER reference to $O(dt^q)$.

Dependencies: Depends on 14.1. Required by 14.3–14.6.

Phase 14.3 – Nucleation at stage times (absolute form)

Goal: nucleation forcing is applied as an **absolute** overstress at each RK stage time, not the ADER telescoped per-substep increment (which would **double-apply** across RK stages that revisit a sub-interval, e.g. RK4 stages 2 and 3 both at $t+dt/2$).

Files to Modify: dynamic/nucleation_method.hpp (+ the concretes in nucleation_factory.cpp), dynamic/spatial_nucleation.{hpp,cpp}, dynamic/rk_time_stepper.cpp.

Detailed Requirements 1. Add virtual void `INucleationMethod::ApplyAbsolute(std::vector<DOFData>& dof, real_t t) = 0`; that **sets** (not accumulates) `dof[i].tau1_nuc/tau2_nuc = SmoothStep(t, T_nuc)·F(r_i)·Δτ{dip/strike}` — the absolute SCEC ramp at time `t`. Implement for `GaussianGradualOverstress`, `CompactCircularGradualOverstress` (write the strike channel); `StaticOverstress` and `InstantaneousOverstress` set their (time-independent / `ApplyOnce`) value and ignore `t`. Back it with free fns `ApplyGradualOverstressAbsolute` / `ApplyGradualOverstressCompactCircularAbsolute(...)` (mirror the existing `*Increment` fns, `spatial_nucleation.cpp:135`, but absolute `SmoothStep(t)` instead of `SmoothStep(t)−SmoothStep(t−dt)`). 2. The RK stepper calls `nuc→ApplyAbsolute(dof_data, t_step_start + c_i·dt)` at the start of each stage (§14.1 req 2). The ADER path keeps using `ApplyIncrement` via `nuc_cb` — **unchanged**.

Edge cases: $t \leq 0 \rightarrow \text{SmoothStep} = 0$ (no forcing); $t \geq T_{\text{nuc}} \rightarrow \text{SmoothStep} = 1$ (full, constant — the absolute form is a no-op-equivalent steady value, correct under RK); `ApplyOnce` kinds must not re-seed per stage (idempotent absolute value).

Acceptance Criteria - [] `ApplyAbsolute` at $t=0 \rightarrow 0$, $t=T_{\text{nuc}} \rightarrow$ full target, monotone in between (unit test on `tau*_nuc`). - [] The RK-stage-summed effective forcing over $[0, T_{\text{nuc}}]$ matches the ADER telescoped total to round-off (no double-apply).

Dependencies: Depends on 14.1, Phase 7 (`INucleationMethod`). Required by 14.5.

Phase 14.4 – Dormand-Prince RK45 + RK CFL recalibration

Goal: `--time-integrator rk45` runs the fixed-step DP45 5th-order stepper, with `dt` set by an **RK** stability bound (not the ADER $1/(3(2N+1))$ de-rating).

Files to Modify: `dynamic/rk_time_stepper.cpp` (DP45 already added in 14.1 req 1 — here it is selected + validated); `drivers/spatial_dyn_driver.cpp` (the `dt` computation, :1464); `dynamic/wave_operator.inl` (`ComputeMaxDt:6079` — make the CFL factor RK-aware, see below).

Detailed Requirements (the math is load-bearing — see BUILD §5.4) 1. Current `dt = cfl_mixed_flux_factor · cfl · h / c_p` (`ComputeMaxDt:6079`, $c_p = \sqrt{((\lambda+2\mu)/\rho)}$, `h`=inscribed diameter; `cfl_mixed_flux_factor` $\in \{\text{None } 1.0, \text{Adjacent } 0.9, \text{AllContinuous } 0.4\}$, flagged “interim placeholder”, :6095) — then the **driver** multiplies by `CflSafetyFactor(cfg)` which for ADER is the $1/(3(2N+1))$ TPV205-derived de-rating (`spatial_dyn_driver.cpp:1464` $\rightarrow$ `spatial_friction.hpp:CflSafetyFactor`). 2. For the RK path, replace the ADER $1/(3(2N+1))$ de-rating with the RK imaginary-axis stability bound $\max|\lambda| \cdot dt < y_{\text{max}}$ ($y_{\text{max}} \approx 2.83$ RK4, RK4-class DP45). Concretely: add an integrator-aware factor (a new `RkCflFactor(cfg)` branch or a `time_integrator` arm in `CflSafetyFactor`) so the ADER branch is **bit-unchanged** and the RK branch uses the DRDG3D-anchored empirical mixed-flux target **CFL** ≈ 0.3 for Adjacent (and the tighter $\approx 0.3\text{--}0.4$ for AllContinuous). Re-tune `cfl_mixed_flux_factor` against the RK stability region (document the chosen numbers + their derivation in-source, replacing the “interim placeholder” note). 3. Selecting `rk45` uses `MakeDormandPrinceRK45Tableau()` + the 5th-order b row.

Edge cases: `interior_flux=matrix` (forbidden with RK+mixed) — already guarded; `cfl` outside $(0, 1)$ — parser guard; an `h/c_p` that makes `nsteps` enormous — log it (existing `[time] nsteps`).

Acceptance Criteria - [] `rk45` selected $\rightarrow$ DP45 tableau used (`tab.name=="DormandPrinceRK45"`, 7 stages, $\Sigma b=1$). - [] On a frictionless linear problem DP45 shows ≥ 4 th-order temporal convergence.

- [] The ADER dt is bit-unchanged when `--time-integrator ader` (the `CflSafetyFactor` branch for ADER is untouched).

Dependencies: Depends on 14.1. Required by 14.5.

Phase 14.5 – Enable mixed flux on the RK path + central-flux stability

Goal: the central-flux SAFS configuration that **ran away under ADER** (BUILD §6.3, job 7751975) runs to `t_final` with bounded `V_max` under `rk45 + mixed_flux=adjacent`, and reproduces the `p=1` mixed-flux accuracy benefit.

Files to Modify: `drivers/spatial_dyn_driver.cpp` (it already calls `wave.SetMixedFluxMode(ParseMixedFlux(cfg):1064` — no change needed beyond confirming it runs on the RK branch; the per-stage `Mult` honors `mf_on_` automatically). Define the RK-stage analogue of the ADER-specific `slip_rate_substep_max` diagnostic (`spatial_dyn_driver.cpp`: ~2230) — per-stage max $|V|$, or the final-stage value.

Detailed Requirements 1. No flux-code change: `wave.SetMixedFluxMode(MixedFluxMode::Adjacent)` (from `mixed_flux="adjacent"`) makes every per-stage `Mult` use central flux on `central_flux_face_set_`; the RK stability region (§14.4) keeps the imaginary fault-adjacent modes bounded. 2. Replace the substep `slip_rate_substep_max` reset/peak (ADER-substep-specific) with an RK-stage `max-|V|` reduction on the RK branch (ADER branch unchanged).

Edge cases: `mixed_flux="all_continuous"` — allowed, needs the tighter CFL (§14.4); MPI: the shared-fault `Mult` path uses `Evaluate` directly (:3736), so the ADER R-1600/R-1601 frame workaround does **not** transfer — multi-rank fault correctness on the RK path **MUST** be tested (below).

Acceptance Criteria - [] `rk45 + mixed_flux=adjacent` on the previously-divergent central-flux SAFS config runs to `t_final` with bounded `V_max` (no run-away). - [] `np>1`: a fault crossing a rank seam is conservation-consistent across the seam on the RK path (the `Mult Evaluate` shared-fault path), verified explicitly (not inherited from ADER). - [] The measured `p=1` mixed-flux accuracy improvement is reproduced, now stable.

Dependencies: Depends on 14.2, 14.3, 14.4.

Phase 14.6 (optional) – Adaptive step-size control (embedded DP 4(5) on the bulk)

Goal: optional `--rk-adaptive` that uses the DP45 embedded `bhat` (4th-order) estimate on the bulk `Q` to control `dt`; ψ /`slip` advance with the accepted 5th-order step. **Note:** off by default — for a CFL-bound hyperbolic system the adaptivity is largely wasted (BUILD §5.0/Phase 0). Error norm on `Q` only (the wave field is CFL-limiting); standard PI controller. **Acceptance:** with a loose tolerance the controller reproduces the fixed-step `rk45` result; not on the default path. **Dependencies:** Depends on 14.4; independent of 14.5.

Testing strategy (Phase 14)

1. **Tableau correctness (14.1):** `ValidateTableau` on RK4 + DP45 ($\Sigma b=1$, $\Sigma bhat=1$, $c=\Sigma a$, strict-lower-triangular).
2. **Bulk linear conservation/convergence (14.1/14.4):** $RK4 \equiv DoRK4Step$ bit-for-bit; RK4 vs ADER-O2 $O(dt^2)$; $DP45 \geq 4th\text{-order}$ — the `test_rk4_conservation.cpp` harness.
3. **Upwind RS equivalence (14.2):** `rk* + mixed=none` reproduces the ADER-O2 upwind SAFS baseline.

4. **Nucleation total (14.3):** absolute stage-sum == ADER telescoped total to round-off.
5. **Central-flux stability (14.5):** the ADER-divergent config is bounded under RK.
6. **MPI fault (14.5):** multi-rank seam-crossing fault conservation on the RK path.
7. **ADER byte-exact contract (all):** make test green; default ader runs bit-identical.

File-touch summary (Phase 14)

- **Create:** `dynamic/rk_time_stepper.{hpp,cpp}`, `tests/unit/test_rk_time_stepper.cpp`.
- **Modify:** `spatial/code/spatial_friction.{hpp,cpp}` (the `TimeIntegratorKind` selector), `drivers/spatial_dyn_driver.cpp` (CLI + time-loop branch + `dt` + RK diagnostic + the RS-only/scalar-only guards), `dynamic/wave_operator.inl` (`ComputeMaxDt` RK-aware factor only — no flux change), `dynamic/nucleation_method.hpp` + `nucleation_factory.cpp` + `dynamic/spatial_nucleation.{hpp,cpp}` (the absolute `ApplyAbsolute`), `miniapps/seas/Makefile`.
- **Untouched (must stay byte-exact):** all `*_substep_iterator.*`, `EvaluateADER*`, `ComputeADERSubStepStates`, `AdvanceADERWithSubStep_Spatial`, the `IFrictionIterator` family, and the standalone `tpv*/bp5` drivers.

Risk assessment (Phase 14)

Risk	Severity	Mitigation
Explicit RK on ψ hits a stiff regime (low ψ / high V on the full SAFS envelope) $\rightarrow \psi$ instability	MED	Wave-CFL $dt \ll \psi$ timescale for dynamic events (TPV102 $dt/\tau \sim 3e-3$); keep the 8461c67 ψ tripwire; keep analytic <code>UpdateStateAnalyticSlipLawSRW</code> as an operator-split fallback for flagged stiff QPs (do NOT remove it)
RK CFL factors mis-tuned $\rightarrow$ either unstable (too big) or wasteful (too small)	MED	Anchor to DRDG3D's empirical $CFL \approx 0.3$ (Adjacent); validate on the §14.5 central-flux config; document the RK stability bound $y_{\max}$ in-source
Per-stage Mult does N face-neighbour MPI exchanges/step (vs ADER predictor-once)	LOW	Profile TPV102/SAFS wall-time; accept the extra exchanges for the central-flux capability; it is the price of method-of-lines
A <code>data.psi</code> write sneaks into <code>Evaluate</code> and double-integrates ψ	LOW	The :428 <code>MFEM_ASSERT(data.psi==psi_at_entry)</code> tripwire already guards it; the RK stepper relies on it
LSW user accidentally runs <code>--time-integrator rk*</code>	LOW	Setup <code>MFEM_VERIFY</code> aborts (RS-only scope) with a clear message

Risk	Severity	Mitigation
matrix + RK + mixed-flux	LOW	Already mutually-excluded (parser + SetMixedFluxMode guards); add a setup MFEM_VERIFY for the explicit combination

Dependencies

Depends on: Phase 3 (RS resolver / RateStatePerD0FParams), Phase 7 (INucleationMethod), Phase 9 (Mult-path mixed-flux dispatch). Required by: nothing (additive, CLI-gated). Independent of Phases 10–13 (TPV31 / depth-profile / PML / class-split) — but if Phase 13 lands first, the RK stepper drives the **scalar** WaveOperator class unchanged (it never needs the bimaterial operator).

12 Appendix

- **PDF build:** ./document/parts_dev/_assets/md2pdf.sh document/fullelasticity_dev/PLAN_tpv_regress (Charter/Avenir/Menlo, TOC, numbered sections; Unicode math via pdf-style.tex).
- **Reference clone:** /Users/chunhuizhao/projects/seas-mfem-heterogenous-riemann-solver (feature/heterogeneous_riemann_solver, e30d003); fetched here as tag hrs-ref; merge-base 0e13b25.
- **Files ported verbatim from hrs-ref (Phases 9–10):** dynamic/godunov_flux_bimaterial.{hpp, cpp} (BimaterialFlux), the owned_flux_pool_dispatch in wave_operator.inl (Mult ~:1311 / ADER ~:5191) + BuildPerFaceBimaterialFluxMatrices_ + ExchangeBiMaterialNeighbours_, MakeDepthProfile1DMaterial in heterogeneous_material.*, and the tpv31/ tree (config/mesh/benchmark/tests). Reproduce diffs with git diff 0e13b25 hrs-ref -- miniapps/seas/dynamic/godunov_flux_bimaterial.hpp etc.
- **Reference RS-for-SAFS plan:** safs/project_7.0_alternative/document/PLAN_rate_state_friction_opti (used as a reference; identifier corrections noted in Phases 1/3/6).
- **Analysis archives:** /tmp/branchcmp/agent{1..5}*.md, /tmp/branchcmp/pass2*.md.